

Università degli Studi di Verona
Dipartimento di Informatica
Corso di Laurea in Informatica / Bioinformatica

PROGRAMMAZIONE II

COMPENDIO GENERALE TEORICO E PRATICO

Guida Completa alle Lezioni (Slide T01 – T20), Analisi Minuziosa del Codice,
Trappole d'Esame e Best Practice di Progettazione ad Oggetti

DOCENTE DEL CORSO

Prof. Michele Pasqua

Dipartimento di Informatica
michele.pasqua@univr.it

REVISIONE E APPUNTI DI STUDIO

Wally

Corso di Programmazione II
Anno Accademico 2025/2026

Settembre 2026 – Edizione Integrale Unificata

Compilato con XeLaTeX – 19 Moduli Didattici – Inclusione Totale di Codice, Teoria ed Esercizi

Prefazione e Metodologia di Studio

Questo volume nasce dall'esigenza di condensare, armonizzare e sistematizzare in un'unica opera organica l'intero patrimonio didattico dell'insegnamento di **Programmazione II** (A.A. 2025/2026, tenuto dal **Prof. Michele Pasqua** presso l'Università degli Studi di Verona).

Il corso si articola lungo tre direttrici formative sinergiche:

1. **I Fondamenti e la Teoria dei Linguaggi ad Oggetti:** dalla scomposizione in classi e incapsulamento, alla semantica formale di memoria (*Call Stack*, *Heap*, garbage collection), fino al polimorfismo dinamico (*dynamic binding*), all'astrazione mediante interfacce, ai tipi generici con polimorfismo parametrico e alle funzionalità dichiarative introdotte da Java 8+ (*Functional Interfaces*, lambda e *Streams API*).
2. **L'Ingegneria del Software Incrementale:** il corso non si limita ad illustrare frammenti di codice isolati, ma costruisce un percorso evolutivo concreto. Partendo da una prima classe rudimentale (*SimpleDate*) con campi pubblici e logica procedurale, il codice viene via via rifattorizzato attraverso 19 passaggi chiave fino alla complessa architettura modulare di *MavenDate*, gestita con Apache Maven e conforme ai principi SOLID e ai classici Design Pattern (*Singleton*, *Builder*, *Decorator*).
3. **Robustezza, Testing e Qualità del Codice:** la gestione professionale delle eccezioni (*checked* vs *unchecked*, clausole di salvaguardia, *try-with-resources*), la documentazione formale secondo lo standard *Javadoc* e la verifica rigorosa della correttezza mediante unit test automatizzati con *JUnit 5*.

Convenzioni Tipografiche e Struttura dei Capitoli

Ogni capitolo del compendio corrisponde a un blocco didattico unitario (**da Slide T01 a T20**) ed è suddiviso in tre macro-sezioni:

- **Concetti Teorici:** sintesi completa e minuziosa di tutte le nozioni esposte nelle slide del docente, con schemi UML e puntualizzazioni semantiche.
- **Analisi del Codice:** disamina riga per riga dei sorgenti ufficiali del docente, evidenziando il razionale di ciascuna scelta architetturale e il confronto *diff* rispetto alla versione precedente.
- **Verifica Pratica & Output:** riscontro operativo da riga di comando tramite `mvn test` e `mvn exec:java`, con decodifica del comportamento della JVM.

Nel testo sono stati inseriti tre ambienti grafici dedicati per supportare lo studio mirato:

Nota di Studio

Approfondimento teorico, formalizzazione del modello di esecuzione Java o regola architetturale inderogabile.

Suggerimento / Best Practice

Consiglio pratico di scrittura del codice, convenzione di stile o pattern per ottimizzare leggibilità e manutenibilità.

Attenzione / Trappola d'Esame

Caso limite insidioso, tipico errore di compilazione o comportamento runtime inatteso frequentemente oggetto di verifica d'esame.

Indice Generale

Prefazione e Metodologia di Studio	i
1 T01 — Course Introduction	1
1.1 Teoria: Concetti Teorici dalle Slide (T01 - Course Introduction.pdf)	1
1.2 Codice: Analisi del Codice della Lezione 1	2
1.3 Setup: Configurazione Operativa dell'Ambiente	2
2 T02 — Paradigma ad Oggetti e Modellazione	4
2.1 Teoria: Concetti Teorici dalle Slide	4
2.1.1 La Tassonomia dei Linguaggi e il Bisogno di Astrazione (Slide 1–13)	4
2.1.2 La Crisi del Procedurale e la Nascita degli Oggetti (Slide 14–23)	5
2.1.3 Modellazione UML e Concetti Fondamentali di OOP (Slide 24–36)	5
2.2 Codice: Esempio Comparativo: Dal Vettore C all'Oggetto Java	6
2.2.1 Il Problema in C (Codice procedurale insicuro)	7
2.2.2 La Soluzione ad Oggetti in Java (Incapsulamento, Information Hiding, Costruttore)	7
3 T03 — Compilazione, Esecuzione e Deploye	9
3.1 Teoria: Concetti Teorici dalle Slide	9
3.1.1 Il Linguaggio Java e le sue Proprietà Fondamentali (Slide 1–11)	9
3.1.2 Ecosistema Java: JDK vs JRE (Slide 11)	10
3.1.3 Compilazione, Esecuzione e Dynamic Class Loading (Slide 12–16)	10
3.1.4 Deployment e Gestione dei File JAR (Slide 17–18)	11
3.1.5 Strumenti di Sviluppo: Da Editor a IntelliJ IDEA (Slide 19–26)	12
3.2 Codice: Collegamento Pratico con il Tuo Progetto: Maven e guida-jar.md	12
3.3 [Test] Verifica dei Comandi da Terminale	13
4 T04 — Sistema di Tipi, Primitive e Refere	14
4.1 Teoria: Concetti Teorici dalle Slide	14
4.1.1 Tipizzazione dei Linguaggi e il "Contratto" di Tipo (Slide 1–7)	14
4.1.2 I Tipi Primitivi in Java e le Conversioni (Slide 8–13)	14
4.1.3 Modello di Memoria, Puntatori e Garbage Collection (Slide 14–21)	15
4.2 Codice: Analisi Dettagliata del Codice: Example.java	16
4.2.1 [Approfondimento] Le Finezze Implementative Spiegate Riga per Riga:	17
4.3 [Test] Output Esatto di Esecuzione	18
5 T05-T06 — Sintassi Java, Strutture di Control	20
5.1 Teoria: Concetti Teorici dalle Slide	20
5.1.1 Sintassi di Base e Costrutti Semplici (Slide T05, 1–19)	20
5.1.2 Costrutti di Controllo di Flusso (Slide T05, 20–29)	21
5.1.3 Classi, Oggetti, this e toString (Slide T06, 1–28)	22
5.2 Codice: Analisi del Codice Ufficiale del Docente	23
5.2.1 SimpleDate/src/Date.java	23
5.2.2 [Approfondimento] Dettagli Tecnici e Scelte del Docente:	24

5.2.3	SimpleDate/src/MainDate.java	24
5.2.4	StringPlayground.java	25
5.2.5	[Approfondimento] Le Finezze di StringPlayground:	25
5.3	□ Confronto con il tuo modulo es01	26
6	T07 — Librerie Standard e Manipolazione d	27
6.1	Teoria: Concetti Teorici dalle Slide (Slide T07)	27
6.1.1	Librerie Standard e Meccanismo di Import (Slide 1–9)	27
6.1.2	La Classe String e le sue Operazioni (Slide 10–26)	28
6.1.3	Stringhe Mutabili con StringBuilder (Slide 27–29)	28
6.2	Codice: Evoluzione del Codice: SimpleDate (Lezione 07)	29
6.2.1	[Approfondimento] Il Diff Concettuale di Date.java:	29
6.2.2	[Idea] Le Novità e le Finezze Implementative da Notare:	30
6.3	□ Corrispondenza con la Struttura del tuo Workspace	31
7	T08 — Costruttori, Inizializzazione ed In	32
7.1	Teoria: Concetti Teorici dalle Slide (Slide T08)	32
7.1.1	Il Ciclo di Vita e i Meccanismi dei Costruttori (Slide 1–12)	32
7.1.2	Incapsulamento, Information Hiding e Modificatori di Accesso (Slide 13–24)	33
7.1.3	Asserzioni e Testing (Slide 25–26)	34
7.2	Codice: Evoluzione del Codice: Lezione08	34
7.2.1	Introduzione del Tipo Enumerato Language.java	34
7.2.2	Refactoring di Date.java	34
7.2.3	[Approfondimento] Dettagli Tecnici e Trabocchetti nel Codice di Date:	35
7.2.4	Novità in MainDate.java: Array di Oggetti ed Enhanced For	35
7.3	□ Corrispondenza con i Tuoi Moduli Workspace	36
8	T09 — Array, Matrici e Tipi Enumerativi (.	37
8.1	Teoria: Concetti Teorici dalle Slide (Slide T09)	37
8.1.1	Gli Array in Java (Slide 1–13)	37
8.1.2	Tipi Enumerati (enum) (Slide 14–19)	38
8.2	Codice: Evoluzione del Codice: SimpleDate (Lezione 09)	39
8.2.1	Immutabilità di Date.java	39
8.2.2	Il Design Pattern Singleton: BirthDay.java	39
8.2.3	[Approfondimento] Analisi Architetture del Singleton:	40
8.2.4	Verifica in MainDate.java e il Bug dei Getter	40
8.3	□ Corrispondenza con i Tuoi Moduli Workspace	40
9	T10 — Variabili, Scope, Call Stack e Memo	42
9.1	Teoria: Concetti Teorici dalle Slide (Slide T10)	42
9.1.1	Il Modello di Memoria della JVM: Stack vs Heap (Slide 1–11)	42
9.1.2	Immutabilità e Passaggio Parametri: Pass-by-Value (Slide 15–25)	42
9.1.3	Classi Wrapper, Autoboxing e i loro Trabocchetti (Slide 26–35)	43
9.2	Codice: Evoluzione del Codice: Il Passaggio a MavenDate	44
9.2.1	Riorganizzazione dei Package	44
9.2.2	Il Test Didattico sulla Collisione dei Nomi di Package	44
9.2.3	Anatomia del pom.xml di MavenDate	44
9.3	□ Corrispondenza con i Tuoi Moduli Workspace	45
10	T11 — Package, Moduli e Regole di Visibil	47
10.1	Teoria: Concetti Teorici dalle Slide (Slide T11)	47
10.1.1	Modularità, Gerarchia e Convenzioni dei Package (Slide 1–11)	47
10.1.2	Visibilità di Classe e Matrice di Accesso dei Membri (Slide 12–19)	48

Indice Generale

10.2 Codice: Evoluzione del Codice: Lezione11/MavenDate	48
10.2.1 Generalizzazione in Date.java	48
10.2.2 [Approfondimento] I 5 Passi Fondamentali dell'Algoritmo equals(Object):	49
10.2.3 Le Nuove Sottoclassi: ItalianDate.java e AmericanDate.java	49
10.2.4 Polimorfismo, Upcasting e Downcasting in MainDate.java	50
10.3 □ Corrispondenza con i Tuoi Moduli Workspace	50
11 T12 — Ereditarietà, Dynamic Binding e Pol	52
11.1 Teoria: Concetti Teorici dalle Slide (Slide T12)	52
11.1.1 Ereditarietà di Classe (extends) e Riuso (Slide 1–17)	52
11.1.2 La Classe Radice Universale: java.lang.Object (Slide 18–31)	53
11.1.3 Polimorfismo e Dynamic Method Dispatch (Late Binding) (Slide 32–50)	53
11.2 Codice: Evoluzione del Codice: Lezione12/MavenDate	54
11.2.1 L'Interfaccia Comparable in Date.java	55
11.2.2 L'Astrazione con Classe Astratta: FormattedDate.java	55
11.2.3 Ereditarietà Multipla di Tipo: TimeStamp.java	56
11.2.4 Dimostrazione Pratica in MainDate.java	56
12 T13 — Classi Astratte ed Interfacce	58
12.1 Teoria: Concetti Teorici dalle Slide (Slide T13)	58
12.1.1 Classi Astratte e Template Method Pattern (Slide 1–10)	58
12.1.2 Le Interfacce in Java (interface) (Slide 11–17)	59
12.1.3 Confronto Sistemático: Classe Astratta vs Interfaccia	59
12.1.4 Interfacce Standard Fondamentali della JDK (Slide 18–22)	59
12.2 Codice: Evoluzione del Codice: Lezione13/MavenDate	60
12.2.1 Il Design Pattern Builder con Classe Statica Annidata (Date.java)	60
12.2.2 L'Interfaccia Funzionale: FormattedDateConverter.java	60
12.2.3 Sintesi Pratica in MainDate.java	61
12.3 [Test] Output di Esecuzione Verificato con Maven	61
13 T14 — Classi Annidate, Interne, Locali ed	63
13.1 Analisi Teorica Approfondita (Slide T14)	63
13.1.1 Static Nested Classes (static class Nested)	63
13.1.2 Member Inner Classes (class Inner non-statica)	64
13.1.3 Local Classes (Classi Locali a un Metodo)	65
13.1.4 Il Meme della Slide 10: <i>Objects inside objects</i>	65
13.1.5 Motivazioni Architeturali & Il Caso di Studio <i>Iterator Pattern</i> (Slide 11-12)	66
13.1.6 Anonymous Classes (Classi Anonime, Slide 13-16)	66
13.1.7 Lambda Expressions (Slide 17-19)	67
13.1.8 Varargs (type... args, Slide 20-21)	67
13.2 Analisi Dettagliata del Codice (Lezione14/MavenDate)	68
13.2.1 Diff di Date.java (Lezione 13 vs Lezione 14)	68
13.2.2 La Nuova Classe Generica Pair.java	68
13.2.3 La Classe Generica con Bounded Type OrderedPair.java	69
13.2.4 Le Sottoclassi Specializzate DatePair.java e DateInterval.java	69
13.2.5 MainDate.java	70
13.3 Risultato di Compilazione ed Esecuzione Reale	71
13.3.1 Dettaglio riga per riga dell'output:	71
14 T15 — Tipi Generici, Wildcard e Type Eras	72
14.1 Analisi Teorica Approfondita (Slide T15)	72
14.1.1 L'Idea Guida & La Tazza Cup<T> (Slide 2-3)	72

Indice Generale

14.1.2	Polimorfismo Parametrico vs Approccio Basato su Object (Slide 4-7)	72
14.1.3	Classi e Metodi Generici (Slide 8-10)	73
14.1.4	Interfacce della Libreria Standard Riscritte con i Generics (Slide 11-13)	73
14.1.5	Il Diamond Operator <> (Slide 14)	74
14.1.6	Bounded Type Parameters: Vincolare i Parametri di Tipo (Slide 15-17)	74
14.1.7	Ereditarietà e Generics: La Trappola dell'Invarianza (Slide 18-20)	75
14.1.8	Wildcards: <?>, <? extends T>, <? super T> (Slide 21-23)	75
14.2	Analisi Dettagliata del Codice (Lezione15/MavenDate)	76
14.2.1	Il Contratto equals / hashCode in Date.java	76
14.2.2	List vs Set: Accettazione vs Rifiuto dei Duplicati in MainDate.java	77
14.2.3	Map<K, V> e Lambdas Registrare Dinamicamente in MainDate.java	77
14.2.4	Ordinamento: Comparable vs Comparator (Slide 11 & MainDate.java)	78
14.3	Risultato di Compilazione ed Esecuzione Reale	78
14.3.1	Decodifica puntuale dell'output:	79
15 T16	— Java Collections Framework (JCF)	80
15.1	Analisi Teorica Approfondita (Slide T16)	80
15.1.1	Architettura e Gerarchia delle Collezioni (Slide 3-4)	80
15.1.2	L'Interfaccia Radice Collection<E> (Slide 6-7)	81
15.1.3	L'Interfaccia List<E> e le sue Implementazioni (Slide 8-9)	81
15.1.4	L'Interfaccia Set<E> e le sue Implementazioni (Slide 10-16)	82
15.1.5	Mappe associative: Map<K, V> (Slide 17-19)	83
15.1.6	Iteratori e l'Errore ConcurrentModificationException (Slide 21-23)	83
15.1.7	Algoritmi di Ordinamento: Comparable vs Comparator (Slide 24-29)	83
15.2	Riscontro Pratico nel Codice (Lezione15/Lezione16)	84
16 T17	— Gestione degli Errori ed Architetture	85
16.1	Analisi Teorica Approfondita (Slide T17)	85
16.1.1	Limiti della Gestione Tradizionale degli Errori (Slide 3-6)	85
16.1.2	La Filosofia delle Eccezioni in Java: try-catch-throw (Slide 7-10)	85
16.1.3	La Gerarchia delle Eccezioni in Java (Slide 20-21)	86
16.1.4	Checked vs Unchecked Exceptions (Slide 22-24)	86
16.1.5	"Exception Dirty Tricks": Trasformare Checked in Unchecked (Slide 25-26)	87
16.1.6	Eccezioni nei Cicli (Exceptions and Loops, Slide 27-28)	87
16.2	Analisi Dettagliata del Codice (Lezione16/MavenDate)	87
16.2.1	Le Nuove Eccezioni: IllegalArgumentException.java e OrderPairException.java	87
16.2.2	Refactoring del Metodo verify() in Date.java	88
16.2.3	MainDate.java e l'Applicazione di Tutti i Pattern della Slide T17	89
16.3	Risultato di Esecuzione Reale da Terminale	90
16.3.1	Test 1: Input con data errata (35 1 2025)	90
16.3.2	Analisi dell'output:	90
17 T18	— Interfacce Funzionali, Espressioni	91
17.1	Analisi Teorica Approfondita (Slide T18)	91
17.1.1	Cos'è una Functional Interface? (Slide 3)	91
17.1.2	Le 4 Interfacce Funzionali Fondamentali (java.util.function) (Slide 4-12)	91
17.1.3	I Quattro Tipi di Method Reference (::) (Slide 13-15)	92
17.1.4	La Stream API (java.util.stream) (Slide 16-18)	92
17.1.5	Ciclo di Vita di una Pipeline Stream (Slide 18-24)	92
17.2	Analisi Dettagliata del Codice (Lezione17/MavenDate)	93
17.2.1	Analisi Riga per Riga di MainStream.java	94
17.3	Risultato di Compilazione ed Esecuzione Reale	94

17.3.1 Decodifica dell'Output:	95
18 T19 — Documentazione con Javadoc e Unit T	96
18.1 Analisi Teorica Approfondita (Slide T19)	96
18.1.1 Documentazione con Javadoc (Slide 3-11)	96
18.1.2 Unit Testing con JUnit (Slide 13-16)	97
18.1.3 Asserzioni in JUnit 5 (org.junit.jupiter.api.Assertions, Slide 17-21)	97
18.1.4 Il Ciclo di Vita del Test e le Annotazioni JUnit 5 (Slide 22)	97
18.1.5 Organizzazione del Progetto e Convenzioni Maven (Slide 23-24)	98
18.2 Analisi Dettagliata del Codice (Lezione18/MavenDate)	98
18.2.1 Configurazione in pom.xml	98
18.2.2 TestDate.java	99
18.2.3 TestItalianDate.java	100
18.3 Risultato di Esecuzione Reale da Terminale	100
19 T20 — Java I/O, Streams, File, NIO.2 e Se	102
19.1 Analisi Teorica Approfondita (Slide T20)	102
19.1.1 L'Astrazione di I/O Stream (Slide 3-4)	102
19.1.2 Il Dualismo dell'I/O in Java: Byte Streams vs Character Streams (Slide 4-9)	102
19.1.3 Gestione delle Risorse: Da try-finally a try-with-resources (Slide 7, 10-11)	103
19.1.4 Gestione di File e Risorse di Rete (URL/URI) (Slide 13-17)	104
19.1.5 Dati Strutturati: CSV e JSON (Slide 18-26)	104
19.2 Analisi Dettagliata del Codice (Lezione19/MavenDate)	105
19.2.1 Configurazione delle Dipendenze in pom.xml	105
19.2.2 Analisi Riga per Riga di MainIO.java	106
19.3 Risultato di Compilazione ed Esecuzione Reale	107
19.3.1 Ispezione dei file generati su disco:	107
19.4 Traguardo Raggiunto: Revisione Integrata del Corso Completata!	108

T01 — Course Introduction & Metodologia del Corso

Obiettivi Formativi, Modalità d'Esame al Calcolatore, Toolchain e Syllabus

Benvenuto in questa revisione completa e metodica di **Programmazione II** (Prof. Michele Pasqua, UniVR 2025/2026). Seguiremo fedelmente le regole stabilite: **nessuna omissione**, analisi minuziosa di ogni concetto teorico, confronto riga per riga del codice, evidenziazione delle finezze d'esame e stop interattivo a ogni blocco.

1.1 Teoria: Concetti Teorici dalle Slide (T01 - Course Introduction.pdf)

La slide introduttiva definisce il perimetro accademico, gli obiettivi formativi e le modalità d'esame:

1. Docente e Background Scientifico:

- **Prof. Michele Pasqua** (Dipartimento di Informatica, Cà Vignal 2, 2° piano, Stanza 24 — michele.pasqua@univr.it)
- Aree di ricerca: *Program verification and validation, Programming language design and implementation, Code protection and software security.*
- Laboratori e progetti: co-fondatore del laboratorio **UNIVERSE** (*University of Verona Software Engineering Lab*, diretto dal Prof. Mariano Ceccato) e co-fondatore dello spin-off universitario **KonTrust Srl** (tracciamento IoT su blockchain e infrastrutture anti-manomissione); referente per Verona del programma nazionale di formazione e gare su cybersecurity (*CyberChallenge.IT*).

2. Organizzazione della Didattica:

- **Modulo di Teoria (40+ ore):** Lunedì (16:30) e Martedì (13:30) in Aula A (2 ore ciascuna).
- **Modulo di Laboratorio (12+ ore):** Martedì (10:30) in Aula Delta (2 ore).
- **Materiale didattico di riferimento:**
 - Paul J. Deitel & Harvey M. Deitel, *Java – How to Program*, Pearson (11/e – 2018).
 - Claudio De Sio Cesari, *Il nuovo Java – Guida alla programmazione moderna*, Hoepli (1/e – 2020).
 - Moodle UniVR: slide, esercizi di laboratorio e soluzioni, simulazioni.

3. Modalità d'Esame e Valutazione:

- **Formato:** Prova pratica al calcolatore in **Aula Delta** (ambiente Linux/IDE).
- **Contenuto:** Risoluzione di una serie di esercizi di programmazione in linguaggio Java (sviluppo classi, gerarchie a oggetti, implementazione interfacce, algoritmi su collezioni, gestione eccezioni, test con asserzioni/JUnit).
- **Voto finale:** Somma algebrica dei punti ottenuti nei singoli esercizi.
- È prevista una **simulazione d'esame** ufficiale a fine corso.

4. Syllabus del Corso (Mappa degli argomenti da T01 a T20):

- *Object-Orientation Principles* (T02)
- *Building, Running, and Deployment* (T03)
- *Types, Classes, and Objects* (T04)

- *Java Syntax and Semantics* (T05)
- *Java Classes and Objects* (T06)
- *Libraries and String Class* (T07)
- *Class Constructors and Encapsulation* (T08)
- *Arrays and Enumerative Types* (T09)
- *Java Variables and Call Stack* (T10)
- *Class and Package Visibility* (T11)
- *Inheritance and Polymorphism* (T12)
- *Abstract Classes and Interfaces* (T13)
- *Nested and Anonymous Classes* (T14)
- *Generic Types* (T15)
- *Collections* (T16)
- *Error Handling with Exceptions* (T17)
- *Functional Interfaces and Streams* (T18)
- *Documentation and Unit Testing* (T19)
- *Java IO* (T20)

1.2 Codice: Analisi del Codice della Lezione 1

- Nella cartella ufficiale delle lezioni [Codice-20260902](#):
 - **Non sono presenti file di codice per le Lezioni 01, 02 e 03.**
 - Il primo codice sorgente ufficiale del docente compare a partire dalla **Lezione 04** con [Example.java](#) (dedicato a tipi primitivi, reference, casting e operatori), mentre il progetto a oggetti incrementale vero e proprio parte dal blocco **Lezioni 05-06** con il progetto SimpleDate (Date.java e MainDate.java).
- Nel tuo workspace [esercizi-wally-25-26](#):
 - La cartella [es01](#) è già predisposta come modulo Maven con la prima incarnazione di Date e MainDate, sincronizzata con la struttura multi-modulo orchestrata dal [pom.xml](#) di root e dallo script [esercizi.sh](#).

1.3 Setup: Configurazione Operativa dell'Ambiente

Per garantire che tutti i passaggi da qui in avanti siano riproducibili su IntelliJ IDEA e da CLI: 1. Verifica che la JDK attiva sia Java 17 o superiore: `bash java -version mvn -v 2`. Il file di root [pom.xml](#) gestisce l'aggregazione di tutti i sottomoduli. Per ogni blocco successivo, quando andremo ad analizzare le classi introdotte a lezione, verificheremo che compilino ed eseguano pulitamente sia tramite: `bash ./esercizi.sh run <numero_modulo>` sia all'interno di IntelliJ IDEA.

Nota di Studio

La **Lezione 1 / T01** esaurisce qui la parte introduttiva/organizzativa. Il prossimo blocco è la **Lezione 2 / Slide T02: *Object-Orientation and UML***, dove entreremo nel vivo con: - Storia e tassonomia dei paradigmi: *Imperativo/Procedurale*, *Dichiarativo/Funzionale*, *Object-Oriented*. - I 4 pilastri dell'OOP: *Astrazione*, *Incapsulamento*, *Ereditarietà*, *Polimorfismo*. - Concetto di *Classe* vs *Oggetto*

(stato, comportamento, identità). - Notazione UML per classi e associazioni (*Visibility markers*, cardinalità, tipi di relazione).

Dammi conferma quando sei pronto per passare a T02 (procederemo analizzando il blocco concettuale un po' alla volta)!

T02 — Paradigma ad Oggetti e Modellazione UML

Astrazione, Incapsulamento, Modularità, Gerarchia e Sintassi dei Class Diagram

In questo blocco analizziamo la transizione teorico-metodologica dal paradigma procedurale a quello ad oggetti, i concetti cardine dell'OOP e la modellazione grafica tramite **UML** ([T02 - Object-Orientations and UML.pdf](#)).

2.1 Teoria: Concetti Teorici dalle Slide

2.1.1 La Tassonomia dei Linguaggi e il Bisogno di Astrazione (Slide 1–13)

- **Nessun linguaggio universale:** citando Edsger W. Dijkstra («*Computer Science is no more about computers than Astronomy is about telescopes*»), il docente introduce il principio del **Golden Hammer** («*If all you have is a hammer, everything looks like a nail*»): non esiste un linguaggio perfetto per tutto.
- **Evoluzione storica degli obiettivi:**
 - *Anni '50/'60:* focus sull'**efficienza del codice macchina**. L'hardware costava milioni e i programmatori poco: l'obiettivo era “tenere occupata la macchina” con linguaggi vicini al silicio.
 - *Oggi:* focus sull'**efficienza dello sviluppo del software**. L'hardware costa poco e i programmatori costano molto: l'obiettivo è “tenere produttivo il programmatore”, abbattendo complessità, bug e costi di manutenzione tramite l'**astrazione**.
- **I Tre Paradigmi Fondamentali:**
 1. **Imperativo / Procedurale** (C, Pascal, Fortran, COBOL):
 - Basato sul concetto di **stato** (celle di memoria che memorizzano valori) e **comandi sequenziali** che mutano tale stato (assegnamenti).
 - Riutilizzo del codice tramite procedure/funzioni che operano su dati esterni.
 2. **Dichiarativo:**
 - *Funzionale* (Haskell, Lisp): il programma è la valutazione di un'espressione matematica pura, basata su applicazione di funzioni e ricorsione (es. `add x y = x + y`). Assenza o minimizzazione degli effetti collaterali (*side effects*).
 - *Logico* (Prolog): il programma definisce relazioni, fatti e regole di deduzione logica; l'esecuzione è una risoluzione/unificazione guidata da query (es. `playsAirGuitar(sam):- listens2Music(sam).`).
 3. **Object-Oriented (OOP)** (Smalltalk, Java, C++):
 - Unisce il nucleo imperativo (gli oggetti contengono uno stato interno) a tratti dichiarativi (gli oggetti espongono comportamenti invocabili senza conoscerne l'algoritmo interno).
 - Basato su **istanze** che comunicano tramite **passaggio di messaggi**.
- **Lo Spettro dell'Astrazione e il Trade-off di Overhead:**

Machine Code → Assembly → Procedurale → Object-Oriented → Funzionale

- Più si sale nell'astrazione:

- * **Programmer Overhead** ↘ (**minimo**): meno tempo perso a gestire registri, puntatori e memoria manuale.
- * **Compiler/Runtime Overhead** ↗ (**massimo**): il compilatore/JVM deve effettuare controlli di tipo a runtime, garbage collection e risoluzione dinamica delle chiamate (*dynamic dispatch*).
- **Linea temporale**: dal 1949 (Assembly), Fortran (1957), C (1972), C++ (1983) fino all'avvento di **Java (1995)**, C# (2000) e linguaggi moderni multiparadigma (Swift 2014).

2.1.2 La Crisi del Procedurale e la Nascita degli Oggetti (Slide 14–23)

Il prof. Pasqua presenta il caso di studio di un programma in C che manipola un vettore globale:

```

1 int vect[20];
2 void sort() { /* ordina vect */ }
3 int search(int n) { /* cerca in vect */ }
4 void init() { /* inizializza vect */ }
5 int i;
6
7 void main() {
8     init();
9     sort();
10    search(13);
11 }
```

I 4 Difetti Letali dell'Approccio Procedurale evidenziati a lezione:

1. **Relazione Implicita Dati-Funzioni**: vect e le funzioni sort(), search() sono entità slegate a livello di linguaggio. Non esiste un tipo autonomo "Vettore" che protegga se stesso.
2. **Nessun Controllo dei Confini (Out-of-Bounds)**: un ciclo come for (i=0; i<=21; i++) vect[i]=0; scrive in memoria oltre i 20 elementi, corrompendo altre variabili o lo stack senza alcun errore del compilatore (*buffer overflow*).
3. **Nessuna Garanzia sull'Inizializzazione**: se il programmatore dimentica di invocare init(), le funzioni sort() e search() lavorano su memoria sporca non inizializzata.
4. **Perdita di Confidenzialità e Integrità**: vect è accessibile in lettura/scrittura da qualsiasi funzione del programma. Su sistemi reali questo genera il cosiddetto **Spaghetti Code**.

- **La Regola delle Linee di Codice (LoC)**: - Per script < 30 LoC: lo stile procedurale è rapido e facile da mantenere. - Per progetti > 1000 LoC con team concorrenti: lo stile procedurale crolla; solo l'OOP permette di modularizzare garantendo la manutenibilità.

- **La Soluzione a Oggetti**: - Unire nello stesso modulo sia i dati (**attributi/campi**) sia le operazioni su di essi (**metodi**). - L'oggetto diventa l'unico responsabile della consistenza del proprio stato.

2.1.3 Modellazione UML e Concetti Fondamentali di OOP (Slide 24–36)

- **UML (Unified Modeling Language)**: linguaggio grafico standard per specificare e documentare sistemi ad oggetti.
- **Differenza Ontologica tra Classe e Oggetto**:
 - **Classe**: è il *tipo* (blueprint / modello astratto). Definisce gli attributi e i metodi. Non occupa memoria nello heap per i dati dell'istanza finché non viene istanziata.
 - **Oggetto**: è l'**istanza concreta** nello **Heap**. Possiede:
 1. **Stato**: i valori correnti associati ai suoi attributi.
 2. **Comportamento**: i metodi che può eseguire.
 3. **Identità**: indirizzo di memoria univoco che lo distingue da tutti gli altri oggetti, anche se aventi lo stesso identico stato!

- **Rappresentazione grafica UML:**

- **Classe (Rettangolo a 3 sezioni):**

1. Nome della classe (es. `Car`).
2. Attributi con tipo (es. `manufacturer : string, model : string`).
3. Metodi con segnatura (es. `canFly(): boolean, setFrameNumber(num : uint)`).

- **Oggetto / Istanza (Rettangolo con nome sottolineato):**

- * Es. `Car1: Car` con `manufacturer = DeLorean, model = DMC-12`.

- **Interazione a Passaggio di Messaggi (*Message Passing*):**

- Gli oggetti non leggono né modificano la memoria altrui direttamente: inviano richieste di servizio (*messaggi*).
 - L'**interfaccia** dell'oggetto è l'insieme di messaggi che esso accetta. Se invio un messaggio non previsto, si genera un errore di compilazione.

- **Visibilità:**

- `+` (`public`): accessibile da qualsiasi classe.
 - `-` (`private`): accessibile esclusivamente dall'interno della classe stessa.

- **Incapsulamento vs Information Hiding:**

- **Incapsulamento:** impacchettare dati e codice in una singola unità logica (la classe).
 - **Information Hiding:** nascondere i dettagli implementativi (rendendo i campi `private`) ed esporre solo un'interfaccia controllata (metodi `public`).
 - *Vantaggio chiave:* se cambiamo la struttura dati interna (es. da array a tabella hash), il codice esterno che usa i metodi pubblici non subisce alcun impatto (*zero refactoring esterno*).

- **Associazioni UML:**

- **Associazione standard:** linea continua con cardinalità (es. 1 a 0..n tra `Insurance` e `Vehicle`).
 - **Ereditarietà (Generalizzazione):** freccia con punta triangolare vuota orientata verso la super-classe (`Car` estende `Vehicle`, `Truck` estende `Vehicle`).

- **I 4 Pilastri dell'OOP:**

1. **Incapsulamento** (*Encapsulation*): bundling e protezione dello stato interno.
2. **Astrazione** (*Abstraction*): separazione netta tra cosa un componente fa (interfaccia pubblica) e come lo fa (implementazione privata).
3. **Ereditarietà** (*Inheritance*): meccanismo di riuso e specializzazione gerarchica di proprietà e comportamenti.
4. **Polimorfismo** (*Polymorphism*): capacità di trattare istanze di classi diverse in modo uniforme attraverso un tipo o interfaccia comune.

2.2 Codice: Esempio Comparativo: Dal Vettore C all'Oggetto Java

Per apprezzare concretamente i concetti di T02, confrontiamo il frammento C difettoso mostrato dal docente con la sua trasposizione nativa ad oggetti in Java.

2.2.1 Il Problema in C (Codice procedurale insicuro)

```
1 // C procedurale: il dato è nudo e vulnerabile
2 int vect[20];
3 void init() { /* ... */ }
4
5 int main() {
6     vect[25] = 999; // Corruzione di memoria silenziosa! Nessun errore a runtime.
7     return 0;
8 }
```

2.2.2 La Soluzione ad Oggetti in Java (Incapsulamento, Information Hiding, Costruttore)

In Java, l'oggetto controlla se stesso. Non esiste memoria non inizializzata e l'accesso fuori limite lancia un'eccezione esplicita a runtime:

```
1 public class SafeVector {
2     // 1. Information Hiding: il dato interno è inaccessibile all'esterno
3     private final int[] elements;
4
5     // 2. Garanzia di Inizializzazione: il costruttore viene eseguito all'atto del 'new'
6     public SafeVector(int size) {
7         if (size <= 0) {
8             throw new IllegalArgumentException("La dimensione deve essere positiva");
9         }
10        this.elements = new int[size]; // Nello Heap, azzerato di default a 0
11    }
12
13    // 3. Controllo dei confini e integrità: interfaccia controllata
14    public void set(int index, int value) {
15        // La JVM effettua l'ArrayBoundsCheck automatico, impedendo corruzioni di memoria
16        this.elements[index] = value;
17    }
18
19    public int get(int index) {
20        return this.elements[index];
21    }
22
23    public int size() {
24        return this.elements.length;
25    }
26 }
```

Punto Importante per l'Esame

Takeaway per l'esame: - Una classe non alloca memoria per lo stato fino alla chiamata a `new` (che alloca l'oggetto nello **Heap**). - L'Information Hiding si realizza marcando i campi di istanza come `private`. - I 4 pilastri dell'OOP (*Encapsulation, Abstraction, Inheritance, Polymorphism*) sono il filo conduttore dell'intero corso.

Nota di Studio

Il prossimo blocco è la **Lezione 3 / Slide T03: *Building, Running and Deployment in Java***, in cui studieremo:

- Il modello di esecuzione Java: **Sorgente (.java)** → **Bytecode (.class)** → **JVM / JIT**.
- I comandi CLI fondamentali: `javac`, `java`, `jar`, `javadoc`.
- I concetti di **Classpath** (`-cp`) e gestione delle dipendenze.
- **Apache Maven**: cicli di vita (*compile*, *test*, *package*, *install*), struttura directory standard (`src/main/java`, `src/test/java`) e il `pom.xml`.

Dimmi quando sei pronto per passare a T03!

T03 — Compilazione, Esecuzione e Deployment

JVM, Bytecode, JDK vs JRE, Strumenti CLI (javac, java, jar) e Apache Maven

In questo blocco analizziamo l’ecosistema tecnologico di Java, l’architettura della **Java Virtual Machine (JVM)**, i meccanismi di compilazione, il **Dynamic Class Loading**, la gestione del **Classpath** e le strategie di impacchettamento con file **JAR** ed eseguibili ([T03 - Building, Running and Deployment in Java.pdf](#)).

3.1 Teoria: Concetti Teorici dalle Slide

3.1.1 Il Linguaggio Java e le sue Proprietà Fondamentali (Slide 1–11)

- **Cenni Storici:**

- Creato da **James Gosling** presso Sun Microsystems (progetto *Oak* nel 1991, rilasciato ufficialmente come Java nel **1995**, poi acquisito da Oracle).
- Motto: “*Write Once, Run Everywhere*” (WORA).

- **Le Caratteristiche Chiave del Linguaggio:**

1. **Robustezza:**

- *Tipizzazione forte e statica (Strongly Typed)*: ogni variabile ed espressione ha un tipo noto a tempo di compilazione; i vincoli di tipo sono verificati dal compilatore prima dell’esecuzione.
- *Nessuna manipolazione esplicita di puntatori*: in Java non esistono l’operatore di indirizzo &, l’aritmetica dei puntatori né la deallocazione manuale (`free()`). Questo elimina alla radice *dangling pointers*, *segmentation fault* e *buffer overflow*.
- *Controlli a runtime*: la JVM esegue verifiche automatiche su ogni accesso ad array (`ArrayIndexOutOfBoundsException`) e sui riferimenti (`NullPointerException`).
- *Garbage Collection (GC) automatica*: un thread demone a bassa priorità della JVM individua ed elimina gli oggetti non più raggiungibili nello Heap, riducendo drasticamente i *memory leak*.
- *Gestione strutturata degli errori*: gestione degli stati anomali tramite eccezioni controllate e non controllate (`try-catch-finally`).

2. **Dinamicità:**

- *Caricamento e linking dinamico (Dynamic Linking)*: le classi non vengono collegate in un unico binario monolitico a compile-time, ma sono caricate in memoria dalla JVM *on-demand*, solo quando effettivamente referenziate dal codice in esecuzione.
- *Allocazione dinamica*: la dimensione delle strutture dati e degli array può essere determinata a runtime (`new int[size]`).

3. **Portabilità e Architettura a Bytecode:**

- Nei linguaggi puramente compilati (C/C++), il compilatore genera direttamente codice macchina specifico per l’architettura target (x86, x64, ARM) e per il sistema operativo (Linux, Windows, macOS). Il programmatore deve ricompilare o mantenere build cross-platform separate.

- In Java, il compilatore (javac) traduce il sorgente .java in un formato intermedio indipendente dall'hardware: il **Bytecode** (.class).
- È la **JVM** a farsi carico della traduzione del bytecode in istruzioni macchina della CPU ospitante. La portabilità cessa di essere un onere del programmatore e diventa un servizio fornito dal runtime.

4. Prestazioni e Compilatore JIT (*Just-In-Time*):

- Java non è puramente interpretato. La JVM include un compilatore **JIT** che monitora a runtime le porzioni di bytecode eseguite più frequentemente (*hot spots*) e le compila al volo in codice macchina nativo ottimizzato, memorizzandole nella cache del codice.

• Java vs JavaScript:

- Citazione di Christian Heilmann: «*Java is to JavaScript what Car is to Carpet*» (hanno in comune solo le prime quattro lettere). JavaScript nacque da Netscape con quel nome per motivi puramente commerciali; i due linguaggi non condividono né modello dei tipi, né macchina virtuale, né filosofia di design.

3.1.2 Ecosistema Java: JDK vs JRE (Slide 11)

La distinzione tra ambiente di sviluppo e ambiente di esecuzione: - **JDK (Java Development Kit)**: ambiente per programmatori: - javac: il compilatore che traduce sorgenti .java in bytecode .class. - java: il launcher applicativo che avvia la JVM. - javadoc: generatore di documentazione HTML a partire dai commenti speciali /** ... */. - javap: disassemblatore di bytecode (permette di ispezionare il bytecode generato). - jdb: debugger da riga di comando. - **JRE (Java Runtime Environment)**: ambiente minimale per l'utente finale: - **JVM (Java Virtual Machine)**: il motore di esecuzione. - Librerie standard delle API Java (il modulo base java.base contenente java.lang, java.util, java.io, ecc.). - Compilatore JIT integrato.

3.1.3 Compilazione, Esecuzione e Dynamic Class Loading (Slide 12–16)

1. La Struttura del Programma Minimo:

```

1 // Salvato obbligatoriamente in MyClass.java (stesso nome della classe pubblica)
2 public class MyClass {
3     public static void main(String[] args) {
4         System.out.println("Hello World!");
5     }
6 }

```

- *Regola aurea*: In un file .java può essere presente al massimo **una sola classe pubblica**, e il nome del file deve coincidere esattamente (incluso il maiuscolo/minuscolo) con il nome di tale classe.
- *Firma del metodo main*:
 - public: deve essere accessibile dall'esterno da parte del launcher della JVM.
 - static: la JVM lo invoca direttamente sulla classe, senza dover prima istanziare un oggetto (MyClass obj = new MyClass()).
 - void: non restituisce un codice numerico di ritorno (l'uscita anomala si gestisce con System.exit(code)).
 - String[] args: array di stringhe contenente i parametri passati da riga di comando.

2. Pipeline di Esecuzione della JVM:

Sorgente (.java) $\xrightarrow{\text{javac}}$ Bytecode (.class) $\xrightarrow{\text{Loader}}$ Bytecode Verifier $\xrightarrow{\text{Interpreter / JIT}}$ OS / Hardware

- **Bytecode Verifier:** componente di sicurezza fondamentale della JVM. Prima di eseguire qualsiasi file `.class`, scansiona le istruzioni per verificare che non violino i limiti dello stack, non convertano puntatori illegalmente e rispettino le regole di visibilità.

3. Dynamic Class Loading e Classpath (-cp):

- La JVM non cerca i file nel filesystem a caso: utilizza il **Classpath** (una lista ordinata di directory e archivi `.jar`).
- Quando durante l'esecuzione il codice fa riferimento per la prima volta a una classe `x`:
 1. Il `ClassLoader` cerca `X.class` nella prima cartella specificata nel `Classpath`.
 2. Se non la trova, passa alla successiva.
 3. Se la trova, la carica in memoria e ne inizializza le strutture statiche.
 4. Se non la trova in nessuna cartella del `Classpath`, solleva `ClassNotFoundException` o `NoClassDefFoundError`.
- Esempio di esecuzione esplicita:

```
1 java -cp . MyClass
```

Il parametro `-cp .` istruisce la JVM a cercare le classi compilate nella directory corrente (`.`). Le classi standard di sistema (`System`, `String`) vengono invece caricate automaticamente dal bootstrap classloader della JDK.

3.1.4 Deployment e Gestione dei File JAR (Slide 17–18)

- **Anatomia di un file JAR (*Java Archive*):**

- Un file `.jar` è fisicamente un **archivio compresso in formato standard ZIP**.
- Raggruppa decine o centinaia di file `.class`, risorse (immagini, file di configurazione) e metadati.

- **Creazione manuale da CLI:**

```
1 jar cvf MyJar.jar MyClass.class
```

- `c`: *create* (crea nuovo archivio).
- `v`: *verbose* (mostra i file aggiunti).
- `f`: *file* (il parametro successivo è il nome del file `.jar` da produrre).

- **Esecuzione tramite Classpath:**

```
1 java -cp MyJar.jar MyClass
```

- **JAR Auto-Eseguibile (`java -jar`):**

- Per consentire all'utente di lanciare l'archivio direttamente con `java -jar MyJar.jar`, il JAR deve specificare quale classe ospita il metodo `main`.
- Questa informazione risiede nel file `META-INF/MANIFEST.MF` sotto la direttiva:

```
1 Main-Class: MyClass
```

- Creazione con file manifest esterno:

```
1 jar cvfm MyClass.jar manifest.txt MyClass.class
```

- Creazione con sintassi moderna senza file di testo intermedio (flag `-e` per entrypoint):

```
1 jar cfe MyClass.jar MyClass MyClass.class
```

3.1.5 Strumenti di Sviluppo: Da Editor a IntelliJ IDEA (Slide 19–26)

- **CLI vs IDE:** per progetti complessi, l'uso del terminale e di un editor di testo semplice diventa poco scalabile.
- **Vantaggi dell'IDE (*Integrated Development Environment*):**
 - *Syntax highlighting* e *live code inspection/linting* (segnalazione immediata degli errori di tipo e code smell prima ancora di compilare).
 - *Debugger integrato* con breakpoint, ispezione dello stack dei frame e monitoraggio variabili.
 - *Integrazione con Version Control (Git)* e *Build Automation Tools (Maven / Gradle)*.
 - *Monito del docente:* l'auto-completamento o la generazione automatica di codice da parte dell'IDE è utile in ambito professionale, ma va limitata durante la fase di apprendimento per comprendere a fondo le regole sintattiche e semantiche del linguaggio.
- **Scelta del corso: IntelliJ IDEA** (gratuito in versione Community o con licenza Education per studenti universitari).

3.2 Codice: Collegamento Pratico con il Tuo Progetto: Maven e guida-jar.md

Nel tuo workspace [esercizi-wally-25-26](#), hai formalizzato questi esatti concetti in modo eccellente in [guida-jar.md](#) e nei file di configurazione Maven:

1. Il Fully Qualified Name e il Filesystem:

- Come visto a lezione, se una classe appartiene al package `es01`, il bytecode generato deve risiedere fisicamente nel percorso `es01/Date.class`.
- Quando si invoca la JVM, il nome della classe da passare deve essere il FQN (`es01.MainDate`).

2. Automazione del Packaging con Maven:

- Invece di lanciare manualmente `jar cfe`, nei singoli moduli (es. [es01/pom.xml](#)) usi il plugin **maven-assembly-plugin**:

```
1 <plugin>
2   <groupId>org.apache.maven.plugins</groupId>
3   <artifactId>maven-assembly-plugin</artifactId>
4   <version>3.7.1</version>
5   <configuration>
6     <archive>
7       <manifest>
8         <mainClass>es01.MainDate</mainClass>
9       </manifest>
10    </archive>
11    <descriptorRefs>
12      <descriptorRef>jar-with-dependencies</descriptorRef>
13    </descriptorRefs>
14  </configuration>
15  <executions>
16    <execution>
17      <id>make-assembly</id>
18      <phase>package</phase>
19      <goals>
20        <goal>single</goal>
21      </goals>
22    </execution>
23  </executions>
24 </plugin>
```

Questo genera automaticamente nella cartella `target/` il cosiddetto *Fat JAR* contenente sia il bytecode dell'applicazione sia il file `META-INF/MANIFEST.MF` precompilato.

3.3 [Test] Verifica dei Comandi da Terminale

Puoi verificare l'intero ciclo di build ed esecuzione da riga di comando testando il modulo `es01` tramite lo script:

```
1 ./esercizi.sh package es01
```

Lo script esegue internamente: 1. `mvn clean package -pl es01` 2. Individua il file JAR autoprodotta in `es01/target/es01-*-jar-with-dependencies.jar` 3. Esegue `java -jar ...` verificando che l'entrypoint `es01.MainDate` risponda correttamente.

Nota di Studio

Con T03 abbiamo concluso l'intera panoramica propedeutica (Introduzione, Paradigmi/UML e Tooling/JVM).

A partire dal prossimo blocco entriamo nel codice sorgente vero e proprio con la **Lezione 4 / Slide T04: *Types and Objects*** e il file ufficiale del docente [Example.java](#): - Tipi primitivi (dimensioni, range, valori di default) vs Reference Types. - Rappresentazione dei caratteri (`char` come intero senza segno a 16 bit, codifica ASCII e Unicode `\u0056`). - Regole di inizializzazione: variabili di istanza/statiche (default zero/null) vs variabili locali dello stack (nessun default → errore a compile-time!). - Operatori logici standard (`&`, `|`) vs operatori a corto circuito (*short-circuit* `&&`, `||`) e relativi effetti collaterali (`k++`). - Scope dei blocchi di codice annidati (`{ ... }`) e shadowing/visibilità delle variabili.

Dammi conferma per aprire la Lezione 4 / T04 e analizzare teoria e codice di `Example.java`!

T04 — Sistema di Tipi, Primitive e Reference

Tipi Primitivi vs Reference, Casting, Wrapper, Operatori e Analisi di Example.java

Con questo blocco entriamo nel vivo della semantica del linguaggio: analizziamo come la JVM gestisce i tipi, la memoria (Stack vs Heap), i puntatori/reference, gli operatori logici e lo scope, confrontando la teoria di [T04 - Types and Objects.pdf](#) con il primo file di codice ufficiale del docente: [Example.java](#).

4.1 Teoria: Concetti Teorici dalle Slide

4.1.1 Tipizzazione dei Linguaggi e il “Contratto” di Tipo (Slide 1–7)

- **Definizione formale informale di Tipo:** Un tipo è una coppia formata da un **insieme di valori ammissibili** e da un **insieme di operazioni consentite** su di essi.
 - Es. `int`: valori da -2^{31} a $2^{31} - 1$, operazioni `+`, `-`, `*`, `/`, `%`.
 - Il tipo definisce un **contratto**: vincola cosa una variabile può contenere e in quali espressioni può comparire.
- **Weakly Typed (es. Python, JavaScript) vs Strongly Typed (Java):**
 - Nei linguaggi *weakly typed*, le variabili sono contenitori generici slegati dal tipo: una variabile `x` può contenere un intero, poi una stringa, poi una tupla.
 - Nei linguaggi *strongly typed* come Java, ogni variabile richiede una dichiarazione esplicita e immutabile di tipo a tempo di compilazione (`int x;`). Assegnare un tipo incompatibile genera un **Compile-Time Error**.
 - *Meme di JavaScript a lezione:*
 - * `"11" + 1` $\Rightarrow$ `"111"` (coercizione implicita a stringa e concatenazione).
 - * `"11" - 1` $\Rightarrow$ `10` (coercizione implicita a numero e sottrazione numerica).
 - * In Java queste ambiguità ed errori silenziosi sono impossibili: il compilatore blocca ogni operazione non conforme prima dell'esecuzione.

4.1.2 I Tipi Primitivi in Java e le Conversioni (Slide 8–13)

Java dispone di **8 tipi primitivi** (allocati direttamente per valore nello Stack o inline negli oggetti, non sono oggetti):

Tipo Primitivo	Dimensione	Range di Valori	Default (campi)
<code>byte</code>	8 bit (1 byte)	da -128 a $+127$ ($-2^7 \dots 2^7 - 1$)	<code>0</code>
<code>short</code>	16 bit (2 byte)	da -32.768 a $+32.767$ ($-2^{15} \dots 2^{15} - 1$)	<code>0</code>

Tipo Primitivo	Dimensione	Range di Valori	Default (campi)
int	32 bit (4 byte)	da $-2.147.483.648$ a $+2.147.483.647$ ($-2^{31} \dots 2^{31} - 1$)	0
long	64 bit (8 byte)	da -2^{63} a $+2^{63} - 1$ (suffisso L o l)	0L
float	32 bit IEEE 754	Precisione singola (suffisso obbligatorio f o F)	0.0f
double	64 bit IEEE 754	Precisione doppia (default per numeri con la virgola)	0.0d
char	16 bit (2 byte)	da 0 a 65.535 (Unicode UTF-16, unsigned)	'\u0000' (NUL)
boolean	1 bit logico	true oppure false	false

- **Aritmetica Modulare e Overflow degli int:**

- In Java non viene lanciata alcuna eccezione se un calcolo intero supera il limite massimo: avviene il **wrap-around** secondo l'aritmetica del complemento a due a 32 bit.
- Se $a = 2147483647$ ($2^{31} - 1$), l'operazione $a + 1$ restituisce -2147483648 (-2^{31}).

- **Conversioni di Tipo (Casting):**

- **Widening (Allargamento / Implicito):** da un tipo più stretto a uno più ampio (nessuna perdita d'ordine di grandezza):

byte $\longrightarrow$ short $\longrightarrow$ int $\longrightarrow$ long $\longrightarrow$ float $\longrightarrow$ double

Es: float f = 3; converte automaticamente int 3 in 3.0f.

- **Narrowing (Restringimento / Esplicito obbligatorio):** da un tipo più ampio a uno più stretto, richiede il cast sintattico (tipo) perché comporta potenziale troncamento o perdita di informazione: Es: int i = (int)3.2; scarta la parte decimale e memorizza 3.

- **I Caratteri sono Numeri Interi Unsigned a 16 bit:**

- Un char memorizza il codice numerico Unicode (UTF-16 code unit).
- 'V', (char)86 e '\u0056' sono tre rappresentazioni letterali dello **stesso identico dato binario** (valore decimale 86).

- **Tipi Non-Primitivi (Reference Types) e la Classe String:**

- Tutto ciò che non è un tipo primitivo è un **oggetto**.
- String è una classe immutabile. L'operatore + tra una stringa e qualsiasi altro tipo (primitivo o oggetto) applica automaticamente l'overloading di concatenazione, invocando internamente la conversione a stringa.

4.1.3 Modello di Memoria, Puntatori e Garbage Collection (Slide 14–21)

- **Assenza di Variabili Globali:** In Java non esiste lo scope globale in stile C. Ogni dato o funzione appartiene a una classe o a un'istanza.
- **La Verità sui Puntatori in Java:**

- «*Java does have pointers, but there is no pointer arithmetic*»: ogni variabile di tipo non primitivo è tecnicamente un **puntatore/riferimento** (*reference*) a un blocco di memoria nello **Heap**.
- La JVM protegge la memoria: il programmatore non può vedere l'indirizzo fisico, né fare `ptr++` o dereferenziare offset arbitrari.
- Il passaggio dei parametri ai metodi in Java è **sempre rigorosamente per valore** (*pass-by-value*): per gli oggetti, viene copiato per valore il **riferimento** (l'indirizzo logico).

- **Rappresentazione di un Reference:**

- Una variabile d'istanza non inizializzata contiene `null` (es. `Vehicle@null`).
- Quando viene invocato `new Vehicle()`, la JVM alloca l'oggetto nello Heap e restituisce l'indirizzo: `myCar` conterrà un valore del tipo `Vehicle@25c7f37d`.

- **Errori di Memoria e `NullPointerException` (NPE):**

- Se si tenta di invocare un metodo o accedere a un campo su una reference che vale `null` (es. `myCar2.setFrameNumber(5)`), la JVM solleva una **`NullPointerException`** a runtime, interrompendo l'esecuzione.

- **Perché i programmatori scrivono `myCar = null;?`**

- L'assegnamento a `null` rimuove il puntatore verso l'oggetto nello Heap. Se nessun'altra reference punta a quell'area di memoria, l'oggetto diventa **irraggiungibile** (*unreachable*) ed eleggibile per la **Garbage Collection (GC)**, consentendo alla JVM di bonificare la memoria.

4.2 Codice: Analisi Dettagliata del Codice: `Example.java`

Analizziamo riga per riga il sorgente della Lezione 04, evidenziando tutte le trappole e i dettagli d'esame:

```

1 public class Example {
2     static float f1;
3     static float f2 = 3.4f;
4     static String s1;
5     static final double PI = 3.14;
6
7     public static void main(String[] args) {
8         int i1 = 6;
9         int i2;
10        char c1 = 'V';
11        char c2 = 86;
12        char c3 = '\u0056';
13        String s2;
14
15        System.out.println("i1: " + i1);
16        // System.out.println("i2: " + i2); // compile-time error
17        System.out.println("f1: " + f1);
18        System.out.println("f2: " + f2);
19        System.out.println("c1: " + c1 + " c2: " + c2 + " c3: " + c3);
20        System.out.println("s1: " + s1);
21        /*
22         System.out.println("s2: " + s2); // compile-time error
23         System.out.println(s1.toString()); // run-time error
24         PI = 3.0; // compile-time error
25        */
26        int k = 0;
27        System.out.println(true | k++ == 0);
28        System.out.println("k (standard eval): " + k);
29        k = 0;

```



```

30     System.out.println(true || k++ == 0);
31     System.out.println("k (short-circuit eval): " + k);
32     //
33     int n = 4;
34     n ++;
35     // int n = 8; // compile-time error
36     {
37         int m = 7;
38         m ++;
39     }
40     {
41         int m = 8;
42         m --;
43     }
44     System.out.println("n: " + n);
45     // System.out.println("m: " + m); // compile-time error
46 }
47 }

```

4.2.1 [Approfondimento] Le Finezze Implementative Spiegate Riga per Riga:

1. Variabili Statiche/Campi vs Variabili Locali (*Definite Assignment Analysis*):

- `static float f1;` e `static String s1;` sono campi a livello di classe. **La JVM li inizializza SEMPRE automaticamente al loro default:** `f1` diventa `0.0f` e `s1` diventa `null`.
- `int i2;` e `String s2;` sono variabili locali allocate nel frame dello **Stack**. **Le variabili locali NON hanno valori di default.**
- Se tentiamo di leggere `i2` o `s2` prima di aver assegnato loro un valore (`System.out.println(i2)`), il compilatore Java blocca la compilazione con l'errore: `variable i2 might not have been initialized`.

2. Suffisso dei Floating Point (`3.4f`):

- `static float f2 = 3.4f;` il letterale `3.4` senza suffissi è un `double` a 64 bit. Scrivere `float f = 3.4;` genera errore di compilazione (*loss of precision*). Il suffisso `f` forza il letterale a 32 bit.

3. Costanti con `final`:

- `static final double PI = 3.14;` la keyword `final` rende la variabile a sola lettura dopo l'inizializzazione. Il tentativo di riassegnamento `PI = 3.0;` viene respinto dal compilatore (`cannot assign a value to final variable PI`). Per convenzione Java, le costanti `static final` si scrivono in `MAIUSCOLO_SNAKE_CASE`.

4. I 3 modi di rappresentare un `char`:

- `c1 = 'V';` letterale carattere.
- `c2 = 86;` valore intero decimale ASCII/Unicode per la 'V'.
- `c3 = '\u0056';` sequenza di escape Unicode a 16 bit in esadecimale ($5 \times 16 + 6 = 86$).
- Stampati a video, producono tutti e tre il carattere `v`.

5. Concatenazione di `null` vs `NullPointerException`:

- `System.out.println("s1: " + s1);` stampa `"s1: null"`. L'operatore `+` converte in sicurezza il riferimento nullo nella stringa letterale `"null"`.
- `System.out.println(s1.toString());` genera invece un **Run-Time Error** (`NullPointerException`): non è possibile dereferenziare un puntatore nullo per invocarvi un metodo d'istanza.

6. Operatori Logici Standard (`|`, `&`) vs Corto-Circuito (*Short-Circuit* `||`, `&&`) e Side Effects:

- Caso `true | k++ == 0`:
 - L'operatore singolo `|` (non a corto circuito) **valuta SEMPRE entrambi gli operandi**.

- Anche se la parte sinistra è già `true`, la parte destra `k++ == 0` viene valutata: `k` viene confrontato con `0` (dando `true`) e poi incrementato di 1.
- Stampa a video: `true`, seguito da `k` (standard eval): 1.
- Caso `true || k++ == 0`:
 - L'operatore doppio `||` (a corto circuito) verifica la parte sinistra: poiché è `true`, il risultato globale dell'or logico è già matematicamente certo (`true`).
 - La parte destra viene **completamente ignorata e mai valutata!**
 - Di conseguenza, `k++` **non viene eseguito**.
 - Stampa a video: `true`, seguito da `k` (short-circuit eval): 0.
- *Importanza pratica*: lo short-circuit `&&` è l'idioma fondamentale per evitare crash su `null`:

```
1 if (s != null && s.length() > 0) { ... } // Se s è null, non chiama mai s.length()!
```

7. Scope delle Variabili e Divieto di Shadowing Locale:

- In Java, il ciclo di vita e la visibilità delle variabili sono delimitati dalle graffe `{ ... }`.
- `int n = 4; n++;` dichiara `n` nello scope del metodo. Scrivere `int n = 8;` nello stesso scope o in un sottoblocco interno genera l'errore: `Variable 'n' is already defined in the scope`. In Java le variabili locali non possono oscurare altre variabili locali omonime.
- Blocchi anonimi indipendenti:

```
1 { int m = 7; m++; }
2 { int m = 8; m--; }
```

La variabile `m` del primo blocco viene distrutta alla chiusura della prima graffa. La seconda `m` è una nuova variabile in uno scope disgiunto. All'esterno dei blocchi, `m` non esiste (cannot find symbol variable `m`).

4.3 [Test] Output Esatto di Esecuzione

Eseguendo `Example.java` da terminale o IntelliJ:

```
1 i1: 6
2 f1: 0.0
3 f2: 3.4
4 c1: V c2: V c3: V
5 s1: null
6 true
7 k (standard eval): 1
8 true
9 k (short-circuit eval): 0
10 n: 5
```

Nota di Studio

Con la **Lezione 4 / T04** abbiamo chiarito le basi di tipo, stack/heap e operatori. Il prossimo blocco raggruppa le **Lezioni 05-06 / Slide T05 (Java Basic Syntax) e T06 (Java Classes and Objects)**, dove nasce il progetto incrementale centrale del corso: - Struttura della prima classe ad oggetti: `SimpleDate` (`Date.java` e `MainDate.java`). - La classe sperimentale `StringPlayground.java`. - La prima validazione delle date: algoritmo dei giorni del mese e calcolo degli anni bisestili secondo

il calendario Gregoriano. - Sintassi Java: costrutti di controllo (if-else, switch, for, while, do-while), etichette (*labeled break/continue*). - Metodi, parametri, ritorno e la keyword `this`. - La classe `MainDate`: test delle istanze, istanziazione con `new` e stampa dello stato.

Dammi conferma per aprire le Lezioni 05-06 / T05-T06 e analizzare la prima versione di `SimpleDate` e `StringPlayground`!

T05–T06 — Sintassi Java, Strutture di Controllo e Classi Base

Costrutti Condizionali e Iterativi, Definizione di Classe e Nascita del Progetto SimpleDate

In questo blocco analizziamo in dettaglio la sintassi fondamentale di Java ([T05 - Java Basic Syntax.pdf](#)), la struttura delle classi e degli oggetti ([T06 - Java Classes and Objects.pdf](#)) e il codice sorgente di debutto del corso: il progetto [SimpleDate](#) e il file di laboratorio [StringPlayground.java](#).

5.1 Teoria: Concetti Teorici dalle Slide

5.1.1 Sintassi di Base e Costrutti Semplici (Slide T05, 1–19)

- **Variabili e Inizializzazione Automatica:**
 - Una variabile associa un nome e un tipo a una locazione di memoria.
 - **Regola cruciale:** Solo i campi di classe/istanza (*fields*) vengono inizializzati automaticamente al default (0, 0.0, false, '\u0000', null). Le variabili locali nello Stack richiedono l'assegnamento esplicito prima dell'uso.
- **Costanti (static final):**
 - `final static int I = 5;` la variabile è a sola lettura; deve essere inizializzata contestualmente o nel costruttore e non può più essere riassegnata.
- **Identificatori e Convenzioni di Naming:**
 - Regola sintattica: sequenza arbitraria di caratteri [a-z], [A-Z], [0-9], _ e \$, che non inizi con una cifra e non coincida con una delle **50 parole chiave riservate** (class, static, new, this, super, switch, goto, const, ecc.).
 - Convenzioni ufficiali Java:
 - * **Classi:** UpperCamelCase (es. SimpleDate, MainDate).
 - * **Metodi, variabili locali, campi e parametri:** lowerCamelCase (es. daysPerMonth, myCar).
 - * **Costanti:** UPPER_SNAKE_CASE (es. FORMAT, PI_CONST).
- **Caratteri di Escape Speciali:**
 - \n (LF linefeed, \u000a), \r (CR carriage return, \u000d), \t (tab orizzontale, \u0009), \b (backspace, \u0008), \f (form feed, \u000c), \" (doppio apice), \' (singolo apice), \\ (backslash).
- **Commenti e Documentazione:**
 - // ... (singola riga).
 - /* ... */ (multi-riga).
 - /** ... */ (**Javadoc**): elaborato dal tool javadoc per generare automaticamente la documentazione HTML delle API.
- **Espressioni, Valutazione Sinistra-Destra e Side Effects:**
 - In Java le sotto-espressioni sono valutate **rigorosamente da sinistra a destra**:
 - * `int i = 1; int j = (i = 2) * i;` $\Rightarrow$ i diventa 2, poi moltiplicato per i (2) $\Rightarrow$ j = 4.

* `int t = 3; eval(t + (t = 2))` $\Rightarrow$ prima si valuta `t` (3), poi `(t = 2)` (2) $\Rightarrow 3 + 2 = 5$.

- **Operatori Aritmetici Unari: Pre-incremento vs Post-incremento:**

- `x++` (post-incremento): restituisce il valore attuale di `x` e *successivamente* incrementa `x` di 1.
- `++x` (pre-incremento): incrementa *subito* `x` di 1 e restituisce il nuovo valore.
- **L'esercizio tranello delle slide:**

```
1 int t = 3;
2 t = -t++ - ++t; // Quanto vale t?
```

* Valutazione da sinistra a destra:

1. `-t++`: valuta `-3`, e incrementa `t` a 4.
2. `++t`: incrementa subito `t` da 4 a 5, e valuta 5.
3. Operazione: $(-3) - 5 = -8$. Assegnato a `t`, il valore finale è **-8**.

- **Operatore Ternario:** `condizione ? expr1 : expr2`.

- **Operatori Logici e Corto-Circuito:**

- `&`, `|`: valutano sempre entrambi i rami.
- `&&`, `||`: valutazione a corto circuito (*short-circuit*). Se il primo operando determina già il risultato, il secondo non viene nemmeno eseguito (essenziale per prevenire crash su puntatori nulli).

- **Assegnamenti Composti:** `x += expr` equivale a `x = x + (expr)`. Se `x = 4`, l'espressione `x+ = x - 7` assegna `x = 4 + (4 - 7) = 1`.

5.1.2 Costrutti di Controllo di Flusso (Slide T05, 20–29)

- **Blocchi { ... }:** delimitano lo scope di vita delle variabili locali.

- **Condizionali:** `if (Bexpr) ... else ...`

- **Costrutto switch-case e il Fall-Through:**

- Seleziona il ramo da eseguire in base al valore di un'espressione (`byte`, `short`, `char`, `int`, `String`, `enum`).
- Se non si inserisce l'istruzione `break`, l'esecuzione **continua nei rami successivi** (*fall-through*). Questo meccanismo, se usato volontariamente, permette di raggruppare casi che condividono la stessa logica (come vedremo in `daysPerMonth`).

- **Cicli:**

- `while (cond){ ... }`: test preliminare (può eseguire 0 volte).
- `do { ... } while (cond);`: test posticipato (esegue almeno 1 volta).
- `for (init; cond; step){ ... }`: ciclo controllato. Supporta inizializzazioni e passi multipli separati da virgola:

```
1 for (int i = 0, j = 9; i <= 9 && j >= 0; i++, j--) { ... }
```

- **Salto di Controllo:**

- `break`: interrompe il ciclo o lo switch ed esce dal blocco.
- `continue`: salta immediatamente alla successiva iterazione del ciclo.
- `return`: esce dal metodo corrente restituendo eventualmente un valore.

5.1.3 Classi, Oggetti, `this` e `toString` (Slide T06, 1–28)

- **Perché modellare una data? (La vignetta a lezione, Slide 7):**
 - Lo slide show mostra il fumetto geek: «*Imagine an object much like a thing in the real world, like... A DATE?*». È da qui che nasce la classe `Date` come filo conduttore del corso.
- **Anatomia di una Classe:**
 - **Campi d'istanza (*Fields*):** variabili che memorizzano lo stato di ciascun oggetto.
 - **Metodi d'istanza (*Methods*):** funzioni che definiscono il comportamento e i messaggi accettati dall'oggetto.
 - **Firma del metodo:** `tipoRitorno nomeMetodo(tipoParametro nomeParametro, ...)`. Se non restituisce nulla si usa `void`.
 - **Overloading:** possibilità di definire più metodi con lo stesso nome purché differiscano per numero, tipo o ordine dei parametri.
- **Entrypoint `main` e Metodi Statici (`static`):**
 - `public static void main(String[] args):` il launcher della JVM non passa il nome del programma in `args[0]` (a differenza del C). `args[0]` è già il primo parametro utente.
 - **Metodi Statici:** appartengono alla classe e non alle singole istanze. Si invocano con `NomeClasse.metodo()`. Non possono accedere allo stato d'istanza né usare `this`.
- **Creazione dell'Oggetto (`new`) e Costruttore:**
 - L'istruzione `new Date(...)`:
 1. Alloca memoria nello Heap per l'oggetto.
 2. Inizializza i campi ai valori di default.
 3. Invoca il **costruttore** (metodo speciale con lo stesso nome della classe e senza tipo di ritorno) per valorizzare lo stato.
 4. Restituisce il riferimento all'oggetto appena creato.
- **Aliasing e Garbage Collection:**
 - Se scriviamo `Date d2 = d1;`, le due variabili puntano allo stesso identico oggetto nello Heap (*aliasing*).
 - Quando una reference viene azzerata (`d1 = null; d2 = null;`), l'oggetto diventa orfano e il Garbage Collector può bonificarlo.
- **La Parola Chiave `this`:**
 - È il riferimento implicito all'oggetto corrente su cui è stato invocato il metodo.
 - È **obbligatoria** quando i parametri del costruttore/metodo hanno lo stesso identico nome dei campi d'istanza (*shadowing dei parametri*):

```
1 public Date(int day, int month, int year) {  
2     this.day = day;    // this.day è il campo dell'oggetto nello Heap  
3     this.month = month; // day è il parametro locale nello Stack  
4     this.year = year;  
5 }
```

- **Rappresentazione a Stringa e `toString()`:**
 - Tutte le classi Java ereditano dalla superclasse universale `java.lang.Object` il metodo `public String toString()`.
 - Il default di `Object.toString()` stampa `NomeClasse@IndirizzoHashCode` (es. `Date@5acf9800`).
 - La JVM invoca **implicitamente** `toString()` ogni volta che un oggetto viene concatenato a una stringa ("Oggi: " + today).

- Sovrascrivendo (*overriding*) `public String toString()` nella classe, possiamo personalizzare la stampa dell'oggetto.

5.2 Codice: Analisi del Codice Ufficiale del Docente

5.2.1 SimpleDate/src/Date.java

```
1 public class Date {
2     // 1. Stato dell'oggetto: campi di istanza (visibilità default di package per ora)
3     int day;
4     int month;
5     int year;
6
7     // 2. Costante di classe condivisa da tutte le istanze
8     static final String FORMAT = "dd/mm/yyyy";
9
10    // 3. Costruttore: usa 'this' per distinguere campi da parametri
11    public Date(int day, int month, int year) {
12        this.day = day;
13        this.month = month;
14        this.year = year;
15        verify(); // Chiamata al metodo di validazione interna
16    }
17
18    // 4. Metodo di validazione dello stato
19    void verify() {
20        if (year < 0 || month < 1 || month > 12)
21            System.out.println("Illegal date!");
22        else
23            if (day < 1 || day > daysPerMonth(month))
24                System.out.println("Illegal date!");
25    }
26
27    String print() {
28        return day + "/" + month + "/" + year + " [" + FORMAT + "];"
29    }
30
31    // 5. Override di toString(): delega al metodo print()
32    public String toString() {
33        return print();
34    }
35
36    // 6. Metodo statico: calcolo puro senza bisogno di istanziare Date
37    static int daysPerMonth(int month) {
38        int days;
39        switch(month) {
40            case 4:
41            case 6:
42            case 9:
43            case 11:
44                days = 30; // Fall-through controllato per i mesi da 30 giorni
45                break;
46            case 2:
47                days = 28; // Febbraio fisso a 28 (il bisestile arriverà a Lezione 07!)
48                break;
49            default:
50                days = 31;
51                break;
52        }
53    }
54 }
```

```

53     return days;
54 }
55 }

```

5.2.2 [Approfondimento] Dettagli Tecnici e Scelte del Docente:

1. **Perché `daysPerMonth(month)` è `static`?** Non ha bisogno di leggere `this.day` o `this.year`: riceve un mese intero e restituisce i giorni. Essendo una funzione pura di utilità, appartiene alla classe e può essere chiamata anche prima di creare oggetti (come fa il `main`).
2. **Lo `switch` con `fall-through`:** I case 4: case 6: case 9: case 11: sono privi di istruzione `break` intermedia; l'esecuzione scorre intenzionalmente fino a `days = 30; break;`.
3. **Validazione con `verify()`:** In questa prima versione didattica, se la data è errata viene stampato a video "Illegal date!". Notare che l'oggetto non valido viene comunque creato in memoria! Solo più avanti (in Lezione 17 con le eccezioni) il docente mostrerà come impedire l'istanziamento lanciando un'eccezione.

5.2.3 SimpleDate/src/MainDate.java

```

1  import java.util.Scanner;
2
3  public class MainDate {
4      public static void main(String[] args) {
5          // Chiamata a metodo statico direttamente da classe
6          System.out.println(Date.daysPerMonth(4));
7
8          // Allocazione istanze nello Heap
9          Date today = new Date(14, 10, 2025);
10         Date tomorrow = new Date(15, 10, 2025);
11
12         System.out.println(today.print());
13         System.out.println(tomorrow.print());
14         System.out.println(today.toString());
15
16         String str = new String("ciao");
17         System.out.println(str.toString());
18
19         // Acquisizione input utente con java.util.Scanner
20         Scanner sc = new Scanner(System.in);
21         System.out.println("Enter the day: ");
22         int day = sc.nextInt();
23         System.out.println("Enter the month: ");
24         int month = sc.nextInt();
25         System.out.println("Enter the year: ");
26         int year = sc.nextInt();
27         sc.close(); // Chiusura dello stream di input
28
29         Date date = new Date(day, month, year);
30         System.out.println(date.toString());
31     }
32 }

```


5.2.4 StringPlayground.java

Questo file illustra 3 concetti cardine sulle stringhe e l'ottimizzazione della memoria:

```
1 public class StringPlayground {
2     public static void main(String[] args) {
3         String empty = new String();
4         String str = "str";
5         System.out.println("len(empty): " + empty.length());
6         System.out.println("len(\"str\"): " + str.length());
7         System.out.println(str.charAt(1)); // Restituisce 't'
8
9         // str.charAt(1) = 'c'; // COMPILE-TIME ERROR: Le String in Java sono IMMUTABILI!
10
11        String same = new String("str");
12        System.out.println("str: " + str + " same: " + same);
13
14        // Confronto di puntatori (identità di memoria) vs contenuto semantico
15        System.out.println("str ?= same: " + (str == same));           // false! Oggetti diversi
16                                nello Heap
17        String verySame = str;
18        System.out.println("str ?= verySame: " + (str == verySame)); // true! Puntano alla stessa
19                                cella
20        System.out.println("str ?= same: " + str.equals(same));       // true! Stessi caratteri
21
22        // StringBuilder e Fluent Interface (Chaining)
23        int n = 5;
24        int sum = n * (n + 1) / 2;
25        StringBuilder sb = new StringBuilder();
26        sb.append("sum(").append(sum).append(")"); // Notazione a catena (fluent)
27        for (int i = 0; i < n; i++)
28            sb.append(" ").append(i);
29        sb.append(" ]");
30        System.out.println(sb.toString()); // sum(15)[ 0 1 2 3 4 ]
31    }
```

5.2.5 [Approfondimento] Le Finezze di StringPlayground:

1. **Immutabilità:** in Java l'oggetto `String` non può essere modificato dopo la creazione. Non esiste un metodo `setCharAt()`.
2. **`==` vs `.equals()`:**
 - `str == same` confronta se le due variabili contengono lo **stesso indirizzo di memoria**. Restituisce `false` perché `same` è stato creato con `new String(...)` forzando una nuova allocazione nello Heap.
 - `str.equals(same)` confronta i caratteri uno a uno e restituisce `true`.
 - **Regola d'esame assoluta:** per confrontare gli oggetti e le stringhe si usa **sempre** `.equals()`, mai `==`!
3. **StringBuilder e Fluent Notation:**
 - Ogni concatenazione con `+` su stringhe ordinarie crea un nuovo oggetto `String` temporaneo. Nei cicli questo genera un enorme sovraccarico di memoria per il Garbage Collector.
 - `StringBuilder` alloca un buffer mutabile ridimensionabile internamente.
 - Il metodo `.append()` restituisce un riferimento a `this` (l'istanza corrente del builder stesso), permettendo di concatenare le chiamate in cascata: `sb.append(...).append(...)` (**Fluent API / Method Chaining**).

5.3 □ Confronto con il tuo modulo es01

Nel tuo file `es01/src/main/java/es01/Date.java`: 1. **Switch Moderno vs Tradizionale**: Avevi già applicato la sintassi Java 14+ con la *Switch Expression*: `java static int daysPerMonth(int month){ return switch (month){ case 4, 6, 9, 11 -> 30; case 2 -> 28; default -> 31; }; }` Questa forma è sintatticamente pulitissima e idiomatica in Java moderno. 2. **Allineamento con la Lezione 05-06**: Per essere al 100% fedeli alla versione del docente: - Aggiungere la chiamata a `verify()` nel costruttore per il controllo di validità. - Correggere il refuso nella costante di formato: `"dd/mm/yyyy"` invece di `"dd/hh/yyyy"`.

Nota di Studio

Con le **Lezioni 05-06** abbiamo stabilito l'architettura iniziale di `Date`, lo scope, i metodi statici, le stringhe e `StringBuilder`.

Il prossimo blocco è la **Lezione 07 / Slide T07: Libraries and String Class**: - Evoluzione di `SimpleDate`: introduzione dell'algoritmo completo del **calendario Gregoriano per gli anni bisestili** (*leap year*). - Analisi della classe statica ausiliaria di calcolo matematico `java.lang.Math`. - Metodi avanzati della classe `String` (`substring`, `indexOf`, `split`, `trim`, `replace`). - Wrapper classes per i primitivi (`Integer`, `Double`, `Character`), *Autoboxing* e *Unboxing*.

Dammi conferma per procedere alla Lezione 07 / T07!

T07 — Librerie Standard e Manipolazione delle Stringhe

Java Class Library, Immutabilità della Classe String, Metodi Fondamentali e SimpleDate v2

In questo blocco analizziamo la gestione delle librerie standard e la classe `String` ([T07 - Libraries and String Class.pdf](#)), insieme all'evoluzione del codice nella cartella [Lezione07](#), dove il docente introduce **incapsulamento** (`private`), **costruttori sovraccaricati con delega** `this(...)`, **copy constructor**, formattazione internazionalizzata e il primo test con **asserzioni** (`assert` e flag `-ea`).

6.1 Teoria: Concetti Teorici dalle Slide (Slide T07)

6.1.1 Librerie Standard e Meccanismo di Import (Slide 1–9)

- **Librerie Standard della JDK:**

- `java.lang`: classi fondamentali del runtime (oggetti base, tipi primitivi wrapper, stringhe, thread, sistema). **È l'unico package importato automaticamente e implicitamente in ogni file sorgente Java.**
- `java.util`: strutture dati (collezioni), utilità temporali, parser e scanner.
- `java.io`: gestione di stream di input/output e filesystem.
- `java.math`: calcolo a precisione arbitraria (`BigInteger`, `BigDecimal`).

- **Sintassi di Importazione:**

- Selettiva: `import java.util.Scanner;` (consigliata per chiarezza e pulizia).
- Wildcard su package: `import java.util.*;` (rende visibili tutte le classi pubbliche del package).
- *Regola del Classpath*: le classi definite dall'utente che risiedono nello stesso package e nella stessa cartella del Classpath non necessitano di alcuna direttiva `import`.

- **Classi di Sistema Notevoli:**

- `java.lang.System`:
 - * Tre stream predefiniti: `System.in` (standard input, byte stream da tastiera), `System.out` (standard output a video), `System.err` (standard error).
 - * `System.currentTimeMillis()`: restituisce un `long` con il timestamp **Unix Epoch** (millisecondi trascorsi dalle ore 00:00:00 UTC del 1° Gennaio 1970).
- `java.util.Scanner`:
 - * Parser per estrarre tipi primitivi e stringhe da flussi di testo:
 - `sc.nextLine()`: legge un'intera riga fino al terminatore `\n` e restituisce una `String`.
 - `sc.nextInt()`, `sc.nextFloat()`, `sc.nextDouble()`: leggono il token successivo effettuando il parsing del tipo. Se l'utente inserisce caratteri incompatibili, lancia un'eccezione (`InputMismatchException`).
 - * Buona norma: invocare `sc.close()` per rilasciare il descrittore del file o dello stream ed evitare *resource leak*.

6.1.2 La Classe String e le sue Operazioni (Slide 10–26)

- **Immutabilità:**

- Le istanze di `String` sono immutabili. Nessun metodo della classe altera i caratteri dell'oggetto esistente: qualunque operazione (es. `replace`, `substring`) restituisce **una nuova istanza di `String`** nello Heap.
- Tentare `str.charAt(1) = 'c'`; produce un **Compile-Time Error**.

- **Operazioni Fondamentali:**

- `str.length()`: numero di caratteri (nota: è un metodo con parentesi tonde, a differenza di `arr.length` per gli array).
- `str.charAt(int index)`: carattere in posizione `index` (0-indexed). Indici negativi o $\geq \text{length}()$ lanciano `StringIndexOutOfBoundsException`.
- `str.indexOf(char c)`: prima occorrenza del carattere (restituisce -1 se non presente).
- `str.substring(int start, int end)`: estrae la sottostringa nell'intervallo semi-aperto $[start, end)$, ovvero il carattere all'indice `end` è **escluso**. Se `end` è omissso (`str.substring(start)`), estrae fino al termine della stringa.
- `str.replace(CharSequence target, CharSequence replacement)`: sostituisce tutte le occorrenze da sinistra a destra.
- `String.format("Format: %s, %d", str, num)`: formattazione con sintassi analoga alla `printf` del C.

- **Confronto di Stringhe: `==` vs `.equals()`:**

- `s == t`: confronta l'**identità dei puntatori** (indirizzi nello Heap). È `true` solo se `s` e `t` puntano allo stesso identico oggetto.
- `s.equals(t)`: confronta l'**equivalenza semantica** dei contenuti carattere per carattere.
- **Lo String Constant Pool e l'ottimizzazione del compilatore:**

```
1 String s = "hello";
2 String t = "hello";
3 System.out.println(s == t); // Stampa TRUE!
```

Perché? Le costanti letterali vengono inserite nel *String Constant Pool*. Se due variabili dichiarano lo stesso letterale, la JVM riusa lo stesso identico oggetto in memoria per risparmiare RAM (*interning*). Tuttavia, se scriviamo `String r = new String("hello");`, costringiamo la JVM a creare un nuovo oggetto nello Heap: `s == r` sarà `false`! **Regola d'oro: usare SEMPRE `.equals()`.**

6.1.3 Stringhe Mutabili con `StringBuilder` (Slide 27–29)

- **Il problema delle prestazioni con `String`:**

- L'operatore `+` tra stringhe invoca internamente `String.valueOf()`. Se usato in un ciclo di N iterazioni, alloca $O(N)$ oggetti temporanei che intasano la memoria del Garbage Collector.

- **La soluzione: `StringBuilder`:**

- Mantiene un buffer interno ridimensionabile mutabile.
- Metodi principali: `.append(...)`, `.delete(start, end)`, `.insert(index, str)`, `.reverse()`.
- Supporta la **Fluent Notation (Method Chaining)** perché restituisce `this`.

6.2 Codice: Evoluzione del Codice: SimpleDate (Lezione 07)

Nel passaggio da Lezioni05-06 a Lezione07, il docente trasforma radicalmente la classe `Date.java` applicando i principi di Information Hiding e aggiunge per la prima volta una classe di test dedicata: `TestDate.java`.

6.2.1 [Approfondimento] Il Diff Concettuale di `Date.java`:

```
1  public class Date {
2  -   int day;
3  -   int month;
4  -   int year;
5  -   static final String FORMAT = "dd/mm/yyyy";
6  +   private int day;
7  +   private int month;
8  +   private int year;
9  +   static final String FORMAT_IT = "dd/mm/yyyy";
10 +   static final String FORMAT_US = "mm/dd/yyyy";
11 +   private byte lang; // 0: IT, 1: US
12
13   // Costruttore principale
14   public Date(int day, int month, int year) {
15       this.day = day;
16       this.month = month;
17       this.year = year;
18       verify();
19 +       lang = 0;
20   }
21
22 +   // Costruttore sovraccaricato: delega tramite this(...)
23 +   public Date(int day, int month) {
24 +       this(day, month, 2025); // default year
25 +   }
26
27 +   // Copy Constructor: clona lo stato da un'altra istanza
28 +   public Date(Date other) {
29 +       this.day = other.day;
30 +       this.month = other.month;
31 +       this.year = other.year;
32 +       verify();
33 +       lang = other.lang;
34 +   }
35
36 +   // Getters
37 +   public int getDay() { return day; }
38 +   public int getMonth() { return day; } // [Attenzione] Attenzione al refuso del prof!
39 +   public int getYear() { return day; } // [Attenzione] Ritorna 'day' anziché 'year'!
40 +   public byte getLang() { return lang; }
41
42 +   // Setters con validazione dell'invariante
43 +   public void setDay(int day) {
44 +       this.day = day;
45 +       verify();
46 +   }
47 +   public void setMonth(int month) {
48 +       this.month = month;
49 +       verify();
50 +   }
51 +   public void setYear(int year) {
52 +       this.year = year;
53 +       verify();
54 +   }
55 +   public void setLang(byte lang) {
```

```

56 +     if (lang == 1) this.lang = 1;
57 +     else this.lang = 0;
58 + }
59 +
60 + public String printFormat() {
61 +     if (lang == 0) return FORMAT_IT;
62 +     else return FORMAT_US;
63 + }
64
65     public String toString() {
66 -     return print();
67 +     if (lang == 0) return day + "/" + month + "/" + year;
68 +     else return month + "/" + day + "/" + year;
69     }

```

6.2.2 [Idea] Le Novità e le Finezze Implementative da Notare:

1. Incapsulamento e Protezione dello Stato (`private`):

- I campi non sono più accessibili direttamente dall'esterno (`d.day = -7`; genera ora un **Compile-Time Error** in `MainDate`).

2. Delega tra Costruttori con `this(...)`:

- Il costruttore `Date(int day, int month)` invoca `this(day, month, 2025);`.
- **Regola tassativa per l'esame:** la chiamata `this(...)` verso un altro costruttore deve essere **la prima istruzione assoluta** del costruttore, pena errore di compilazione («*Call to 'this()' must be first statement in constructor body*»).

3. Copy Constructor:

- `public Date(Date other)` permette di creare una nuova istanza duplicando i valori di un oggetto esistente, evitando l'aliasing accidentale dei puntatori.

4. Validazione Continua dell'Invariante nei Setter:

- Ogni volta che un setter muta lo stato (`setDay`, `setMonth`, `setYear`), invoca subito `verify()`. Lo stato dell'oggetto viene così monitorato costantemente.

5. Il Refuso di Copia-Incolla del Docente nei Getter:

- Nei getter di `Date.java` (righe 38–39 del codice ufficiale), il prof ha inavvertitamente scritto:

```

1 public int getMonth() { return day; }
2 public int getYear() { return day; }

```

Sia `getMonth()` che `getYear()` restituiscono il campo `day` anziché `month` e `year`! Nel tuo codice assicurati di restituire i campi corretti (`return month`; e `return year`).

6. Testing con Asserzioni e Flag `-ea` (`TestDate.java`):

```

1 public class TestDate {
2     private static Date date = new Date(1, 1, 1970);
3
4     private static void printFormatTest() {
5         date.setLang((byte) 1);
6         assert date.printFormat().equals(Date.FORMAT_US);
7     }

```

```
8
9     public static void main(String[] args) {
10         printFormatTest();
11     }
12 }
```

- **Concetto Chiave d'Esame:** La keyword `assert` verifica che una condizione booleana sia vera; se è falsa, solleva un `AssertionError`.
- **Attenzione:** Nella JVM le asserzioni sono **disabilitate di default**. Se lanci semplicemente `java TestDate`, l'istruzione `assert` viene ignorata. Per attivarle, è obbligatorio passare il flag `-ea` (*Enable Assertions*):

```
1 java -ea TestDate
```

- Nel tuo script `esercizi.sh`, infatti, il comando `./esercizi.sh test` usa proprio `-ea`!

6.3 □ Corrispondenza con la Struttura del tuo Workspace

Nel tuo repository `esercizi-wally-25-26`, hai modularizzato perfettamente questi argomenti: - **es02:** `MainScanner.java` copre l'uso di `java.util.Scanner` introdotto in T07. - **es04:** `MainString.java` testa tutti i metodi di `String` (`length`, `charAt`, `indexOf`, `substring`, `replace`, `constant pool ==` vs `.equals()`) e le mutazioni con `StringBuilder` (`append`, `delete`, `insert`, `reverse`). - **es05:** `Date.java` implementa l'overloading dei costruttori, il constructor chaining con `this(...)` e il copy constructor.

Nota di Studio

Con la **Lezione 07** abbiamo completato le librerie standard, le stringhe, i costruttori multipli e le asserzioni.

Il prossimo blocco è la **Lezione 08 / Slide T08: Class Constructors and Encapsulation**: - Studio approfondito della teoria su costruttori (default constructor, costruttore falso, copy constructor, overloading). - Analisi dettagliata dell'Information Hiding e dei **modificatori di accesso** (`public`, `protected`, `package-private` di default, `private`). - Evoluzione del codice in Lezione08: sostituzione del `byte lang` con un tipo enumerato ad hoc: `Language.java` (`enum Language { IT, US }`)! - Array costanti per i nomi dei mesi (`MONTHS_IT`, `MONTHS_US`) e nuovo metodo `getMonthAsString()`.

Dammi conferma quando vuoi procedere alla Lezione 08 / T08!

T08 — Costruttori, Inizializzazione ed Incapsulamento

Ciclo di Inizializzazione, Overloading dei Costruttori, Modificatori e SimpleDate v3

In questo blocco analizziamo in dettaglio la teoria dei costruttori, dell'incapsulamento e dell'information hiding ([T08 - Class Constructors and Encapsulation.pdf](#)), confrontandola con il codice ufficiale in [Lezione08](#), dove nasce l'enum `Language.java`, vengono aggiunti gli array dei mesi per la stampa estesa (`prettyPrint()`) e si introduce l'**Enhanced For (for-each)**.

7.1 Teoria: Concetti Teorici dalle Slide (Slide T08)

7.1.1 Il Ciclo di Vita e i Meccanismi dei Costruttori (Slide 1–12)

- Cosa avviene esattamente dietro le quinte con `newClazz()`:

1. La JVM calcola l'ingombro in byte di tutti i campi e alloca un blocco contiguo di memoria nello **Heap**.
2. Genera un riferimento (*reference*) univoco che punta a quell'area di memoria.
3. Inizializza tutti i campi ai rispettivi valori di default (0, false, null).
4. Invoca il **costruttore** designato passando implicitamente il puntatore `this`.
5. Restituisce l'indirizzo dell'oggetto alla variabile a sinistra dell'assegnamento.

- Caratteristiche Tassative di un Costruttore:

- Deve avere lo **stesso identico nome della classe**.
- **Non ha alcun tipo di ritorno** (nemmeno `void`!).
- Viene eseguito una sola volta, all'atto della creazione dell'istanza.

- Default Constructor (Costruttore di Default):

- Se il programmatore **non dichiara alcun costruttore**, il compilatore Java inserisce automaticamente un costruttore vuoto senza argomenti (`publicClazz(){ }`).
- [Attenzione] **Regola aurea d'esame**: se il programmatore dichiara *anche un solo* costruttore con parametri (es. `publicCar(Stringcolor)`), il compilatore **non genera più il costruttore di default**! Invocare `newCar()` senza argomenti provocherà un **Compile-Time Error**.

- False Constructor (Il Falso Costruttore — Trabocchetto d'Esame):

- Se si inserisce un tipo di ritorno (ad esempio `voidCar(Stringcolor)`):

```
1 public class Car {
2     void Car(String color) { ... } // [Attenzione] NON È UN COSTRUTTORE!
3 }
```

- Per il compilatore questo è un **normale metodo d'istanza** (avente incidentalmente lo stesso nome della classe). Il compilatore aggiungerà comunque il costruttore di default `car()`, e il metodo `voidCar(...)` non verrà mai invocato al momento del `new`!

- Copy Constructor:

- Costruttore che accetta un'altra istanza della medesima classe per clonarne lo stato (`Car(Car other)`). In Java non esiste la copia automatica bit-a-bit del C++: deve essere scritta esplicitamente dal programmatore.
 - **Constructor Overloading e Chaining con `this(...)`:**
 - Più costruttori con firme distinte.
 - La chiamata `this(...)` verso un costruttore fratello serve a evitare duplicazione di logica di validazione e **deve essere tassativamente la prima riga di codice nel corpo del costruttore**.
 - *Meme delle slide (p. 11):* Il matematico in lacrime che grida “*Abuse of notation*” vs il Chad Programmer che usa l'overloading per semplificare il codice.
 - **Distruttori e il Metodo `finalize()`:**
 - In Java non esiste la deallocazione manuale: non esistono distruttori (`~Car()`).
 - Esisteva il metodo protetto `protected void finalize()` invocato prima della rimozione da parte del Garbage Collector.
 - [Attenzione] **Attenzione:** `finalize()` è ufficialmente **deprecato da Java 9** in poi (e rimosso/reso no-op nelle versioni moderne) per l'imprevedibilità temporale dell'esecuzione del GC e rischi di deadlock. Non va mai utilizzato.
-

7.1.2 Incapsulamento, Information Hiding e Modificatori di Accesso (Slide 13–24)

- **Incapsulamento:** aggregazione fisica nello stesso file sorgente di dati (campi) e procedure (metodi).
 - **Information Hiding:** occultamento della rappresentazione interna dello stato.
 - *Meme della Cipolla (p. 18):* «*Most software is written like an onion: The more layers you peel back, the more you want to cry*». L'Information Hiding crea confini netti per proteggere il codice dal disfacimento.
 - *Meme di Gandalf (p. 24):* «*YOU SHALL NOT ACCESS MY MEMBERS*».
 - **I 4 Livelli di Visibilità in Java:**
 1. `private`: visibile esclusivamente all'interno della stessa classe.
 2. *Default (Package-Private)*: nessun modificatore; visibile all'interno della classe e a tutte le classi residenti nello stesso package.
 3. `protected`: visibile nel package e a tutte le sottoclassi derivate (anche in package diversi).
 4. `public`: visibile ovunque, da qualsiasi package del Classpath.
 - **Vantaggi Architetture di Getter e Setter:**
 - *Nei Setter:* possibilità di intercettare valori anomali, applicare regole di business e loggare le modifiche prima di mutare lo stato.
 - *Nei Getter:* possibilità di alterare la rappresentazione interna della memoria senza rompere il codice dei client (**Retro-compatibilità / Loose Coupling**). Ad esempio, cambiare il campo interno da `int age` a `short age` per dimezzare l'occupazione di RAM, mantenendo il getter `public int getAge(){ return age; }` (promozione implicita di tipo) senza che il mondo esterno debba ricompilare o cambiare una sola riga di codice.
-

7.1.3 Asserzioni e Testing (Slide 25–26)

- La parola chiave `assert`:

```
1 assert person.getAge() != 0 : "Age cannot be 0";
```

- Se l'espressione a sinistra valuta `false`, la JVM interrompe l'esecuzione e lancia un `AssertionError` mostrando il messaggio a destra dei due punti.
- Richiede obbligatoriamente l'attivazione a runtime con il flag JVM `-ea` (*Enable Assertions*):

```
1 java -ea TestPerson
```

7.2 Codice: Evoluzione del Codice: [Lezione08](#)

In [Lezione08](#) il docente applica un refactoring strutturale a `SimpleDate`:

7.2.1 Introduzione del Tipo Enumerato `Language.java`

Viene abbandonato il primitivo `byte lang` (dove 0 era IT e 1 era US, fragile e poco leggibile) a favore di un tipo `enum` fortemente tipizzato:

```
1 public enum Language {  
2     IT, US;  
3  
4     public String toString() {  
5         switch (this) {  
6             case IT: return "italiano";  
7             case US: return "american";  
8             default: return null;  
9         }  
10    }  
11 }
```

Finezza: Anche gli `enum` in Java sono tipi `reference` completi e possono sovrascrivere `toString()` per restituire una descrizione testuale appropriata.

7.2.2 Refactoring di `Date.java`

```
1 public class Date {  
2     - private byte lang; // 0: IT, 1: US  
3     + private Language lang;  
4     + private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile",  
5         "maggio", "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };  
6     + private static final String[] MONTHS_US = { "January", "February", "March", "April", "May",  
7         "June", "July", "August", "September", "October", "November", "December" };  
8  
9     public Date(int day, int month, int year) {  
10        this.day = day;  
11        this.month = month;  
12        this.year = year;  
13        verify();  
14        - lang = 0;  
15        + lang = Language.IT;  
16    }  
17 }
```

```

15
16 -   public void setLang(byte lang) { ... }
17 +   public void setAmerican() { lang = Language.US; }
18 +   public void setItalian() { lang = Language.IT; }
19
20 +   public String getMonthAsString() {
21 +       if (lang == Language.IT) return MONTHS_IT[month-1];
22 +       else return MONTHS_US[month-1];
23 +   }
24
25 +   public String prettyPrint() {
26 +       if (lang == Language.IT) return day + " " + MONTHS_IT[month-1] + " " + year;
27 +       else return MONTHS_US[month-1] + " " + day + ", " + year;
28 +   }

```

7.2.3 [Approfondimento] Dettagli Tecnici e Trabocchetti nel Codice di Date:

1. **Separazione Semantica dei Metodi di Configurazione:** Invece di un setter generico che accetta valori arbitrari, il prof definisce due metodi espliciti auto-esplicativi: `setItalian()` e `setAmerican()`.
2. **Array di Costanti di Classe (`private static final String[]`):** Gli elenchi dei nomi dei mesi sono `static final` (condivisi da tutte le istanze senza duplicare memoria nello Heap) e `private` (inaccessibili all'esterno). *Curiosità nel codice del prof:* nell'array `MONTHS_IT` c'è un piccolo refuso: "setembre" con una sola 't'.
3. **Persistenza del Refuso del Prof nei Getter:** Anche in questa lezione, alle righe 43–44 di `Date.java`:

```

1 public int getDay() { return day; }
2 public int getMonth() { return day; } // Ritorna day anziché month!
3 public int getYear() { return day; } // Ritorna day anziché year!

```

Un chiaro refuso di copia-incolla che non è stato corretto dal docente nel rilascio ufficiale. Nel proprio codice personale è doveroso restituire i rispettivi campi `month` e `year`.

4. `prettyPrint()` e Formattazione Culturale:

- In italiano: giorno, mese per esteso, anno (es. 7 gennaio 2025).
- In inglese/americano: mese per esteso, giorno, virgola, anno (es. February 20, 2025).

7.2.4 Novità in `MainDate.java`: Array di Oggetti ed Enhanced For

```

1 // Inizializzazione rapida di un array di oggetti nello Heap
2 Date[] dates = { d1, d2, d3, new Date(14, 2, 2024) };
3
4 // 1. Scansione all'indietro classica tramite indice
5 for (int i = dates.length - 1; i >= 0; i--)
6     System.out.println(dates[i].toString() + ": " + dates[i].getMonthAsString());
7
8 // 2. Scansione in avanti con Enhanced For (For-Each)
9 for (Date date : dates)
10    System.out.println(date.toString() + ": " + date.getMonthAsString());

```

- Il costrutto `for` (**Tipo elemento : collezione**) evita di manipolare manualmente indici di scorrimento, rendendo il codice più leggibile ed eliminando il rischio di errori di off-by-one o out-of-bounds.

7.3 □ Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository [esercizi-wally-25-26](#): - **es06**: `Person.java` implementa l'esempio accademico della persona con i metodi di validazione `verifyAge` e `verifyName`, incapsulamento rigoroso e gestione dei setter/getter. - **es07**: `Language.java` definisce l'enum `Language` sfruttando modernamente la *Switch Expression* di Java 14+ (`case IT -> "italiano";`).

Nota di Studio

Con la **Lezione 08** abbiamo completato l'intero ciclo di vita dei costruttori, l'incapsulamento dei dati, gli enum base e la gestione dei formati estesi.

Il prossimo blocco è la **Lezione 09 / Slide T09: *Arrays and Enumerative Types***: - Approfondimento teorico sistematico sugli **Array in Java** (dichiarazione, allocazione nello Heap, proprietà immutabile `.length`, array multidimensionali, copie superficiali vs profonde). - Analisi rigorosa degli **Enumerative Types (enum)**: costruttori di enum, campi interni, metodi `values()` e `ordinal()`. - Evoluzione del codice in Lezione09: - Potenziamento di `Language.java` con campi interni (`format`, `description`) e costruttore privato. - Riscrittura ed espansione di `Date.java`.

Dammi conferma per aprire la Lezione 09 / T09 e continuare!

T09 — Array, Matrici e Tipi Enumerativi (enum)

Array Mono e Multidimensionali, Enum Tipizzati, Immutabilità e Singleton in SimpleDate v4

In questo blocco approfondiamo la gestione a basso livello degli array nello Heap, i tipi enumerati (enum) ([T09 - Arrays and Enumerative Types.pdf](#)) e analizziamo il codice di [Lezione09](#), dove Date diventa una **classe immutabile** (campi `final` e rimozione dei setter) e compare per la prima volta un design pattern fondamentale: il **Singleton Pattern** implementato in [BirthDay.java](#).

8.1 Teoria: Concetti Teorici dalle Slide (Slide T09)

8.1.1 Gli Array in Java (Slide 1–13)

- **Natura degli Array:**

- Un array è una sequenza contigua e ordinata di variabili omogenee (dello stesso tipo), accessibili tramite indice numerico intero che parte da 0.
- In Java gli array **non sono tipi primitivi**: sono **oggetti speciali allocati nello Heap**.
- Possono memorizzare valori primitivi oppure **riferimenti/puntatori a oggetti**, mai oggetti per valore diretto.
- La dimensione di un array può essere stabilita dinamicamente a runtime all'atto dell'allocazione (`new int[size]`), ma **è immutabile una volta creata**.

- **Dichiarazione vs Creazione:**

- Sintassi raccomandata: `int[] arr;` (la notazione `int arr[]`; è ammessa per retrocompatibilità col C, ma sconsigliata).
- La dichiarazione `int[] arr;` alloca solo una variabile reference nello Stack (inizializzata a `null`), **senza riservare spazio per gli elementi nello Heap**.
- Allocazione:
 - * Tramite `new`: `float[] arr = new float[10];` (gli elementi assumono il valore di default del tipo, es. `0.0f`).
 - * Tramite inizializzazione statica: `int[] primes = {2, 3, 5, 7};` (la dimensione viene inferita automaticamente dal compilatore).

- **Il Pseudo-Campo `.length`:**

- Ogni array possiede la proprietà `public final int length`.
- [Attenzione] **Distinzione d'esame**: per gli array si scrive `arr.length` (**senza parentesi**, è una variabile `final`), mentre per le stringhe si invoca il metodo `str.length()` (**con le parentesi**).
- Se si tenta di leggere `arr.length` su un array nullo (`arr = null;`), la JVM lancia una `NullPointerException`.

- **Confronto e Stampa degli Array (I Trabocchetti Classici):**

1. `arr1 == arr2`: confronta solo gli indirizzi nello Heap. Se due array contengono gli stessi identici elementi ma risiedono in aree diverse, restituisce `false`.

2. `arr1.equals(arr2)`: **NON funziona**. Gli array non sovrascrivono il metodo `.equals()` di `Object`, quindi `equals` si comporta esattamente come `==`.
3. **Soluzione standard**: per confrontare il contenuto si usa il metodo statico `java.util.Arrays.equals(arr1, arr2)`.
4. `arr.toString()`: stampa la firma bytecode della reference (es. `[I@6ce253f1`, dove `[` indica un array e `I` indica il tipo `int`).
5. Per visualizzare gli elementi a video in formato leggibile: `java.util.Arrays.toString(arr)`.

- **Enhanced For Loop (For-Each, Java 5+):**

```
1 for (String arg : args) {  
2     System.out.println(arg);  
3 }
```

Sostituisce il `for` contatore eliminando il rischio di errori di *off-by-one* e *ArrayIndexOutOfBoundsException*.

- **Array Multidimensionali e Matrici Frastagliate (Jagged Arrays):**

- In Java non esistono matrici bidimensionali a blocco contiguo in stile Fortran/C: **un array multidimensionale è un array di riferimenti ad altri array**.
- *Conseguenze pratiche:*
 1. **Scambio di righe in tempo $O(1)$** : per invertire la prima e l'ultima riga di una matrice non serve copiare gli elementi uno per uno, basta scambiare i loro puntatori:

```
1 int[] temp = matrix[0];  
2 matrix[0] = matrix[matrix.length - 1];  
3 matrix[matrix.length - 1] = temp;
```

2. **Righe a lunghezza eterogenea (Jagged/Pyramid Arrays):**

```
1 int[][] pyramid = new int[3][];  
2 pyramid[0] = new int[1]; // Riga 0 ha 1 elemento  
3 pyramid[1] = new int[2]; // Riga 1 ha 2 elementi  
4 pyramid[2] = new int[3]; // Riga 2 ha 3 elementi
```

3. Per stampare matrici annidate, `Arrays.toString(matrix)` stampa gli indirizzi delle singole righe. Si deve iterare sulle righe o usare `java.util.Arrays.deepToString(matrix)`.

8.1.2 Tipi Enumerati (enum) (Slide 14–19)

- Introdotti in Java 5 con la keyword `enum` per gestire insiemi chiusi e prefissati di valori costanti (es. giorni della settimana, punti cardinali, lingue).
- **Regole e Caratteristiche:**
 - Possono essere dichiarati in un file `.java` dedicato (come classe autonoma) o all'interno di una classe come membro statico, ma **mai all'interno di un metodo**.
 - Non sono semplici interi o stringhe: sono classi speciali che estendono implicitamente `java.lang.Enum`. Il loro costruttore è privato e non può essere invocato con `new`.
 - **Metodi Predefiniti Fondamentali:**
 - * `e.name()`: restituisce il nome letterale della costante come `String` (es. `"WEST"`).
 - * `e.ordinal()`: restituisce la posizione ordinale intera partendo da 0 (es. 3).
 - * `EnumClass.values()`: restituisce un array contenente tutte le costanti dichiarate nell'`enum`, comodo per cicli `for-each`.

- **Confronto tra Enum: Perché usare == invece di .equals():**
 - La JVM garantisce che in memoria esista **una e una sola istanza** per ciascuna costante di un enum (*garanzia di unicità singleton*).
 - Pertanto, il confronto con == è perfettamente sicuro ed è **più robusto di .equals()**: se una variabile vale null, l'espressione `move == Direction.WEST` restituisce false senza errori, mentre `move.equals(...)` causerebbe un crash con `NullPointerException`!
- **Uso di Enum negli switch:**
 - All'interno dei blocchi case, **non si qualifica** il tipo: si scrive `case IT:` e non `case Language.IT:`.

8.2 Codice: Evoluzione del Codice: SimpleDate (Lezione 09)

In [Lezione09](#), il docente introduce due cambiamenti strutturali fondamentali:

8.2.1 Immutabilità di Date.java

```
1 public class Date {
2     - private int day;
3     - private int month;
4     - private int year;
5     + private final int day;
6     + private final int month;
7     + private final int year;
8
9     // Costruttori...
10
11     - public void setDay(int day) { ... }
12     - public void setMonth(int month) { ... }
13     - public void setYear(int year) { ... }
```

- **Campi final:** Una volta inizializzati nel costruttore, day, month e year non possono più essere riassegnati.
- **Rimozione dei Setter:** I metodi mutatori setDay, setMonth e setYear vengono eliminati.
- **Pattern Immutabile:** Un oggetto Date è ora a prova di manomissione. Se un programma vuole calcolare “il giorno successivo”, non può mutare l’oggetto esistente, ma **deve istanziare un nuovo oggetto Date** con le coordinate aggiornate (esattamente come avviene per le String).

8.2.2 Il Design Pattern Singleton: Birthday.java

```
1 public class Birthday {
2     private static Date date = new Date(1, 1, 1970);
3     private static Birthday instance; // Riferimento all'unica istanza
4
5     // 1. Costruttore privato: impedisce istanziazioni esterne con 'new Birthday()'
6     private Birthday() { }
7
8     // 2. Metodo statico di fabbrica con Lazy Initialization
9     public static Birthday getInstance() {
10         if (instance == null)
11             instance = new Birthday();
12         return instance;
13     }
```

```

13     }
14
15     public Date getDate() {
16         return date;
17     }
18 }

```

8.2.3 [Approfondimento] Analisi Architetture del Singleton:

- **Scopo del Pattern:** Garantire che per tutta la durata dell'applicazione esista **una e una sola istanza** di una classe nello Heap e fornire un punto di accesso globale ad essa.
- **I Tre Pilastri del Singleton in Java:**
 1. **Costruttore privato:** blocca l'accesso a `new Birthday()` da qualsiasi altra classe (tentare di chiamarlo genera un errore a compile-time).
 2. **Campo statico privato (instance):** contiene l'unico riferimento all'oggetto.
 3. **Metodo factory pubblico statico (getInstance()):** implementa la *Lazy Initialization* (crea l'oggetto solo alla prima richiesta, riusandolo per tutte le successive).

8.2.4 Verifica in MainDate.java e il Bug dei Getter

```

1 Birthday bDay = Birthday.getInstance();
2 // Birthday day1 = new Birthday(); // COMPILE-TIME ERROR!
3 Birthday day1 = Birthday.getInstance();
4 System.out.println("bDay ?= day1: " + (bDay == day1)); // STAMPA TRUE!
5
6 Date today = new Date(27, 10, 2025);
7 // today.setDay(today.getDay() + 1); // COMPILE-TIME ERROR: l'oggetto è immutabile!
8 Date tomorrow = new Date(today.getDay() + 1, today.getMonth(), today.getYear());

```

Attenzione / Trappola d'Esame

Il Bug Persistente dei Getter e l'Effetto a Runtime: Anche in Lezione09, Date.java ha ancora il refuso di copia-incolla alle righe 43–44:

```

1 public int getMonth() { return day; } // Restituisce il campo day!
2 public int getYear() { return day; } // Restituisce il campo day!

```

Quando MainDate crea tomorrow invocando today.getMonth() su una data con day = 27, ottiene 27 come mese! Di conseguenza, il costruttore riceve (28, 27, 27) e verify() stampa: Illegal date! tomorrow: 28/27/27 Nel tuo codice è opportuno correggere questi getter (return month; e return year;).

8.3 □ Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository [esercizi-wally-25-26](#): - Nel modulo `es09`: hai formalizzato questo esatto pattern creando `SingletonPattern.java` con costruttore privato e metodo factory `getInstance()`.

Nota di Studio

Con la **Lezione 09** abbiamo consolidato la manipolazione avanzata degli array, le proprietà degli enum, l'immutabilità e il pattern Singleton.

Il prossimo blocco è la **Lezione 10 / Slide T10: *Java Variables and Call Stack***: - Modello formale della memoria JVM: **Call Stack (Stack Frames)** vs **Heap (Objects & Class Data)**. - Variabili d'istanza, variabili di classe (`static`), variabili locali e parametri di metodo. - Il passaggio dei parametri per valore (*pass-by-value*) per tipi primitivi vs reference types. - Il blocco di inizializzazione statica (`static { ... }`). - **Il grande spartiacque del corso**: il debutto del progetto **MavenDate**, con l'adozione ufficiale di Apache Maven, `pom.xml`, layout directory standard e primo packaging!

Dammi conferma per aprire la Lezione 10 / T10 e analizzare Call Stack e MavenDate!

T10 — Variabili, Scope, Call Stack e Memoria Heap

Stack Frame, Passaggio dei Parametri per Valore, Garbage Collection e Debutto di MavenDate

In questo blocco analizziamo in profondità il modello di memoria della JVM (Stack vs Heap), il passaggio dei parametri per valore, gli oggetti immutabili e i tipi wrapper ([T10 - Java Variables and Call Stack.pdf](#)). Sul fronte pratico analizziamo il **grande spartiacque architetturale del corso**: il passaggio dal progetto manuale a pacchetti [SimpleDate](#) al progetto ufficiale [MavenDate](#) con Apache Maven, file `pom.xml`, layout `src/main/java` e collisione di nomi tra package diversi.

9.1 Teoria: Concetti Teorici dalle Slide (Slide T10)

9.1.1 Il Modello di Memoria della JVM: Stack vs Heap (Slide 1–11)

La memoria a disposizione di un processo Java è partizionata in tre aree principali: 1. **Call Stack (Stack dei Thread)**: - Memoria estremamente veloce, gestita a politica LIFO (Push/Pop). - È suddivisa in **Stack Frame** (o record di attivazione): ogni invocazione di metodo crea un nuovo frame nello stack contenente: - La tabella delle variabili locali. - I parametri formali passati al metodo. - Lo stack degli operandi (per i calcoli intermedi della CPU/bytecode). - I riferimenti alla costante del metodo e l'indirizzo di ritorno. - Quando il metodo termina (con `return` o eccezione), il frame viene rimosso istantaneamente dallo Stack e tutte le sue variabili locali vengono distrutte. 2. **Heap (Memoria Dinamica degli Oggetti)**: - Memoria globale condivisa da tutti i thread dell'applicazione. - Ospita tutti gli **oggetti** istanziati con `new`, inclusi gli **array**. - I dati nello Heap non si deallocano all'uscita dal metodo: sopravvivono finché esiste almeno un riferimento attivo nello Stack o in altri oggetti che punta ad essi. Quando diventano irraggiungibili (*unreachable*), vengono eliminati dal Garbage Collector. 3. **Method Area / Metaspace (Memoria di Classe)**: - Ospita il bytecode compilato delle classi caricate dal ClassLoader, i metadati di tipo e le **variabili statiche** (`static`). - **Blocco di Inizializzazione Statica** (`static { ... }`): - Blocco speciale eseguito **una sola volta** nel ciclo di vita dell'applicazione, al momento in cui la classe viene caricata in memoria dalla JVM: `java static int[] arr; static { arr = new int[3]; arr[0] = 1; arr[1] = 2; arr[2] = 0; }` - Si usa per inizializzare strutture statiche complesse che richiedono cicli o controlli prima dell'uso.

9.1.2 Immutabilità e Passaggio Parametri: Pass-by-Value (Slide 15–25)

- **Oggetti Immutabili (Immutable Objects):**

- Un oggetto il cui stato interno non può essere modificato dopo la costruzione (es. `String`, `Integer`, `Date` da Lezione 09).
- *Regole per creare una classe immutabile:*
 1. Rendere tutti i campi `private final`.
 2. Non fornire metodi mutatori (nessun setter).
 3. Dichiarare la classe `final` (per impedire che le sottoclassi violino l'immutabilità con `override` malevoli).
 4. Se l'oggetto contiene riferimenti a strutture mutabili (es. un array o un'altra data mutabile), eseguire copie difensive (*defensive copy*) nel costruttore e nei getter.

- *Vantaggi*: Immuni da side-effects da aliasing, intrinsecamente *thread-safe* (possono essere condivisi tra thread concorrenti senza sincronizzazione o lock).
- **Come Java passa i parametri ai metodi: SEMPRE Strictly Pass-by-Value:**
 - In Java **non esiste il passaggio per riferimento del C++** (`int& x`).
 - **Per i tipi primitivi** (`int`, `double`): il metodo riceve una copia del valore binario. Qualsiasi modifica locale sul parametro non ha alcun effetto sulla variabile del chiamante.
 - **Per i tipi reference (oggetti/array)**: il metodo riceve **una copia per valore del riferimento** (dell'indirizzo puntato):
 - * *Caso 1 (Mutazione dell'oggetto)*: se usiamo il riferimento ricevuto per chiamare un metodo mutatore (`p.setAge(25)`), l'oggetto puntato nello Heap viene effettivamente modificato per il chiamante!
 - * *Caso 2 (Riassegnamento del puntatore — Il tranello d'esame)*:

```
1 void reset(Person p) {
2     p = new Person("Bob"); // Assegna un nuovo oggetto al puntatore locale!
3 }
```

All'esterno, la variabile del chiamante **non cambia affatto!** Ha semplicemente mutato la copia locale del puntatore nel proprio stack frame.

9.1.3 Classi Wrapper, Autoboxing e i loro Trabocchetti (Slide 26–35)

- Per ciascuno degli 8 tipi primitivi, Java mette a disposizione una corrispondente **Wrapper Class** nel package `java.lang`:
 - `byte` → `Byte`
 - `short` → `Short`
 - `int` → `Integer`
 - `long` → `Long`
 - `float` → `Float`
 - `double` → `Double`
 - `char` → `Character`
 - `boolean` → `Boolean`
- Tutte le classi wrapper sono **immutabili**.
- **Autoboxing e Unboxing (Java 5+)**:
 - *Autoboxing*: conversione automatica da primitivo a wrapper (`Integer x = 5;` ⇒ compilato come `Integer.valueOf(5)`).
 - *Unboxing*: estrazione automatica del valore primitivo dal wrapper (`int y = x;` ⇒ compilato come `x.intValue()`).
- [Attenzione] **I Due Grandi Pericoli delle Wrapper Classes:**

1. Il Trabocchetto dell'Integer Cache (−128 ⋯ + 127):

```
1 Integer a = 100, b = 100;
2 System.out.println(a == b); // TRUE! (La JVM ricicla lo stesso oggetto cacheato)
3
4 Integer c = 200, d = 200;
5 System.out.println(c == d); // FALSE! (Fuori range [-128, 127], alloca 2 oggetti
   distincti nello Heap!)
```

Conclusione: **Mai usare == per confrontare i wrapper**, usare sempre `.equals()`!

2. Crash da Unboxing su null:

```
1 Integer obj = null;
2 int val = obj; // RUNTIME ERROR: NullPointerException!
```

La JVM tenta di invocare `obj.intValue()`, fallendo rovinosamente se `obj` è nullo.

9.2 Codice: Evoluzione del Codice: Il Passaggio a MavenDate

In [Lezione10](#), il docente archivia il vecchio progetto non strutturato e adotta lo standard industriale **Apache Maven**.

9.2.1 Riorganizzazione dei Package

Il progetto viene suddiviso in due package distinti: - `it.oop.core`: il dominio applicativo (le classi di modello: `Date.java`, `Language.java`, `BirthDay.java`). - `it.oop.ui`: l'interfaccia utente contenente il main (`MainDate.java`).

9.2.2 Il Test Didattico sulla Collisione dei Nomi di Package

Nel package `it.oop.ui`, il docente aggiunge appositamente una seconda classe: `it/oop/ui/Date.java`:

```
1 package it.oop.ui;
2
3 class Date { // Visibilità package-private
4     public int time;
5     private int start = 0;
6
7     public Date(int time) { this.time = time; }
8 }
```

E in `MainDate.java` mostra come la JVM risolve l'ambiguità:

```
1 package it.oop.ui;
2
3 import it.oop.core.Date; // 1. L'import esplicito ha la precedenza!
4
5 public class MainDate {
6     public static void main(String[] args) {
7         Date d1 = new Date(7, 1, 2025); // Istanza it.oop.core.Date!
8
9         // 2. Per istanziare la classe Date del package it.oop.ui,
10        // è obbligatorio usare il Fully Qualified Name (FQN):
11        it.oop.ui.Date d = new it.oop.ui.Date(12345);
12        System.out.println(d.time); // OK: 'time' è public
13        // System.out.println(d.start); // COMPILE-TIME ERROR: 'start' è private!
14    }
15 }
```

9.2.3 Anatomia del pom.xml di MavenDate

```
1 <project ...>
2     <modelVersion>4.0.0</modelVersion>
3     <groupId>it.oop</groupId>
4     <artifactId>MavenDate</artifactId>
5     <version>1.0-SNAPSHOT</version>
6
7     <properties>
```

```

8      <maven.compiler.source>17</maven.compiler.source>
9      <maven.compiler.target>17</maven.compiler.target>
10     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
11   </properties>
12
13   <build>
14     <plugins>
15       <!-- Plugin per creare il JAR eseguibile con dipendenze -->
16       <plugin>
17         <groupId>org.apache.maven.plugins</groupId>
18         <artifactId>maven-assembly-plugin</artifactId>
19         <version>3.7.1</version>
20         <configuration>
21           <descriptorRefs>
22             <descriptorRef>jar-with-dependencies</descriptorRef>
23           </descriptorRefs>
24           <archive>
25             <manifest>
26               <addClasspath>>true</addClasspath>
27               <!-- Entrypoint del JAR -->
28               <mainClass>it.oop.ui.MainDate</mainClass>
29             </manifest>
30           </archive>
31         </configuration>
32         <executions>
33           <execution>
34             <id>assemble-all</id>
35             <phase>package</phase>
36             <goals>
37               <goal>single</goal>
38             </goals>
39           </execution>
40         </executions>
41       </plugin>
42     </plugins>
43   </build>
44 </project>

```

Questo file è il prototipo esatto che hai esteso in tutti i moduli del tuo workspace `esercizi-wally-25-26`:

- `compile`: compila da `src/main/java` verso `target/classes`.
- `test`: compila ed esegue i test da `src/test/java` verso `target/test-classes`.
- `package`: genera il JAR auto-eseguibile con `META-INF/MANIFEST.MF` configurato con `Main-Class: it.oop.ui.MainDate`.

9.3 □ Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository `esercizi-wally-25-26`:

- **es08**: `Memory.java` implementa l'allocazione Stack vs Heap e il blocco static { ... }.
- **es10**: `ImmutablePerson.java` applica formalmente tutte le regole di immutabilità di T10 (final class, campi final, validazione e assenza di setter).

Nota di Studio

Con la **Lezione 10** abbiamo completato l'intero quadro su memoria, call stack, tipi wrapper e la transizione a Maven.

Il prossimo blocco è la **Lezione 11 / Slide T11: Packages and Class Visibility**: - Tassonomia gerarchica dei package e convenzioni DNS invertite (`it.univr...`). - Regole di visibilità e accessibilità inter-

package: membri public, protected, package-private e private. - Evoluzione di MavenDate: - Refactoring dell'algoritmo Gregoriano: introduzione definitiva del **calcolo dell'anno bisestile (isLeapYear)** in Date.java! - Integrazione di isLeapYear(year) in daysPerMonth(month, year) e fine dei mesi errati per Febbraio!

Dammi conferma per procedere alla Lezione 11 / T11!

T11 — Package, Moduli e Regole di Visibilità

Struttura Cartelle, Modificatori public/protected/default/private e MavenDate v2 con equals()

In questo blocco analizziamo la modularità avanzata offerta dai package, le regole di risoluzione dei nomi e i livelli di visibilità dei membri ([T11 - Packages and Class Visibility.pdf](#)). Sul piano pratico, in [Lezione11/MavenDate](#) assistiamo a una grande svolta architetturale: 1. Nascono le sottoclassi specializzate `ItalianDate` e `AmericanDate` tramite estensione (`extends Date`). 2. Viene introdotto il modificatore `protected`. 3. Viene implementato l'algoritmo canonico per l'overriding del metodo `equals(Object)`. 4. Viene risolto il bug storico dei getter `getMonth()` e `getYear()`.

10.1 Teoria: Concetti Teorici dalle Slide (Slide T11)

10.1.1 Modularità, Gerarchia e Convenzioni dei Package (Slide 1–11)

- **Definizione di Package:**

- Un package è un raggruppamento logico di classi correlate che scala il concetto di incapsulamento a livello di libreria.
- Definisce un **namespace isolato**: classi con lo stesso nome possono coesistere senza conflitti se collocate in package differenti (es. `it.oop.ui.Date` VS `it.oop.core.Date`).

- **Convenzione di Naming Standard:**

- Si usa il nome di dominio Internet invertito in caratteri minuscoli: `it.univr.mypackage`, `it.oop.core`.

- **Dichiarazione e Corrispondenza col Filesystem:**

- La direttiva `package nomepkg;` deve essere **la prima istruzione non di commento** all'inizio del file `.java`.
- Java impone una corrispondenza 1-a-1 tra la gerarchia dei package e la struttura delle cartelle su disco: la classe `it.oop.core.Date` deve trovarsi nel percorso `it/oop/core/Date.java`.

- **Sotto-Package e Wildcard:**

- I package possono essere annidati (es. `java.awt.event` è un sottopackage di `java.awt`).
- [Attenzione] **Regola d'esame**: la direttiva `import pkg.*;` importa solo le classi direttamente contenute in `pkg`. **Non importa le classi contenute nei suoi sotto-package** (per importare `java.awt.event` è obbligatorio scrivere `import java.awt.event.*;`).

- **Il Default Package (Unnamed Package):**

- Se un file `.java` omette la dichiarazione `package`, la classe appartiene al package predefinito senza nome.
- **Limitazione critica**: Le classi nel default package **non possono essere importate né utilizzate** da classi situate in package con nome. Nel software professionale e agli esami il suo uso è fortemente scoraggiato.

- **Compilazione e Packaging con Strumenti Standard e Maven:**

- `javac -d bin/ src/pkg/MyClass.java`: il flag `-d` crea automaticamente l'albero delle sottocartelle nella directory di destinazione `bin/`.
- Con Maven: il layout `src/main/java` e `src/test/java` unito al `pom.xml` automatizza compilazione, test surefire e packaging in Fat JAR.

10.1.2 Visibilità di Classe e Matrice di Accesso dei Membri (Slide 12–19)

- **Visibilità a Livello di Classe (Top-Level Classes):**

- `public classClazz`: accessibile e istanziabile da qualsiasi package del Classpath.
- `classClazz` (*Package-Private* / default): visibile e utilizzabile **esclusivamente all'interno dello stesso package**. Non può essere usata all'esterno, anche se i suoi metodi interni sono dichiarati `public`.
- *Nota*: le classi di primo livello non possono essere dichiarate `private` né `protected` (questi modificatori sono ammessi solo per classi annidate/interne).

- **La Matrice dei Modificatori di Accesso (Fields e Methods):**

Modificatore	Stessa Classe	Stesso Package	Sottoclasse (Altro Package)	Mondo Esterno
private	[OK]			
Default (<i>package-private</i>)	[OK]	[OK]		
protected	[OK]	[OK]	[OK]	
public	[OK]	[OK]	[OK]	[OK]

- **Risoluzione dei Nomi:**

- Se una classe importa due tipi omonimi o usa una classe locale avente lo stesso nome di una classe importata, per disambiguare è obbligatorio usare il **Fully Qualified Name (FQN)**: `it.oop.ui.Date`
`d = new it.oop.ui.Date(12345);`

10.2 Codice: Evoluzione del Codice: [Lezione11/MavenDate](#)

In *Lezione11*, il codice di *MavenDate* compie un balzo in avanti con l'introduzione di gerarchie di classi e l'overriding:

10.2.1 Generalizzazione in *Date.java*

```

1 package it.oop.core;
2
3 public class Date {
4     // 1. Campo 'protected': accessibile direttamente dalle sottoclassi
5     protected final int day;
6     // 2. Campi 'private': incapsulati e accessibili solo tramite getter
7     private final int month;
8     private final int year;
9 }

```



```

10 public Date(int day, int month, int year) {
11     this.day = day;
12     this.month = month;
13     this.year = year;
14     verify();
15 }
16
17 // 3. I GETTER SONO STATI CORRETTI!
18 public int getDay() { return day; }
19 public int getMonth() { return month; } // Non restituisce più day!
20 public int getYear() { return year; } // Non restituisce più day!
21
22 // 4. Implementazione canonica di equals(Object)
23 @Override
24 public boolean equals(Object other) {
25     if (other == null) return false; // Controllo null-safety
26     if (this == other) return true; // Controllo identità (stesso indirizzo)
27     if (!(other instanceof Date)) return false; // Controllo di tipo/ereditarietà
28     Date otherAsDate = (Date) other; // Downcast sicuro
29     return day == otherAsDate.getDay() // Confronto dei campi
30         && month == otherAsDate.getMonth()
31         && year == otherAsDate.getYear();
32 }
33 }

```

10.2.2 [Approfondimento] I 5 Passi Fondamentali dell'Algoritmo equals(Object):

1. `if (other == null) return false;`: garantisce che `d.equals(null)` ritorni `false` senza lanciare `NullPointerException`.
2. `if (this == other) return true;`: ottimizzazione immediata (riflessività). Se sono lo stesso oggetto nello Heap, sono identici.
3. `if (!(other instanceof Date)) return false;`: verifica se l'oggetto passato è compatibile con la gerarchia `Date`.
4. `Date otherAsDate = (Date) other;`: downcast necessario perché la firma di `equals` accetta un generico `Object`.
5. Confronto logico dei campi che definiscono lo stato: `day`, `month`, `year`.

10.2.3 Le Nuove Sottoclassi: `ItalianDate.java` e `AmericanDate.java`

```

1 package it.oop.core;
2
3 public class ItalianDate extends Date {
4     private static final String FORMAT = "dd/mm/yyyy";
5     private static final String[] MONTHS = { "gennaio", "febbraio", ... };
6
7     // Invocazione del costruttore genitore con 'super'
8     public ItalianDate(int day, int month, int year) {
9         super(day, month, year);
10    }
11
12    public String printFormat() { return FORMAT; }
13
14    public String prettyPrint() {
15        // 'day' è accessibile direttamente perché 'protected'!
16        // 'getYear()' è invocato tramite metodo perché 'year' è 'private' in Date!
17        return day + " " + getMonthAsString() + " " + getYear();
18    }
19 }

```

```

20     @Override
21     public String toString() {
22         return day + "/" + getMonth() + "/" + getYear();
23     }
24 }

```

E simmetricamente per `AmericanDate`:

```

1  public class AmericanDate extends Date {
2      private static final String FORMAT = "mm/dd/yyyy";
3
4      public AmericanDate(int day, int month, int year) {
5          super(day, month, year);
6      }
7
8      @Override
9      public String toString() {
10         // Mese prima del giorno!
11         return getMonth() + "/" + getDay() + "/" + getYear();
12     }
13 }

```

10.2.4 Polimorfismo, Upcasting e Downcasting in `MainDate.java`

Analizziamo i comportamenti a runtime testati dal docente:

```

1  Date date = new Date(3, 11, 2025);
2  ItalianDate itDate = new ItalianDate(3, 11, 2025);
3  AmericanDate usDate = new AmericanDate(3, 11, 2025);
4
5  // 1. Confronti con equals():
6  System.out.println("date ?= itDate: " + date.equals(itDate)); // Stampa TRUE!
7  System.out.println("itDate ?= usDate: " + itDate.equals(usDate)); // Stampa TRUE!

```

Perché stampano true? Perché sia `itDate` che `usDate` estendono `Date`: l'operatore `instanceof` `Date` è soddisfatto, e i campi (giorno 3, mese 11, anno 2025) coincidono perfettamente!

```

1  // 2. Upcasting (Polimorfismo di Inclusione):
2  Date d = new ItalianDate(1, 1, 1970); // LECITO: ItalianDate È UN Date!
3
4  // 3. Tipi Incompatibili tra rami fratelli:
5  // AmericanDate ad = new ItalianDate(1, 1, 1970); // COMPILE-TIME ERROR!
6
7  // 4. Downcasting esplicito lecito:
8  ItalianDate id = (ItalianDate) d; // LECITO a runtime: l'oggetto nello Heap è davvero ItalianDate!
9  System.out.println(id.printFormat()); // Stampa dd/mm/yyyy
10
11 // 5. Downcasting illegale a runtime:
12 // ItalianDate idFalso = (ItalianDate) date; // CRASH RUNTIME: ClassCastException!
13 // (perché 'date' è un Date generico, non contiene la specializzazione ItalianDate)

```

10.3 ☐ Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository [esercizi-wally-25-26](#): - **es13**: `Calculator.java` e `TestCalculator.java` applicano la separazione `es13.model` e `es13` illustrata a lezione (Slide T11, pp. 3–14). - **es14**: implementa esattamente questa architettura a oggetti multi-package (`it.oop.core` e `it.oop.ui`) con `ItalianDate`, `AmericanDate`, `Date` e i relativi test in `src/test/java/it/oop/core/TestItalianDate.java`.

Nota di Studio

Con la **Lezione 11** abbiamo visto l'introduzione dell'ereditarietà (`extends`), la visibilità `protected`, l'overriding di `equals()` e le regole di cast tra classi.

Il prossimo blocco è la **Lezione 12 / Slide T12: *Inheritance and Polymorphism***: - Teoria approfondita sull'ereditarietà: riuso, estensione e specializzazione. - La classe radice universale `java.lang.Object` e i suoi metodi (`toString`, `equals`, `hashCode`, `getClass`, `clone`). - Il costrutto `super`: invocazione di costruttori (`super(...)`) e invocazione di metodi della superclasse (`super.metodo()`). - **Polimorfismo e Dynamic Method Dispatch (Late Binding)**: come la JVM decide a runtime quale metodo eseguire tramite la *vtable*. - Regole di overriding vs overloading e l'annotazione `@Override`. - Evoluzione del codice in Lezione12: - La classe `Date` diventa la superclasse polimorfica di riferimento. - Aggiunta del metodo `isLeapYear(int year)` per la validazione completa dei bisestili gregoriani!

Dammi conferma per aprire la Lezione 12 / T12 e proseguire!

T12 — Ereditarietà, Dynamic Binding e Polimorfismo

Estensione di Classi, super, Overriding vs Overloading, Polimorfismo e MavenDate v3

In questo blocco analizziamo in modo esaustivo i due pilastri più importanti dell'OOP in Java: l'**Ereditarietà** e il **Polimorfismo** ([T12 - Inheritance and Polymorphism.pdf](#)). Nel codice di [Lezione12/MavenDate](#), la gerarchia evolve introducendo la classe astratta intermedia `FormattedDate`, l'interfaccia `Time`, la classe estesa `TimeStamp` e l'interfaccia standard `Comparable` per l'ordinamento naturale degli array.

11.1 Teoria: Concetti Teorici dalle Slide (Slide T12)

11.1.1 Ereditarietà di Classe (`extends`) e Riuso (Slide 1–17)

- **Relazione “IS-A” vs “HAS-A”:**

- *Ereditarietà (“IS-A”)*: un'auto è un veicolo (`Car extends Vehicle`); una data italiana è una data (`ItalianDate extends Date`).
- *Composizione (“HAS-A”)*: un'auto ha un motore (`Car` ha un campo `Engine`); una persona ha una data di nascita (`Person` ha un campo `Date`).

- **Ereditarietà Singola in Java:**

- In Java **non esiste l'ereditarietà multipla tra classi** (a differenza del C++). Una classe può estendere al massimo **una sola classe genitore diretta** (`extends SuperClass`). Questo previene il cosiddetto *Diamond Problem*.

- **Overriding vs Overloading:**

- *Method Overloading*: metodi nella stessa classe con lo stesso nome ma parametri diversi (risolto staticamente a compile-time).
- *Method Overriding*: una sottoclasse riscrive l'implementazione di un metodo ereditato mantenendo **la stessa identica segnatura** (nome, tipi dei parametri, ordine) e un tipo di ritorno identico o covariante.
- **L'Annotazione `@Override`**: Non è sintatticamente obbligatoria per far funzionare l'overriding, ma è una best practice fondamentale: dice al compilatore di verificare che quel metodo esista davvero nella superclasse, intercettando errori di battitura (es. scrivere per sbaglio `toString()` con la 's' minuscola verrebbe segnalato come errore).

- **Costruttori ed Ereditarietà: La Parola Chiave `super`:**

- [Attenzione] **I costruttori NON vengono mai ereditati dalle sottoclassi.**
- All'interno del costruttore di una sottoclasse, la prima istruzione deve essere una chiamata a `super(...)` oppure a un altro costruttore con `this(...)`.
- Se il programmatore non scrive nulla, il compilatore inserisce automaticamente `super();` (invocazione del costruttore vuoto della superclasse).
- [Attenzione] **Trappola d'esame**: Se la superclasse non possiede un costruttore senza parametri (perché ne ha definito uno con parametri e non ha aggiunto quello di default), omettere `super(...)` genera un **Compile-Time Error!**

- `super.metodo()` permette inoltre di invocare l'implementazione originaria della superclasse, bypassando l'override locale.
- **Classi e Metodi `final`:**
 - `final class`: non può essere estesa da nessun'altra classe (es. `String`, `Integer`).
 - `final method`: non può essere sovrascritto (*overridden*) da nessuna sottoclasse.

11.1.2 La Classe Radice Universale: `java.lang.Object` (Slide 18–31)

- «*The One Ring*»: in Java qualsiasi classe estende implicitamente `java.lang.Object`. Se una classe non specifica `extends`, estende direttamente `Object`.
- **I Metodi Universali di `Object`:**
 - `public String toString()`: rappresentazione testuale (`NomeClasse@hashCode`).
 - `public boolean equals(Object obj)`: confronto semantico (di default confronta l'identità con `==`).
 - `public int hashCode()`: valore hash associato all'oggetto.
 - `public final Class<?> getClass()`: restituisce il descrittore a runtime del tipo della classe.
- **Il Contratto Formale di `equals`:** Ogni implementazione di `equals` deve rispettare 5 proprietà matematiche:
 1. *Riflessiva*: `x.equals(x)` è sempre `true`.
 2. *Simmetrica*: `x.equals(y)` ritorna `true` se e solo se `y.equals(x)` ritorna `true`.
 3. *Transitiva*: se `x.equals(y)` è `true` e `y.equals(z)` è `true`, allora `x.equals(z)` è `true`.
 4. *Consistente*: invocazioni multiple producono lo stesso risultato finché lo stato non muta.
 5. *Non-Nullità*: per qualsiasi `x != null`, la chiamata `x.equals(null)` **deve restituire `false`** senza lanciare eccezioni.

11.1.3 Polimorfismo e Dynamic Method Dispatch (Late Binding) (Slide 32–50)

- **Principio di Sostituzione di Liskov (LSP):** Se S è sottotipo di T , qualsiasi porzione di codice scritta per interagire con T deve poter operare in modo trasparente e corretto con un'istanza di S .
- **Tipo Statico vs Tipo Dinamico:**

```
1 Date d = new ItalianDate(1, 1, 1970);
```

- **Tipo Statico (a Compile-Time):** è il tipo dichiarato della variabile reference (`Date`). Il compilatore accetta solo invocazioni a metodi e accessi a campi definiti nella classe `Date`.
- **Tipo Dinamico (a Runtime):** è il tipo effettivo dell'oggetto allocato nello Heap (`ItalianDate`).
- **Dynamic Method Dispatch (Late Binding):** Quando a runtime viene eseguita una chiamata a un metodo d'istanza (es. `d.toString()`), la JVM consulta la **Virtual Method Table (vtable)** dell'oggetto concreto nello Heap e seleziona **la versione del metodo definita dal Tipo Dinamico!** Non importa che `d` sia dichiarata come `Date`: verrà eseguito il `toString()` specializzato di `ItalianDate`.
- **Regole di Conversione di Tipo (Casting tra Oggetti):**
 - **Upcasting (verso la superclasse):** `Date d = new ItalianDate(...);` $\Rightarrow$ **Implicito, automatico e sempre sicuro al 100%.**

- **Downcasting (verso la sottoclasse):** `ItalianDate id = (ItalianDate)d;` $\Rightarrow$ **Esplicito obbligatorio.**
 - * Se l'oggetto concreto nello Heap è effettivamente un'istanza compatibile, il cast ha successo.
 - * Se l'oggetto concreto nello Heap è di tipo diverso (es. un `Date` generico o un `AmericanDate`), la JVM interrompe l'esecuzione e lancia una **`ClassCastException`**!
- **Downcasting Sicuro con `instanceof`:** Per evitare crash a runtime si verifica preventivamente la compatibilità:

```

1  if (d instanceof ItalianDate) {
2      ItalianDate id = (ItalianDate) d;
3  }

```

11.2 Codice: Evoluzione del Codice: [Lezione12/MavenDate](#)

In *Lezione12* la gerarchia del progetto diventa ricca e completa:

```

1  classDiagram
2      class Object {
3          +toString()
4          +equals(Object)
5      }
6      class Comparable {
7          <<interface>>
8          +compareTo(Object)
9      }
10     class Time {
11         <<interface>>
12         +getSeconds()
13         +getMinutes()
14         +getHours()
15     }
16     class Date {
17         #day: int
18         -month: int
19         -year: int
20         +toString()
21         +equals(Object)
22         +compareTo(Object)
23     }
24     class FormattedDate {
25         <<abstract>>
26         #format: String
27         #months: String[]
28         +printFormat() final
29         +getMonthAsString() final
30         +prettyPrint()* String
31     }
32     class ItalianDate {
33         +prettyPrint()
34         +toString()
35     }
36     class AmericanDate {
37         +prettyPrint()
38         +toString()
39     }
40     class TimeStamp {

```

```

41     #seconds: int
42     #minutes: int
43     #hours: int
44     +toString()
45     +equals(Object)
46 }
47
48 Object <|-- Date
49 Comparable <|.. Date
50 Date <|-- FormattedDate
51 FormattedDate <|-- ItalianDate
52 FormattedDate <|-- AmericanDate
53 Date <|-- TimeStamp
54 Time <|.. TimeStamp

```

11.2.1 L'Interfaccia Comparable in Date.java

Date implementa l'interfaccia standard `java.lang.Comparable` per consentire l'ordinamento naturale:

```

1  public class Date implements Comparable {
2      ...
3      @Override
4      public int compareTo(Object other) {
5          Date otherAsDate = (Date) other;
6          int diff = this.year - otherAsDate.getYear();
7          if (diff != 0) return diff;
8          diff = this.month - otherAsDate.getMonth();
9          if (diff != 0) return diff;
10         return this.day - otherAsDate.getDay();
11     }
12 }

```

Logica di confronto: confronta prima gli anni, se uguali confronta i mesi, se uguali confronta i giorni. Restituisce un intero negativo se `this < other`, zero se uguali, positivo se `this > other`.

11.2.2 L'Astrazione con Classe Astratta: FormattedDate.java

Centralizza i dati di formattazione evitando duplicazioni tra `ItalianDate` e `AmericanDate`:

```

1  public abstract class FormattedDate extends Date {
2      protected final String format;
3      protected final String[] months;
4
5      public FormattedDate(int day, int month, int year, String format, String[] months) {
6          super(day, month, year);
7          this.format = format;
8          this.months = months;
9      }
10
11     // Metodi final: le sottoclassi non possono sovrascriverli (Template Method Pattern)
12     public final String printFormat() { return format; }
13     public final String getMonthAsString() { return months[getMonth() - 1]; }
14
15     // Metodo astratto: DEVE essere implementato dalle sottoclassi concrete
16     public abstract String prettyPrint();
17 }

```

11.2.3 Ereditarietà Multipla di Tipo: TimeStamp.java

Mostra come combinare l'estensione di una classe concreta con l'implementazione di un'interfaccia:

```

1 public class TimeStamp extends Date implements Time {
2     protected final int seconds;
3     protected final int minutes;
4     protected final int hours;
5
6     public TimeStamp(int seconds, int minutes, int hours, int day, int month, int year) {
7         super(day, month, year);
8         this.seconds = seconds;
9         this.minutes = minutes;
10        this.hours = hours;
11    }
12
13    @Override
14    public String toString() {
15        // Riusa il toString() della superclasse aggiungendo l'orario con zero-padding a due cifre:
16        return String.format("%s[%02d:%02d:%02d]", super.toString(), hours, minutes, seconds);
17    }
18
19    @Override
20    public boolean equals(Object other) {
21        // 1. Verifica prima l'uguaglianza della data con super.equals():
22        if (super.equals(other) == false) return false;
23        if (!(other instanceof TimeStamp)) return false;
24        TimeStamp otherAsTimeStamp = (TimeStamp) other;
25        // 2. Poi verifica l'orario:
26        return hours == otherAsTimeStamp.getHours() &&
27            minutes == otherAsTimeStamp.getMinutes() &&
28            seconds == otherAsTimeStamp.getSeconds();
29    }
30 }

```

11.2.4 Dimostrazione Pratica in MainDate.java

```

1 Time ts1 = new TimeStamp(0, 0, 10, 10, 11, 2025);
2 System.out.println(ts1.toString()); // Esegue TimeStamp.toString() tramite Late Binding!
3
4 // ts1.getDay(); // COMPILER-TIME ERROR: il tipo statico Time non possiede il metodo getDay()!
5 System.out.println(((Date) ts1).getDay()); // OK: cast esplicito a Date!
6
7 Date[] arr = { ts2, itDate, new Date(9, 9, 2025) };
8 Arrays.sort(arr); // Ordina l'array eterogeneo grazie a compareTo()!
9 System.out.println(Arrays.toString(arr));

```

Output a video prodotto dall'ordinamento polimorfico:

```
1 [y2025m9d9, 3/11/2025, y2025m11d10[10:00:00]]
```

- L'ordinamento colloca correttamente prima il 9 settembre, poi il 3 novembre (itDate) e infine il 10 novembre (TimeStamp).
- In fase di stampa, ciascun elemento viene formattato dal **proprio** metodo toString() dinamico.

Nota di Studio

Con la **Lezione 12** abbiamo analizzato la gerarchia completa di ereditarietà, l'overriding con `super`, il late binding, la classe astratta `FormattedDate` e l'ordinamento con `Comparable`.

Il prossimo blocco è la **Lezione 13 / Slide T13: *Abstract Classes and Interfaces***: - Differenze ontologiche e strutturali tra **Classi Astratte** e **Interfacce**. - Metodi `default` e metodi `static` nelle interfacce (Java 8+). - Ereditarietà multipla tramite interfacce e risoluzione dei conflitti (*Diamond Problem* sui metodi `default`). - Evoluzione di `MavenDate` in `Lezione13`: - La classe `Date` viene ripensata per gestire il calendario completo. - Analisi delle nuove interfacce e classi di supporto.

Dammi conferma per aprire la Lezione 13 / T13 e analizzare a fondo Classi Astratte e Interfacce!

T13 — Classi Astratte ed Interfacce

Contratti Comportamentali, Metodi Astratti, Default e Static Methods, Builder Pattern e Lambda

In questo blocco analizziamo a fondo i meccanismi di astrazione pura di Java: le **Classi Astratte**, le **Interfacce**, il design pattern **Template Method** e le interfacce cardine della libreria standard (`Comparable`, `Iterable`, `Iterator`) ([T13 - Abstract Classes and Interfaces.pdf](#)). Nel codice di [Lezione13/MavenDate](#), il docente compie un'evoluzione straordinaria introducendo: 1. La classe statica annidata `Date.Builder` per la costruzione controllata degli oggetti. 2. L'interfaccia funzionale `FormattedDateConverter`. 3. L'uso congiunto in `MainDate.java` di **Classi Anonime** (`new Time(){ ... }`) e **Espressioni Lambda** (`d -> new AmericanDate(...)`).

12.1 Teoria: Concetti Teorici dalle Slide (Slide T13)

12.1.1 Classi Astratte e Template Method Pattern (Slide 1–10)

- **Metodo Astratto:**

- Un metodo dichiarato con la parola chiave `abstract` e **privo di corpo** (termina con il punto e virgola `;` anziché con le parentesi graffe `{ ... }`):

```
1 public abstract String prettyPrint();
```

- Definisce una “firma obbligatoria”: impone alle sottoclassi concrete il compito di implementarne la logica.

- **Classe Astratta (`abstract class`):**

- Se una classe contiene anche un solo metodo astratto, **deve obbligatoriamente essere dichiarata `abstract`**.
- [Attenzione] **Regola aurea d'esame:** Una classe astratta **non può MAI essere istanziata direttamente** (`new FormattedDate(...)` genera un **Compile-Time Error!**).
- Può contenere costruttori (invocabili dalle sottoclassi tramite `super(...)`), campi di istanza (`protected/private`), metodi concreti e metodi astratti.
- Una sottoclasse che estende una classe astratta deve:
 - * O implementare **tutti** i metodi astratti ereditati.
 - * Oppure essere dichiarata essa stessa `abstract`.

- **Il Template Method Pattern:**

- Pattern comportamentale in cui la superclasse definisce lo scheletro immutabile di un algoritmo all'interno di un metodo concreto marcato **`final`** (il template), mentre delega i singoli passi variabili o dipendenti dal contesto a metodi `abstract` implementati dalle sottoclassi.
- Nel nostro progetto: `printFormat()` e `getMonthAsString()` SONO `final` in `FormattedDate`, mentre `prettyPrint()` è `abstract`.

12.1.2 Le Interfacce in Java (interface) (Slide 11–17)

- **Definizione e Filosofia:**

- Un’interfaccia è un **contratto puro di comportamento**. Rappresenta ciò che una classe *sa fare* (*can-do*), non la sua identità ontologica.
- **Membri di un’interfaccia standard:**
 - * **Campi:** sono implicitamente ed esclusivamente **public static final** (costanti di classe). Non possono esistere variabili d’istanza o stato mutabile.
 - * **Metodi:** sono implicitamente **public abstract** (non serve specificarlo).

- **Ereditarietà Multipla di Tipo (implements):**

- Una classe può estendere al massimo una sola superclasse, ma può implementare **un numero arbitrario di interfacce separate da virgola:**

```
1 public class TimeStamp extends Date implements Time, Serializable, Cloneable
```

- Questo realizza in Java l’ereditarietà multipla dei tipi senza incorrere nei problemi di ambiguità di memoria del C++.

- **Ereditarietà tra Interfacce:**

- Un’interfaccia può estendere altre interfacce tramite `extends` (anche multiple contemporaneamente: `interface C extends A, B`).

12.1.3 Confronto Sistemático: Classe Astratta vs Interfaccia

Proprietà	Classe Astratta (abstract class)	Interfaccia (interface)
Istanziabilità	No (new vietato)	No (new vietato)
Stato d’istanza (campi)	[OK] Sì (qualsiasi visibilità)	No (solo costanti <code>public static final</code>)
Costruttori	[OK] Sì (invocabili con <code>super(...)</code>)	No
Ereditarietà	Singola (<code>extends</code> una sola classe)	Multipla (<code>implements N</code> interfacce)
Relazione concettuale	Identità ontologica forte (<i>IS-A</i>)	Capacità / Contratto comportamentale (<i>CAN-DO</i>)

12.1.4 Interfacce Standard Fondamentali della JDK (Slide 18–22)

1. **`java.lang.Comparable<T>`:**

- Definisce il metodo `int compareTo(T other)`.
- Permette agli algoritmi di ordinamento (`Arrays.sort()`, `Collections.sort()`) e agli insiemi ordinati (`TreeSet`, `TreeMap`) di ordinare gli oggetti secondo il loro “ordine naturale”.

2. **`java.lang.Iterable<T>` e `java.util.Iterator<T>`:**

- **`Iterable<T>`:** espone il metodo `Iterator<T> iterator()`.

- **Regola cruciale:** Qualsiasi classe che implementi `Iterable` può essere utilizzata come sorgente nel **ciclo For-Each** (`for (T elem : collection)`)!
- **`Iterator<T>`:** l'oggetto cursore che scorre la sequenza:
 - `boolean hasNext()`: verifica se vi sono ulteriori elementi.
 - `T next()`: restituisce il prossimo elemento avanzando il cursore.
 - `default void remove()`: rimuove l'ultimo elemento restituito.

12.2 Codice: Evoluzione del Codice: [Lezione13/MavenDate](#)

12.2.1 Il Design Pattern Builder con Classe Statica Annidata (`Date.java`)

All'interno di `Date.java`, il docente definisce una **Static Nested Class** per costruire date in modo protetto:

```

1 public class Date implements Comparable {
2     ...
3     public static class Builder {
4         private final int year;
5
6         public Builder(int year) {
7             if (year > 0)
8                 this.year = year;
9             else
10                this.year = 1970; // Anno di default sicuro
11        }
12
13        public Date build(int day, int month) {
14            // Se i parametri sono illegali, ricade sulla data sicura 1/1/year!
15            if (month < 1 || month > 12 || day < 1 || day > daysPerMonth(month))
16                return new Date(1, 1, year);
17            return new Date(day, month, year);
18        }
19    }
20 }

```

Vantaggio del Builder: Incapsula e pre-configura l'anno; se l'utente fornisce parametri non validi (`month = -1`), il costruttore non fallisce a video ma restituisce una data valida di fallback (`01/01/year`).

12.2.2 L'Interfaccia Funzionale: `FormattedDateConverter.java`

```

1 package it.oop.core;
2
3 public interface FormattedDateConverter {
4     FormattedDate convert(FormattedDate date);
5 }

```

Questa interfaccia dichiara **un solo metodo astratto** (SAM: *Single Abstract Method*). In Java è a tutti gli effetti un' **Interfaccia Funzionale**, target ideale per le espressioni Lambda!

12.2.3 Sintesi Pratica in MainDate.java

Questo file è una miniera di concetti d'esame avanzati:

```
1 package it.oop.ui;
2
3 import it.oop.core.*;
4 import it.oop.core.Date;
5
6 public class MainDate {
7     public static void main(String[] args) {
8         // 1. Uso della Static Nested Class Builder:
9         Date.Builder dateBuilder = new Date.Builder(2025);
10        Date d1 = dateBuilder.build(10, 9); // Giorno 10, Mese 9 -> Valida!
11        Date d2 = dateBuilder.build(10, -1); // Mese -1 illegale -> Fallback su 1/1/2025!
12        System.out.println(d1.toString()); // y2025m9d10
13        System.out.println(d2.toString()); // y2025m1d1
14
15        // 2. Classe Anonima (Anonymous Inner Class):
16        // Implementa al volo l'interfaccia Time senza creare un file .java separato!
17        Time init = new Time() {
18            @Override
19            public int getHours() { return 0; }
20            @Override
21            public int getMinutes() { return 0; }
22            @Override
23            public int getSeconds() { return 0; }
24            @Override
25            public String toString() {
26                return String.format("%02d:%02d:%02d", getHours(), getMinutes(), getSeconds());
27            }
28        };
29        System.out.println(init.toString()); // Stampa 00:00:00
30
31        // 3. Espressione Lambda (Sintassi compatta per Interfaccia Funzionale):
32        FormattedDateConverter toAmerican =
33            d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear());
34
35        // Test di conversione polimorfica:
36        System.out.println(toAmerican.convert(new ItalianDate(11, 11, 2025)) instanceof
37            AmericanDate);
38        // Stampa TRUE!
39    }
```

12.3 [Test] Output di Esecuzione Verificato con Maven

Eseguendo `mvn exec:java -Dexec.mainClass="it.oop.ui.MainDate"`:

```
1 y2025m9d10
2 y2025m1d1
3 00:00:00
4 true
```

Nota di Studio

Con la **Lezione 13** abbiamo chiuso il cerchio su Classi Astratte, Interfacce, Template Method e abbiamo anticipato le classi annidate e le lambda.

Il prossimo blocco è la **Lezione 14 / Slide T14: *Nested and Anonymous Classes***: - Tassonomia completa delle classi interne: **Static Nested Classes**, **Inner Member Classes** (non statiche), **Local Classes** e **Anonymous Classes**. - Accesso allo stato della classe contenitore (*Enclosing instance*) e la sintassi speciale `EnclosingClass.this`. - Cattura delle variabili locali nelle classi locali/anonime e il vincolo di essere **effectively final**. - Evoluzione del codice in `Lezione14`: consolidamento delle classi interne e preparativi per i Generics.

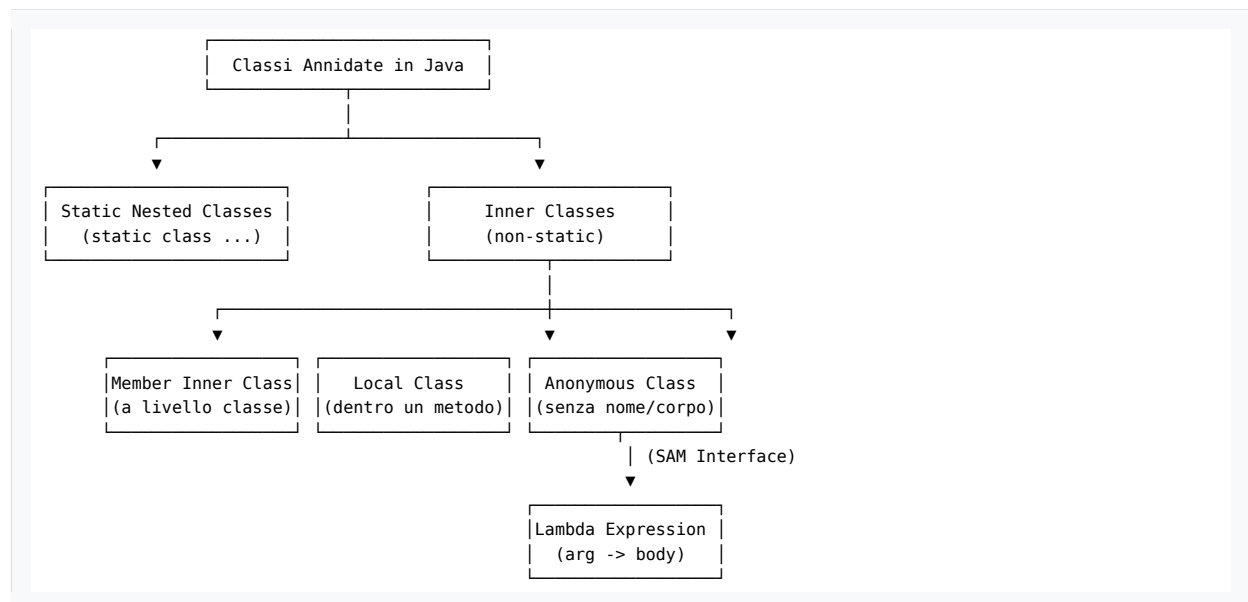
Dammi conferma per aprire la Lezione 14 / T14 e continuare la revisione!

T14 — Classi Annidate, Interne, Locali ed Anonime

Static Nested vs Inner Classes, Enclosing Pointer, Shadowing, Effectively Final e Classi Anonime

13.1 Analisi Teorica Approfondita (Slide T14)

La lezione 14 esplora la gerarchia delle classi definite all'interno di altre classi (meccanismo introdotto fin da Java 1.1), le classi anonime, l'evoluzione verso le **espressioni Lambda** (Java 8) e la feature dei **Varargs**.



13.1.1 Static Nested Classes (static class Nested)

Una classe dichiarata con il modificatore `static` all'interno di un'altra classe contenitore (*enclosing class*):

```

1 package pkg;
2 class Outer {
3     static class Nested {
4         // ...
5     }
6 }
  
```

- **Natura:** È a tutti gli effetti una classe di primo livello (*top-level*), ma inserita nel *namespace* della classe che la ospita.
- **Nome completo (Fully Qualified Name):** `pkg.Outer.Nested` (a livello di bytecode su disco viene generato il file `Outer$Nested.class`).
- **Accesso ai membri:**

- Trattandosi di un membro statico, **ha accesso a tutti i membri statici** della classe contenitore, **compresi quelli privati (private)**!
- **NON ha accesso** ai membri di istanza (campi o metodi non statici) di `Outer`, poiché non esiste alcun riferimento implicito ad un'istanza di `Outer`.
- **Istanziamento:** Può essere istanziata in modo totalmente indipendente dall'esistenza di un oggetto `Outer`:


```
1 Outer.Nested nestedObject = new Outer.Nested();
```
- **Caso d'uso tipico:** Il pattern **Builder** (visto in [Date.java](#)), classi di utilità, nodi interni di strutture dati (es. `static class Node<E>` in un albero o grafo).

13.1.2 Member Inner Classes (class Inner non-statica)

Una classe dichiarata all'interno di un'altra classe **senza** la parola chiave `static`:

```
1 package pkg;
2 class Outer {
3     class Inner {
4         // ...
5     }
6 }
```

- **Natura:** Un'istanza di `Inner` **non può esistere senza un'istanza associata di `Outer`**.
- **Riferimento implicito (`Outer.this` o `this$0`):**
 - Ogni oggetto `Inner` memorizza internamente un puntatore sintetico nascosto all'oggetto `Outer` che l'ha generato.
 - Tramite questo puntatore, `Inner` ha accesso a **tutti** i membri di `Outer`, sia statici che di istanza, inclusi quelli `private`!
- **Sintassi di istanziamento da codice esterno:**

```
1 Outer outerObject = new Outer();
2 Outer.Inner innerObject = outerObject.new Inner(); // Sintassi specifica con outerObject.new
```

Confronto: Nested vs Inner (Slide 7)

```
1 class Outer {
2     private static int I = 5;
3     private int i;
4
5     static class Nested {
6         // NON ha accesso al campo di istanza i!
7         static void print() { System.out.println(I); } // OK: accede a membro statico privato I
8     }
9
10    class Inner {
11        // Ha accesso sia a I che al campo di istanza i della sua Outer associata
12        void print() { System.out.println(i); }
13    }
14
15    public void outerMethod() {
```



```
16     Nested.print();
17     Inner innerObject = new Inner(); // Qui 'this.new Inner()' è implicito
18     innerObject.print();
19 }
20 }
```

13.1.3 Local Classes (Classi Locali a un Metodo)

Una classe definita all'interno del corpo di un blocco di codice (tipicamente un metodo):

```
1 public void outerMethod() {
2     final int j = 1;
3
4     class Local { // NESSUN modificatore di visibilità ammesso (no public, private, protected)
5         int addOne() { return j + 1; }
6     }
7
8     Local localObject = new Local();
9     System.out.println(localObject.addOne()); // Stampa 2
10 }
```

- **Regole di visibilità:** La classe è visibile e istanziabile **esclusivamente all'interno del blocco/metodo** in cui è dichiarata.
- **Cattura delle variabili locali (Variable Capture & Closures):**
 - Può accedere alle variabili locali e ai parametri del metodo contenitore **solo se sono dichiarati final oppure sono effectively final** (ovvero il loro valore non viene mai modificato dopo l'inizializzazione).
 - **Perché questo vincolo severo? (Domanda classica d'esame):**
 - * Le variabili locali risiedono nello **Stack Frame** del metodo e vengono distrutte non appena il metodo termina (return).
 - * L'oggetto della classe locale (localObject) risiede nell'**Heap** e potrebbe sopravvivere ben oltre la fine del metodo (ad esempio se restituito come interfaccia o registrato come listener).
 - * Per permettere all'oggetto di accedere a j, il compilatore Java crea un campo sintetico nascosto dentro Local e vi **copia** il valore di j al momento della creazione.
 - * Se j potesse essere riassegnato (j++), il valore nello stack e la copia nell'heap diverrebbero disallineati, rompendo la coerenza del linguaggio. Rendendo la variabile immutabile (final), la copia rimane identica e coerente per sempre.

13.1.4 Il Meme della Slide 10: Objects inside objects

Nella Slide 10, il prof. Pasqua inserisce il celebre meme di Xzibit (*Pimp My Ride*): > “YO DAWG, I HEAR YOU LIKE OBJECT ORIENTED PROGRAMMING SO I NESTED A CLASS INSIDE YOUR CLASS SO YOU CAN CREATE OBJECTS WHILE YOU CREATE OBJECTS”

A sottolineare con ironia la tentazione (e il potere) di creare gerarchie di classi dentro classi per incapsulare comportamenti strettamente accoppiati.

13.1.5 Motivazioni Architetture & Il Caso di Studio *Iterator Pattern* (Slide 11-12)

Perché annidare le classi? 1. **Migliore organizzazione del codice**: collocare classi di supporto logico all'interno del namespace della sola classe che le utilizza. 2. **Politica di visibilità privilegiata (*Privileged Access*)**: le classi annidate possono leggere lo stato privato della classe ospitante senza costringere a esporre metodi public o package-private pericolosi verso l'esterno. 3. **Meno complessità nel codebase**: evita la proliferazione di decine di minuscoli file .java separati.

L'Esempio del pattern *Iterable* / *Iterator* (Slide 12)

- **Senza annidamento (Due classi separate)**: *CRIterator* deve essere dichiarata all'esterno, riceve l'array *Student[] arr* nel costruttore e necessita di visibilità speciale:

```

1 class Classroom implements Iterable {
2     private Student[] students;
3     @Override
4     public Iterator iterator() { return new CRIterator(students); }
5 }
6 class CRIterator implements Iterator {
7     private Student[] arr;
8     private int next = 0;
9     CRIterator(Student[] arr) { this.arr = arr; }
10    @Override public boolean hasNext() { return next < arr.length; }
11    @Override public Object next() { return new Student(arr[next++]); }
12 }
```

- **Con Inner Class (Soluzione idiomatica Java)**: *CRIterator* è una Inner Class dentro *ClassRoom*. Non serve memorizzare un array duplicato: accede direttamente al campo privato *students* di *ClassRoom*!

```

1 class Classroom implements Iterable {
2     private Student[] students;
3     @Override
4     public Iterator iterator() { return new CRIterator(); }
5
6     class CRIterator implements Iterator {
7         private int next = 0;
8         @Override public boolean hasNext() { return next < students.length; }
9         @Override public Object next() { return new Student(students[next++]); }
10    }
11 }
```

13.1.6 Anonymous Classes (Classi Anonime, Slide 13-16)

Una classe locale **priva di nome**, definita e istanziata contestualmente all'interno di una singola espressione.

* **Vincolo**: Può essere creata solo estendendo una classe esistente o implementando un'interfaccia: “java // 1. Estendendo una classe esistente: `Person sam = new Person(“Sam”) { @Override public String toString() { return “It’s Sam!”; } }`;

// 2. Implementando un'interfaccia: `Time init = new Time() { @Override public int getHours() { return 0; } @Override public int getMinutes() { return 0; } @Override public int getSeconds() { return 0; } }`; *
****Regola aurea sui Costruttori****: **Una classe anonima NON può dichiarare alcun costruttore esplicito!**
(Poiché i costruttori devono avere per definizione lo stesso nome della classe, e una classe anonima non ha nome). Eventuali inizializzazioni devono sfruttare gli *instance initializer blocks* `{ ... }` o i parametri passati al costruttore della superclasse `(`new SuperClass(arg1, arg2){ ... }`)`. * ****Espressioni****: Essendo espressioni, devono terminare con un punto e virgola ``;` (se assegnate a una variabile) o possono essere passate direttamente come parametri attuali di un metodo: `java tasks.add(new Runnable() { @Override public void run() { System.out.println(“Running!”); } });` “

13.1.7 Lambda Expressions (Slide 17-19)

Introdotte in Java 8, sono una notazione sintattica ultra-compatta per implementare **interfacce funzionali** (interfacce con un **unico metodo astratto**, note come SAM — *Single Abstract Method*).

```
1 (parametri) -> corpo
```

- **Parametri:**

- Nessun parametro: `() -> 42`
- Un parametro: `x -> x + 1` (le parentesi tonde sono opzionali se non si specifica il tipo esplicito)
- Due o più parametri: `(x, y) -> x + y`

- **Corpo:**

- Espressione singola: `param -> param - 1` (restituzione implicita del valore calcolato, niente `return` né graffe)
- Blocco di codice: `param -> { int res = param - 1; return res; }` (richiede graffe esplicite e `return`)

- **Inferenza di tipo (*Type Inference*):** Il compilatore Java deduce i tipi dei parametri e del valore di ritorno dal contesto d'uso (*target typing* dell'interfaccia funzionale).

- **Differenza tra Classi Anonime e Lambda:**

- La classe anonima può implementare interfacce con più metodi astratti o estendere classi concrete/astratte; la lambda può implementare **solo interfacce funzionali SAM**.
- La classe anonima crea un nuovo scope per `this` (che punta all'istanza anonima stessa); nella lambda `this` è **lessicale** (punta all'istanza della classe circostante).
- La classe anonima può dichiarare campi di istanza che mantengono stato; la lambda è per natura *stateless*.

13.1.8 Varargs (`type... args`, Slide 20-21)

Consente a un metodo di accettare un numero variabile (da zero a molti) di argomenti dello stesso tipo:

```
1 static int min(int... values) {  
2     int res = Integer.MAX_VALUE;  
3     for (int v : values)  
4         if (v < res) res = v;  
5     return res;  
6 }
```

- **Dietro le quinte del compilatore:** Il parametro `int... values` viene tradotto esattamente in un array `int[] values`. Al momento della chiamata:

- `min(1, 0, 5)` → il compilatore inietta il codice: `min(new int[]{ 1, 0, 5 })`.
- `min()` (senza argomenti) → crea un array vuoto `min(new int[]{})`.

- **Regole tassative:**

1. Ci può essere **al massimo un** parametro `varargs` per metodo.
2. Il parametro `varargs` deve essere tassativamente l'**ultimo** nella lista dei parametri formali (`void foo(String s, int... nums)` è valido; `void foo(int... nums, String s)` genera errore di compilazione).

13.2 Analisi Dettagliata del Codice (Lezione14/MavenDate)

Nella Lezione 14 il progetto [MavenDate](#) introduce il primo ponte fondamentale verso i **Tipi Generici** (anticipando la teoria formale di Slide T15) e sfrutta le classi annidate, le classi anonime e le lambda viste nella Lezione 13.

13.2.1 Diff di Date.java (Lezione 13 vs Lezione 14)

Fino alla Lezione 13, Date implementava l'interfaccia non tipizzata (raw) Comparable:

```

1 --- Lezione13/Date.java
2 +++ Lezione14/Date.java
3 @@ -1,6 +1,6 @@
4   package it.oop.core;
5
6  -public class Date implements Comparable {
7  +public class Date implements Comparable<Date> {
8
9  @@ -73,15 +73,14 @@
10     @Override
11     - public int compareTo(Object other) {
12     -     Date otherAsDate = (Date) other;
13     -     int diff = this.year - otherAsDate.getYear();
14  + public int compareTo(Date other) {
15  +     int diff = this.year - other.getYear();
16     if (diff != 0)
17         return diff;
18     -     diff = this.month - otherAsDate.getMonth();
19  +     diff = this.month - other.getMonth();
20     if (diff != 0)
21         return diff;
22     -     return this.day - otherAsDate.getDay();
23  +     return this.day - other.getDay();
24     }

```

- **Beneficio immediato del Generics:**

1. La firma del metodo diventa fortemente tipizzata a compile-time: `compareTo(Date other)` anziché `compareTo(Object other)`.
2. Viene eliminato il cast esplicito `(Date)other`, azzerando il rischio di incorrere a runtime in una `ClassCastException`.

13.2.2 La Nuova Classe Generica Pair.java

```

1 package it.oop.core;
2
3 public class Pair<F, S> {
4     private final F first;
5     private final S second;
6
7     public Pair(F first, S second) {
8         this.first = first;
9         this.second = second;
10    }
11
12    public F getFirst() { return first; }
13    public S getSecond() { return second; }
14 }

```

- *F* (*First*) e *s* (*Second*) sono due **Type Parameters** indipendenti.
- I campi sono `private final`, rendendo la coppia immutabile una volta istanziata.

13.2.3 La Classe Generica con Bounded Type `OrderedPair.java`

```

1 package it.oop.core;
2
3 public class OrderedPair <T extends Comparable<T>> {
4     private final T first;
5     private final T second;
6
7     public OrderedPair(T first, T second) {
8         this.first = first;
9         this.second = second;
10        if (first.compareTo(second) > 0)
11            System.out.println("Pair not ordered!");
12    }
13
14    public T getFirst() { return first; }
15    public T getSecond() { return second; }
16 }

```

- **Bounded Type Parameter** (`<T extends Comparable<T>>`):
 - Il tipo `T` non può essere un qualsiasi tipo `Object`: deve tassativamente implementare l'interfaccia `Comparable<T>`.
 - Grazie a questo vincolo, il compilatore consente all'interno del costruttore di invocare legalmente `first.compareTo(second)`.
 - Se si tentasse di istanziare `OrderedPair<Object>`, il compilatore bloccherebbe l'operazione con un errore prima ancora di eseguire.

13.2.4 Le Sottoclassi Specializzate `DatePair.java` e `DateInterval.java`

`DatePair.java`

```

1 package it.oop.core;
2
3 public class DatePair extends Pair<Date, Date> {
4     public DatePair(Date left, Date right) {
5         super(left, right);
6     }
7
8     public Date getLeft() { return getFirst(); }
9     public Date getRight() { return getSecond(); }
10
11    @Override
12    public String toString() {
13        Date l = getLeft();
14        Date r = getRight();
15        return String.format("[%s .. %s]", l.toString(), r.toString());
16    }
17 }

```

- Fissa sia *F* che *s* al tipo concreto `Date` (`extends Pair<Date, Date>`).
- Offre metodi con semantica di dominio più chiara: `getLeft()` e `getRight()`.

DateInterval.java

```

1 package it.oop.core;
2
3 public class DateInterval extends OrderedPair<Date> {
4     public DateInterval(Date left, Date right) {
5         super(left, right);
6     }
7
8     public Date getLeft() { return getFirst(); }
9     public Date getRight() { return getSecond(); }
10
11     @Override
12     public String toString() {
13         Date l = getLeft();
14         Date r = getRight();
15         return String.format("[%s .. %s]", l.toString(), r.toString());
16     }
17 }

```

- Eredita da `OrderedPair<Date>`: è legale perché `Date` implements `Comparable<Date>`.
- Se le date passate non sono cronologicamente ordinate (`left > right`), il costruttore di `OrderedPair` emette il messaggio d'avviso "Pair not ordered!".

13.2.5 MainDate.java

Raccoglie in un unico punto di ingresso tutte le feature viste nelle lezioni 13 e 14:

```

1 package it.oop.ui;
2
3 import it.oop.core.*;
4 import it.oop.core.Date;
5
6 public class MainDate {
7
8     public static void main(String[] args) {
9         // 1. Static Nested Class (Builder Pattern da Lezione 13)
10        Date.Builder dateBuilder = new Date.Builder(2025);
11        Date d1 = dateBuilder.build(10,9);
12        Date d2 = dateBuilder.build(10,-1);
13        System.out.println(d1.toString());
14        System.out.println(d2.toString());
15
16        // 2. Anonymous Class che implementa l'interfaccia Time (Lezione 13/14)
17        Time init = new Time() {
18            @Override public int getHours() { return 0; }
19            @Override public int getMinutes() { return 0; }
20            @Override public int getSeconds() { return 0; }
21            @Override public String toString() {
22                return String.format("%02d:%02d:%02d", getHours(), getMinutes(), getSeconds());
23            }
24        };
25        System.out.println(init.toString());
26
27        // 3. Lambda Expression che implementa FormattedDateConverter (Lezione 13/14)
28        FormattedDateConverter toAmerican =
29            d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear());
30        System.out.println(toAmerican.convert(new ItalianDate(11, 11, 2025)) instanceof
            AmericanDate);

```

```

31
32 // 4. Generics: Pair con Diamond Operator <>
33 Pair<String, FormattedDate> event =
34     new Pair<>("00P exam", new ItalianDate(2, 2, 2026));
35 System.out.println(event.getFirst() + " on " + event.getSecond().prettyPrint());
36
37 // 5. DatePair e DateInterval
38 DatePair dp = new DatePair(new Date(1,2,2025), new Date(1,1,2025));
39 System.out.println("date pair: " + dp.toString());
40
41 // Qui scatta l'avviso di ordinamento perché 1/2/2025 è successivo a 1/1/2025!
42 DateInterval di = new DateInterval(new Date(1,2,2025), new Date(1,1,2025));
43
44 // 6. Wildcard con Bounded Type (anticipazione Lezione 15)
45 Pair<?, ? extends FormattedDate> unknown = event;
46 System.out.println(unknown.getFirst().toString() + " on " +
47     unknown.getSecond().prettyPrint());
48 }

```

13.3 Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel workspace del corso:

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainDate"
```

Output ottenuto:

```

1 y2025m9d10
2 y2025m1d1
3 00:00:00
4 true
5 00P exam on 2 febbraio 2026
6 date pair: [y2025m2d1 .. y2025m1d1]
7 Pair not ordered!
8 00P exam on 2 febbraio 2026

```

13.3.1 Dettaglio riga per riga dell'output:

1. y2025m9d10 → generato da `d1 = dateBuilder.build(10, 9)` (10 settembre 2025).
2. y2025m1d1 → generato da `d2 = dateBuilder.build(10, -1)` (mese errato fallback su 1/1/2025).
3. 00:00:00 → generato dall'override di `toString()` della **classe anonima** `init` che implementa `Time`.
4. true → la **lambda** `toAmerican` converte l'istanza di `ItalianDate` in `AmericanDate`.
5. 00P exam on 2 febbraio 2026 → `Pair<String, FormattedDate>` con formattazione italiana.
6. date pair: [y2025m2d1 .. y2025m1d1] → `DatePair` stampa le due date senza controllo di sequenzialità.
7. Pair not ordered! → emesso dal costruttore di `OrderedPair` dentro `new DateInterval(1/2/2025, 1/1/2025)` perché `1/2/2025 > 1/1/2025`.
8. 00P exam on 2 febbraio 2026 → invocazione polimorfica tramite wildcard generica `Pair<?, ? extends FormattedDate>`.

Dimmi “vai” per procedere con il **Blocco 14** (Lezione 15 / Slide T15 — *Generic Types, Type Erasure, Wildcards, PECS e diff in Lezione15/MavenDate*).

T15 — Tipi Generici, Wildcard e Type Erasure

Polimorfismo Parametrico, Bounded Type Parameters, Invarianza, Wildcard (PECS) e Type Erasure

14.1 Analisi Teorica Approfondita (Slide T15)

La lezione 15 introduce formalmente i **Tipi Generici** (*Generics*), introdotti in Java 5 per abilitare il **polimorfismo parametrico**, eliminare la fragilità dei cast a `Object` e garantire la **Type Safety** a tempo di compilazione.

14.1.1 L'Idea Guida & La Tazza `Cup<T>` (Slide 2-3)

Nella Slide 3 campeggia l'immagine di una tazza con la scritta:

Cup<T>

accompagnata dal motto: **“Same container, different content”**. Un contenitore (una tazza, una coppia, una lista, un albero) possiede una logica strutturale identica a prescindere dal tipo di dato che custodisce (caffè, tè, stringhe, interi, date). Prima di Java 5, per ottenere contenitori riusabili si usava `Object`, con conseguenze disastrose sull'affidabilità del software.

14.1.2 Polimorfismo Parametrico vs Approccio Basato su `Object` (Slide 4-7)

Consideriamo la funzione identità matematica $\lambda x. x$: * Se riceve un `int`, restituisce lo stesso `int`. * Se riceve una `String`, restituisce la stessa `String`.

L'approccio ingenuo (pre-Java 5): segnaposto `Object`

```
1 public class Pair {
2     private Object first, second;
3     public Pair(Object first, Object second) {
4         this.first = first;
5         this.second = second;
6     }
7     public Object getFirst() { return first; }
8     public Object getSecond() { return second; }
9 }
```

- **Casting obbligatorio:**

```
1 Pair p = new Pair("one", "two");
2 String first = (String) p.getFirst(); // Cast esplicito verboso e tedioso
```


- **Il disastro della mancanza di Type Safety:**

```
1 Pair q = new Pair(1, 2); // Autoboxing da int a Integer
2 String second = (String) q.getSecond(); // COMPILA SENZA ERRORI!
3 // Ma a RUNTIME crasha con: java.lang.ClassCastException: class java.lang.Integer cannot be
  cast to class java.lang.String
```

14.1.3 Classi e Metodi Generici (Slide 8-10)

Con i Generics, la classe introduce una o più **variabili di tipo** (*Type Parameters*):

```
1 public class Pair<T> {
2     private T first, second;
3     public Pair(T first, T second) {
4         this.first = first;
5         this.second = second;
6     }
7     public T getFirst() { return first; }
8     public T getSecond() { return second; }
9 }
```

- **Sintassi d'uso:**

```
1 Pair<String> p = new Pair<String>("one", "two");
2 Pair<Integer> q = new Pair<Integer>(1, 2);
```

- **Controllo a compile-time:**

```
1 String first = p.getFirst(); // Nessun cast necessario: p restituisce String garantita!
2 String second = q.getSecond(); // COMPILE-TIME ERROR: Type mismatch (Integer non convertibile
  a String)
```

L'errore viene intercettato prima ancora di mandare in esecuzione il programma!

- **Regola fondamentale sui Tipi Primitivi:** Un parametro di tipo *T* può essere sostituito solo da tipi riferimento (oggetti), MAI da tipi primitivi! Non è consentito scrivere `Pair<int>`, ma si deve usare il wrapper `Pair<Integer>`.
- **Convenzioni di Naming:** Si usano lettere maiuscole singole:
 - *T*: *Type* generico.
 - *E*: *Element* (usato nelle collezioni: `List<E>`, `Set<E>`).
 - *K*, *V*: *Key* e *Value* (usato nelle mappe: `Map<K, V>`).
 - *N*: *Number*.
 - *S*, *U*: Tipi secondari ausiliari.

14.1.4 Interfacce della Libreria Standard Riscritte con i Generics (Slide 11-13)

- **Comparable<T>:**

```
1 public interface Comparable<T> {
2     int compareTo(T obj);
3 }
```

Consente alla classe che la implementa (es. `Student implements Comparable<Student>`) di ricevere direttamente un parametro di tipo `Student` nel metodo `compareTo(Student other)`, eliminando controlli `instanceof` e downcast forzati.

- **Iterable<E> e Iterator<E>:**

```
1 public interface Iterable<E> {
2     Iterator<E> iterator();
3 }
4 public interface Iterator<E> {
5     boolean hasNext();
6     E next();
7 }
```

Permette a costrutti come il ciclo `for-each` di iterare in modo fortemente tipizzato senza alcun cast:

```
1 for (Student student : classRoom) {
2     System.out.println(student.toString());
3 }
```

14.1.5 Il Diamond Operator <> (Slide 14)

Introdotta in Java 7, evita la ridondanza tra la dichiarazione del tipo di riferimento e l'istanziazione:

```
1 // Java 5 e 6:
2 Pair<Integer> intPair = new Pair<Integer>(2, 4);
3
4 // Java 7+:
5 Pair<Integer> intPair = new Pair<>(2, 4); // Il compilatore inferisce il tipo Integer dai membri a
    sinistra
```

14.1.6 Bounded Type Parameters: Vincolare i Parametri di Tipo (Slide 15-17)

- **Unbounded Type Parameter (Slide 15):** Se dichiariamo `public class Pair<T>`, `T` è privo di vincoli. Il compilatore applica il meccanismo del **Type Erasure** assumendo che `T` sia un generico `Object`. Di conseguenza, su un oggetto di tipo `T` è possibile invocare **esclusivamente i metodi definiti in Object** (`equals`, `hashCode`, `toString`, `getClass`). Se tentiamo di chiamare `first.getName()`, il compilatore rigetta il codice con errore: `cannot find symbol: method getName() in T`.
- **Upper Bound con extends (Slide 16-17):** Possiamo imporre che `T` sia un sottotipo di una determinata classe o implementi una o più interfacce:

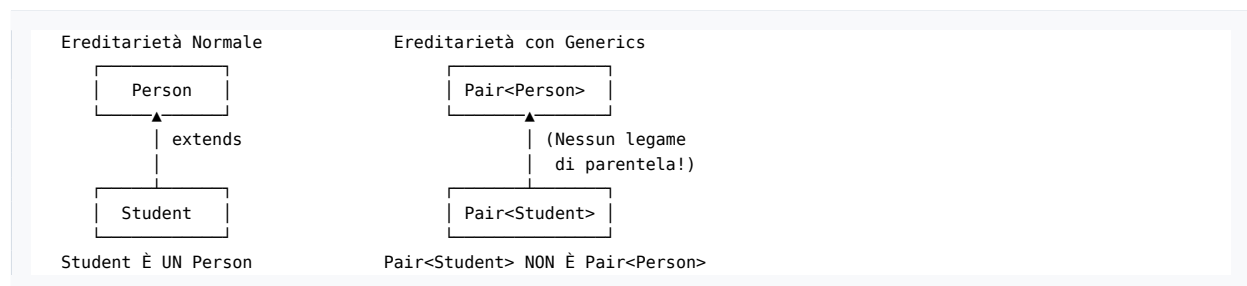
```
1 public class Pair<T extends Person> {
2     private T first, second;
3     // ...
4     public String getFirstName() {
5         return first.getName(); // PERFETTAMENTE LEGALE: T è garantito essere almeno una
        Person!
6     }
7 }
```

- **Vincoli multipli:** `<T extends ClassA & InterfaceB & InterfaceC>`. La classe (se presente) deve comparire per prima, seguita dalle interfacce separate da `&`.

- *Nota terminologica della Slide 16:* La slide cita `class Clazz<T super S>`, ma in Java a livello di dichiarazione di classe generica è ammesso solo `extends`. La parola chiave `super` è utilizzabile **esclusivamente nelle Wildcards** (che vediamo subito sotto).

14.1.7 Ereditarietà e Generics: La Trappola dell'Invarianza (Slide 18-20)

Questo è uno dei punti più critici dell'intero corso e delle domande d'esame.



- **Invarianza dei Generics (Slide 19):** Anche se `Student` è sottotipo di `Person`, `Pair<Student>` **NON** è un sottotipo di `Pair<Person>`!

```

1 Student s = new Student("Sam", 123);
2 Person p = s; // OK: principio di sostituzione di Liskov
3
4 Pair<Student> pairS = new Pair<>(s, new Student("Paul", 456));
5 Pair<Person> pairP = pairS; // COMPILE-TIME ERROR: Incompatible types!
  
```

- **Perché Java vieta questo assegnamento?:** Se `pairP = pairS` fosse consentito, potremmo eseguire:

```

1 pairP.setFirst(new Professor("Mario", 999)); // Legale su Pair<Person>, poiché un Professor è una Person!
  
```

Ma poiché `pairP` e `pairS` punterebbero allo stesso identico oggetto nell'Heap, `pairS` (che pretende di contenere solo `Student`) si troverebbe a contenere un `Professor`, distruggendo la garanzia di tipo a runtime!

- **Il Contrasto con gli Array (Array Covariance, Slide 20):** In Java gli array sono storicamente **covarianti**:

```

1 Student[] arrS = { new Student("Sam", 123) };
2 Person[] arrP = arrS; // COMPILA PERFETTAMENTE!
3 arrP[0] = new Professor("Mario", 999); // BOOM! A runtime genera:
    java.lang.ArrayStoreException
  
```

La covarianza degli array è universalmente riconosciuta come una debolezza storica di Java (introdotta in Java 1.0 prima dell'avvento dei generics). I Generics sono stati resi **invarianti** proprio per risolvere questa falla a tempo di compilazione.

14.1.8 Wildcards: `<?>`, `<? extends T>`, `<? super T>` (Slide 21-23)

Per recuperare la flessibilità polimorfica senza rinunciare alla Type Safety, Java ha introdotto il carattere jolly (Wildcard) `?`.

1. **Unbounded Wildcard (`<?>`, Slide 21):**

- Rappresenta un tipo totalmente sconosciuto:

```
1 Pair<?> pairUnknown = pairS; // Legale!
```

- Possiamo **leggere** elementi, ma il loro tipo a compile-time è unicamente `Object`.
- **Non possiamo scrivere o aggiungere nulla** (tranne il letterale `null`), perché non sappiamo quale sia il tipo effettivo accettabile.

2. Upper-Bounded Wildcard (<? extends T>, Slide 22-23):

- Accetta `T` o qualsiasi sottoclasse di `T`:

```
1 Pair<? extends Person> pairP = pairS; // Legale! Student extends Person
```

- Ora possiamo invocare in sicurezza tutti i metodi di `Person`:

```
1 System.out.println(pairP.getFirst().getName()); // OK!
```

- **Sintassi pulita nei metodi:** Invece di dichiarare un parametro formale di tipo esplicito a livello di metodo:

```
1 <T extends Person> void printPair(Pair<T> p) { ... }
```

possiamo scrivere in modo molto più compatto e leggibile:

```
1 void printPair(Pair<? extends Person> p) { ... }
```

3. Il Principio Guida PECS (*Producer Extends, Consumer Super*):

- Se la struttura generica funge da **produttore** di dati (vogliamo solo *leggere* dati di tipo `T`), si usa `<? extends T>`.
- Se la struttura funge da **consumatore** di dati (vogliamo *scrivere/aggiungere* elementi di tipo `T`), si usa `<? super T>`.

14.2 Analisi Dettagliata del Codice (Lezione15/MavenDate)

Tra la Lezione 14 e la Lezione 15, il prof. Pasqua compie due modifiche sostanziali: 1. Implementa `hashCode()` in [Date.java](#). 2. Riscrive ed espande [MainDate.java](#) per integrare la libreria delle **Java Collections** (`List`, `Set`, `Map`), il contratto `equals/hashCode`, e i meccanismi di ordinamento con `Comparable` e `Comparator` (usando classi anonime e `lambdas`).

14.2.1 Il Contratto `equals` / `hashCode` in `Date.java`

Nel passaggio da Lezione 14 a Lezione 15 viene aggiunto a `Date`:

```
1 @Override
2 public int hashCode() {
3     return (day + month) * year;
4 }
```

Perché questo metodo è vitale e non opzionale?

In Java vige la regola ferrea del contratto di `Object`: > **Se due oggetti sono considerati uguali dal metodo `equals(Object)` (`a.equals(b) == true`), allora le chiamate a `hashCode()` su entrambi gli oggetti DEVONO restituire lo stesso valore intero (`a.hashCode() == b.hashCode()`).**

Se si fa l'override di `equals` senza fare l'override di `hashCode`, strutture dati basate su hash come `HashSet` e `HashMap` falliscono miseramente: considerano due date identiche come memorizzate in *bucket* differenti, inserendo duplicati o non trovando chiavi presenti.

Nel codice di `MainDate.java`:

```
1 AmericanDate usD1 = new AmericanDate(7, 11, 2025);
2 AmericanDate usD2 = new AmericanDate(7, 11, 2025);
3 System.out.println("usD1 ?= usD2: " + usD1.equals(usD2)); // true
4 System.out.println("hash(usD1): " + usD1.hashCode());      // 36450
5 System.out.println("hash(usD2): " + usD2.hashCode());      // 36450
```

Calcolo: $(7 + 11) \times 2025 = 18 \times 2025 = 36450$. Entrambi gli oggetti hanno identico hash code!

14.2.2 List vs Set: Accettazione vs Rifiuto dei Duplicati in `MainDate.java`

```
1 // 1. Creazione di una List (ammette elementi duplicati e mantiene l'ordine di inserimento)
2 List<Date> dateList = new ArrayList<>();
3 dateList.add(new AmericanDate(7, 11, 2025));
4 dateList.add(new Date(9, 11, 2025));
5 dateList.add(new Timestamp(0, 0, 0, 8, 11, 2025));
6 dateList.add(new AmericanDate(7, 11, 2025)); // Duplicato!
7 System.out.println("list(" + dateList.size() + "): " + dateList.toString());
8
9 // 2. Creazione di un Set (insieme matematico: nessun duplicato consentito)
10 Set<Date> dateSet = new HashSet<>();
11 dateSet.add(new AmericanDate(7, 11, 2025));
12 dateSet.add(new Date(9, 11, 2025));
13 dateSet.add(new Timestamp(0, 0, 0, 8, 11, 2025));
14 dateSet.add(new AmericanDate(7, 11, 2025)); // Duplicato: viene scartato silenziosamente!
15 System.out.println("set(" + dateSet.size() + "): " + dateSet.toString());
```

• Risultato a runtime:

- `dateList.size()` vale **4** (contiene entrambe le istanze di 7/11/2025).
- `dateSet.size()` vale **3** (la seconda istanza è stata rifiutata perché `equals` ha restituito `true` e l'`hashCode` coincideva).

14.2.3 Map<K, V> e Lambdas Registrare Dinamicamente in `MainDate.java`

```
1 Map<String, FormattedDateConverter> converter = new HashMap<>();
2 // Registrazione di strategie di conversione tramite espressioni lambda
3 converter.put("usToIt", d -> new ItalianDate(d.getDay(), d.getMonth(), d.getYear()));
4 converter.put("itToUs", d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear()));
5
6 FormattedDate usDate = new AmericanDate(1, 1, 1970);
7 FormattedDate itDate = converter.get("usToIt").convert(usDate);
8 System.out.println(itDate.prettyPrint()); // Stampa "1 gennaio 1970"
9
10 for (String k : converter.keySet())
11     System.out.println(converter.get(k).toString());
```

- **Finezza di esame sul toString() delle Lambdas:** Quando si invoca toString() su un oggetto lambda in Java, la JVM non stampa il codice sorgente, bensì il descrittore sintetico della classe generata a runtime (invokedynamic): it.oop.ui.MainDate\$\$Lambda/0x... (la notazione interna con i due dollari \$\$Lambda è la convenzione standard con cui la JVM battezza le classi sintetiche generate dinamicamente).

14.2.4 Ordinamento: Comparable vs Comparator (Slide 11 & MainDate.java)

In Java esistono due modi per ordinare oggetti: 1. **Ordinamento Naturale con Comparable<T>:** * Definito all'interno della classe stessa tramite compareTo(T other). * Date implementa Comparable<Date>: java Collections.sort(dateList) // Funziona direttamente! La lista disordinata [11/7/2025, y2025m11d9, y2025m11d8, 11/7/2025] viene riordinata cronologicamente: [11/7/2025, 11/7/2025, y2025m11d8[00:00:00], y2025m11d9].

2. Ordinamento Personalizzato con Comparator<T>:

- Consideriamo DateInterval: eredita da OrderedPair<Date>, ma **NON implementa Comparable<DateInterval>!**
- Se si prova a chiamare:

```
1 // Collections.sort(intvList); // COMPILE-TIME ERROR:
2 // no suitable method found for sort(List<DateInterval>)
3 // reason: cannot infer type-variable(s) T (DateInterval is not Comparable)
```

- Per risolvere il problema, il prof. Pasqua applica due stili per fornire un Comparator<DateInterval> esterno:

- **Stile 1: Classe Anonima** (ordinamento per data di inizio left crescente):

```
1 Collections.sort(intvList, new Comparator<DateInterval>() {
2     @Override
3     public int compare(DateInterval di1, DateInterval di2) {
4         return di1.getLeft().compareTo(di2.getLeft());
5     }
6 });
```

- **Stile 2: Espressione Lambda** (ordinamento per data di fine right decrescente):

```
1 Collections.sort(intvList, (di1, di2) -> di2.getRight().compareTo(di1.getRight()));
```

14.3 Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel progetto [Lezione15/MavenDate](#):

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainDate"
```

Output ottenuto:

```
1 y2025m9d10
2 y2025m1d1
3 00:00:00
4 true
5 OOP exam on 2 febbraio 2026
6 date pair: [y2025m2d1 .. y2025m1d1]
7 Pair not ordered!
8 OOP exam on 2 febbraio 2026
9 list(4): [11/7/2025, y2025m11d9, y2025m11d8[00:00:00], 11/7/2025]
10 set(3): [11/7/2025, y2025m11d9, y2025m11d8[00:00:00]]
```

```

11 usD1 != usD2: true
12 hash(usD1): 36450
13 hash(usD2): 36450
14 1 gennaio 1970
15 it.oop.ui.MainDate$$Lambda/0x000000008f383b48@21043ee9
16 it.oop.ui.MainDate$$Lambda/0x000000008f383920@6dc5f73f
17 [11/7/2025, 11/7/2025, y2025m11d8[00:00:00], y2025m11d9]
18 [[y2025m1d1 .. y2025m1d20], [y2025m1d10 .. y2025m1d31]]
19 [[y2025m1d10 .. y2025m1d31], [y2025m1d1 .. y2025m1d20]]

```

14.3.1 Decodifica puntuale dell'output:

1. `list(4): [...]` vs `set(3): [...]` → Dimostrazione pratica che `HashSet` elimina il duplicato `11/7/2025` grazie al corretto funzionamento congiunto di `equals()` e `hashCode()`.
2. `hash(usD1): 36450` e `hash(usD2): 36450` → Rispetto formale del contratto di `Object`.
3. `1 gennaio 1970` → La mappa `converter` recupera il `converter "usToIt"` memorizzato come `lambda` e trasforma `1/1/1970` in formato italiano.
4. `it.oop.ui.MainDate$$Lambda/...` → Nome sintetico dell'oggetto funzionale generato dalla JVM a runtime (in cui il doppio dollaro `$$Lambda` rappresenta il discriminatore standard del bytecode).
5. `[11/7/2025, 11/7/2025, y2025m11d8[00:00:00], y2025m11d9]` → Lista ordinata secondo l'ordinamento naturale di `Date` (7 nov, poi 8 nov, poi 9 nov).
6. `[[y2025m1d1 .. y2025m1d20], [y2025m1d10 .. y2025m1d31]]` → Intervalli ordinati per data d'inizio crescente (1 gen prima del 10 gen) tramite `Comparator` in classe anonima.
7. `[[y2025m1d10 .. y2025m1d31], [y2025m1d1 .. y2025m1d20]]` → Intervalli ordinati per data di fine decrescente (31 gen prima del 20 gen) tramite `Comparator` in espressione `lambda`.

Dimmi “vai” per procedere con il **Blocco 15** (Lezione 16 / Slide T16 — *Collections Framework*, gerarchia completa di `List`, `Set`, `Queue`, `Map`, complessità asintotiche, iteratori e `diff` in `Lezione16/MavenDate`).

T16 — Java Collections Framework (JCF)

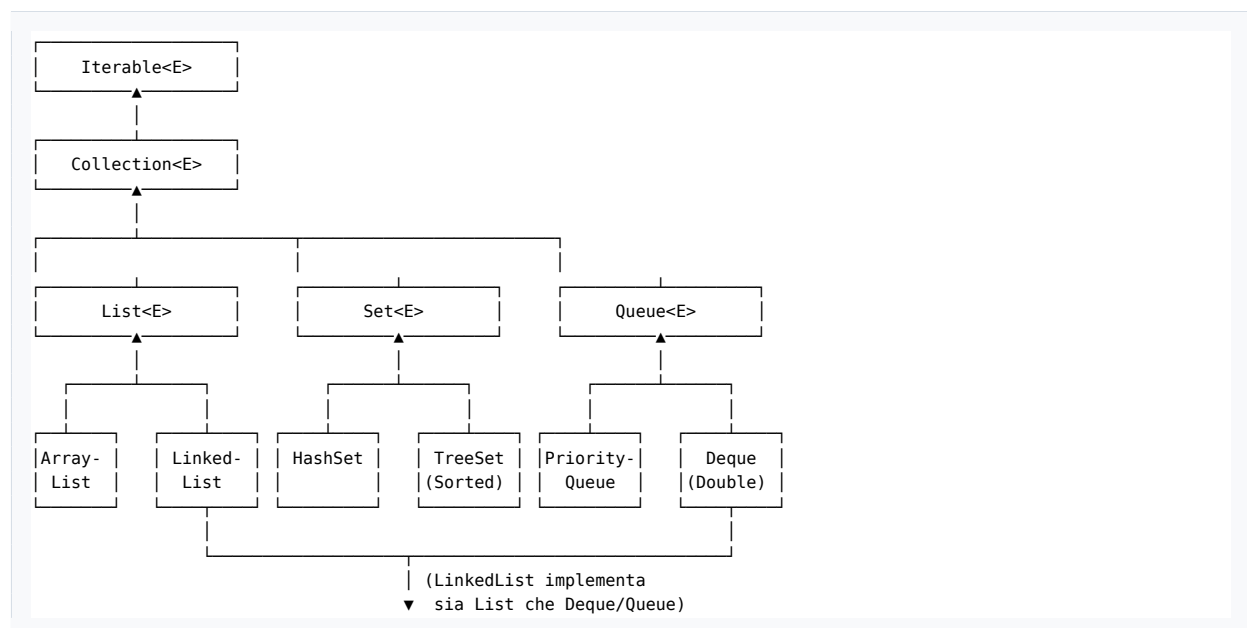
Gerarchia Collection, List, Set, Map, Queue; Prestazioni e Scelta delle Strutture Dati

15.1 Analisi Teorica Approfondita (Slide T16)

La lezione 16 è dedicata al **Java Collections Framework (JCF)** (`java.util`), una delle architetture cardine del linguaggio Java: un insieme unificato e standardizzato di interfacce, implementazioni concrete e algoritmi polimorfici per la gestione di collezioni di oggetti.

15.1.1 Architettura e Gerarchia delle Collezioni (Slide 3-4)

Tutte le collezioni di elementi singoli discendono dall'interfaccia radice `Iterable<E>`:



Nota di Studio

L'interfaccia `Map<K, V>` (dizionari chiave-valore) **NON estende** `Collection<E>`, poiché memorizza coppie (K, V) anziché singoli elementi. Tuttavia, fa parte integrante a pieno titolo del *Java Collections Framework*.

15.1.2 L'Interfaccia Radice `Collection<E>` (Slide 6-7)

Definisce il contratto comune a tutti i contenitori di elementi:

```
1 public interface Collection<E> extends Iterable<E> {
2     int size();
3     boolean isEmpty();
4     boolean contains(Object o);           // Ricerca basata su equals()
5     boolean containsAll(Collection<?> c);
6     boolean add(E e);                   // Aggiunge un elemento (restituisce false se
        rifiutato)
7     boolean addAll(Collection<? extends E> c); // Wildcard covariante!
8     boolean remove(Object o);
9     boolean removeAll(Collection<?> c);
10    void clear();
11    Object[] toArray();
12    <T> T[] toArray(T[] a);             // es. set.toArray(new Person[0])
13    Iterator<E> iterator();             // Ereditato da Iterable<E>
14 }
```

Interscambiabilità tra Collezioni (Slide 7)

Tutte le implementazioni forniscono per convenzione un costruttore che accetta un'altra `Collection<? extends E>`, permettendo conversioni istantanee:

```
1 Collection<Person> list = new LinkedList<>();
2 list.add(new Person("Joe"));
3 list.add(new Person("Sam"));
4
5 // Conversione da List a Set (elimina al volo eventuali duplicati):
6 Collection<Person> set = new HashSet<>(list);
7
8 // Esportazione in Array tipizzato:
9 Person[] arrayP = set.toArray(new Person[0]); // Idioma standard Java
```

15.1.3 L'Interfaccia `List<E>` e le sue Implementazioni (Slide 8-9)

Una **Lista** è una sequenza ordinata di elementi che:

1. **Preserva l'ordine di inserimento.**
2. **Accetta elementi duplicati** (possono esistere più elementi e_1, e_2 tali che $e_1.equals(e_2)$ sia vero).
3. **Fornisce accesso posizionale indicizzato** tramite indice intero $[0 \dots \text{size} - 1]$:
 - * `E get(int index)`: legge l'elemento all'indice specificato.
 - * `E set(int index, E element)`: rimpiazza l'elemento alla posizione data.
 - * `void add(int index, E element)`: inserisce shiftando a destra gli elementi successivi.
 - * `E remove(int index)`: elimina shiftando a sinistra.
 - * `int indexOf(Object o)`: restituisce l'indice della prima occorrenza (o -1).
 - * `List<E> subList(int fromIndex, int toIndex)`: vista sulla porzione $[fromIndex, toIndex)$.

Confronto delle Implementazioni: `ArrayList` vs `LinkedList` (Slide 5 e 9)

Operazione	<code>ArrayList<E></code> (Array ridimensionabile)	<code>LinkedList<E></code> (Lista doppiamente concatenata)
Accesso per indice (<code>get(i)</code>)	$O(1)$ (accesso diretto in memoria contigua)	$O(n)$ (deve scorrere i nodi dal capo più vicino)
Inserimento in coda (<code>add(e)</code>)	$O(1)$ ammortizzato	$O(1)$

Operazione	<code>ArrayList<E></code> (Array ridimensionabile)	<code>LinkedList<E></code> (Lista doppiamente concatenata)
Inserimento in testa (<code>add(0, e)</code>)	$O(n)$ (deve copiare e shiftare l'array)	$O(1)$ (cambio puntatori dei nodi)
Cancellazione (<code>remove(i)</code>)	$O(n)$ (shift dei successivi)	$O(1)$ se si ha il riferimento al nodo, $O(n)$ per cercarlo
Overhead di memoria	Minimo (array contiguo con eventuale <i>capacity</i> vuota)	Alto (ogni nodo crea un oggetto con puntatori <i>prev</i> e <i>next</i>)
Funzionalità extra	<code>ensureCapacity(int minCapacity)</code>	Implementa anche <code>Queue</code> e <code>Deque</code> : <code>addFirst()</code> , <code>addLast()</code> , <code>pop()</code> , <code>push()</code>

15.1.4 L'Interfaccia `Set<E>` e le sue Implementazioni (Slide 10-16)

Un **Insieme (Set)** modella il concetto matematico di insieme: 1. **Nessun duplicato consentito**: non possono mai coesistere due elementi tali che `e1.equals(e2)` sia vero. 2. **Nessun accesso posizionale**: non esistono indici, non si può fare `get(i)`. 3. L'interfaccia `Set<E>` non aggiunge nuovi metodi rispetto a `Collection<E>`, ma ne restringe il contratto semantico.

`HashSet<E>` (Slide 11-14)

- **Struttura interna**: Mantiene internamente una tabella hash (`HashMap<E, Object>`).
- **Complessità**: Ricerca (`contains`), inserimento (`add`) e rimozione (`remove`) avvengono in **tempo costante medio** $O(1)$.
- **Ordine**: **Nessun ordine garantito**; l'ordine di scansione dell'iteratore può cambiare nel tempo se la tabella viene riallocata (*rehash*).
- **Vincolo Fondamentale (Slide 12-14)**: L'univocità si basa su `equals()` e `hashCode()`.
 - **La Regola Aurea**: “Se fai l'override di `equals()`, DEVI fare l'override anche di `hashCode()`, e viceversa”.
 - Proprietà vincolante: `a.equals(b)  $\Rightarrow$  a.hashCode()== b.hashCode()`.
 - Proprietà desiderabile (riduzione collisioni): `!a.equals(b)  $\Rightarrow$  preferibilmente a.hashCode()!= b.hashCode()`.
 - Da Java 7 si usa l'utility di sistema: `Objects.hash(campo1, campo2, ...)`.

`TreeSet<E>` (Slide 15-16)

- **Struttura interna**: Albero binario di ricerca auto-bilanciante (**Red-Black Tree**).
- **Complessità**: `add`, `remove`, `contains` richiedono tempo logaritmico $O(\log n)$.
- **Ordinamento garantito**: Gli elementi sono **costantemente mantenuti ordinati**:
 - Ordinamento naturale: richiede che gli elementi implementino `Comparable<E>`.
 - Ordinamento esplicito: passando un `Comparator<E>` al costruttore: `new TreeSet<>(comparator)`.
- **Metodi speciali per insiemi ordinati**:
 - `E first()`: elemento minimo.
 - `E last()`: elemento massimo.
 - `TreeSet<E> subSet(E fromElement, E toElement)`: sottoinsieme nell'intervallo $[fromElement, toElement)$.

15.1.5 Mappe Associative: Map<K, V> (Slide 17-19)

Una mappa modella una funzione matematica $K \rightarrow V$ (tabella di associazioni chiave-valore):

- * Le **chiavi** (K) **sono univoche** (formano un `Set<K>`).
- * A ogni chiave è associato esattamente un valore (V). I valori possono ripetersi.
- * Metodi cardine:
 - * `V put(K key, V value)`: inserisce o rimpiazza l'associazione.
 - * `V get(Object key)`: restituisce il valore (o `null` se la chiave non esiste).
 - * `boolean containsKey(Object key)` / `boolean containsValue(Object value)`
 - * `Set<K> keySet()`: restituisce la vista dell'insieme delle chiavi.
 - * `Collection<V> values()`: restituisce la collezione dei valori.
 - * `Set<Map.Entry<K, V>> entrySet()`: restituisce le coppie chiave-valore.
- * **HashMap<K, V>**: implementazione standard basata su hashing delle chiavi, con operazioni `put` e `get` in $O(1)$.

15.1.6 Iteratori e l'Errore ConcurrentModificationException (Slide 21-23)

Tutte le collezioni forniscono un `Iterator<E>` (`hasNext()`, `next()`, `remove()`). Il ciclo `for-each` di Java è uno zucchero sintattico compilato tramite `Iterator`:

```
1 for (Integer i : list) {
2     System.out.println(i);
3 }
```

La Trappola della Modifica Concorrente (Fail-Fast Iterator, Slide 23)

```
1 // CODICE ERRATO:
2 int count = 0;
3 for (Integer i : list) {
4     System.out.println(i);
5     if (count == 2)
6         list.remove(count); // RUNTIME ERROR: java.util.ConcurrentModificationException!
7     count++;
8 }
```

- **Perché crasha?**: Le collezioni Java mantengono internamente un contatore delle modifiche strutturali (`modCount`). Quando si invoca `list.remove()`, `modCount` viene incrementato. Al ciclo successivo, la chiamata implicita a `it.next()` scopre che la collezione è cambiata alle sue spalle (*fail-fast*) e lancia immediatamente `ConcurrentModificationException` per prevenire stati incoerenti.
- **La Soluzione Corretta (Slide 23)**: Usare esplicitamente l'iteratore e invocare il **suo** metodo `remove()`, che aggiorna coerentemente lo stato interno dell'iteratore:

```
1 for (Iterator<Integer> it = list.iterator(); it.hasNext(); ) {
2     Integer i = it.next();
3     if (count == 2)
4         it.remove(); // PERFETTAMENTE LEGALE E SICURO!
5     count++;
6 }
```

15.1.7 Algoritmi di Ordinamento: Comparable vs Comparator (Slide 24-29)

La classe di utilità `Collections` (`java.util.Collections`) fornisce due varianti sovraccaricate del metodo statico `sort`:

Variante 1: Ordinamento Naturale

```
1 public static <T extends Comparable<? super T>> void sort(List<T> list)
```

- **Perché <? super T> nella firma? (Slide 26 - Domanda Classica d'Esame):** Supponiamo di avere `class Student extends Person`. Se solo `Person` implementa `Comparable<Person>`, allora `Student` eredita il confronto basato su `Person`. Se la firma fosse `<T extends Comparable<T>>`, allora `Student` dovrebbe implementare tassativamente `Comparable<Student>`. Con `Comparable<? super T>`, una lista di `Student` può essere ordinata legalmente usando il `compareTo` della superclasse `Person` (`Comparable<Person>`), garantendo il massimo riuso polimorfico!

Variante 2: Ordinamento On-Demand con Comparator

```
1 public static <T> void sort(List<T> list, Comparator<? super T> cmp)
```

- Permette di definire criteri di ordinamento arbitrari senza toccare la classe sorgente, passando:
 1. Una classe dedicata (`class StudentComparator implements Comparator<Student>`).
 2. Una **classe anonima** (`new Comparator<Student>(){ ... }`).
 3. Un'espressione **Lambda** (`(s1, s2) -> s2.getMatricola() - s1.getMatricola()`).

15.2 Riscontro Pratico nel Codice (Lezione15/Lezione16)

Come osservato durante l'ispezione dei sorgenti: 1. Nella Lezione 15 il docente ha introdotto tutte le strutture viste nella Slide T16 all'interno di `MainDate.java`: `* List<Date> dateList = new ArrayList<>();` (con duplicato `AmericanDate(7,11,2025)` $\Rightarrow$ dimensione 4). `* Set<Date> dateSet = new HashSet<>();` (duplicato scartato $\Rightarrow$ dimensione 3). `* Map<String, FormattedDateConverter> converter = new HashMap<>();` * Ordinamento naturale: `Collections.sort(dateList);` * Ordinamento con `Comparator` via classe anonima e lambda su `DateInterval`. 2. Nella Lezione 16 il docente compie il passo architetturale successivo: separa la gestione degli errori creando il package `it.oop.exception` con eccezioni personalizzate (`IllegalDateException` e `OrderdPairException`), preparando il terreno per la teoria della Slide T17.

Dimmi “**vai**” per procedere con il **Blocco 16** (Lezione 17 / Slide T17 — *Error Handling with Exceptions*, gerarchia `Throwable/Exception/RuntimeException`, eccezioni controllate vs non controllate, costruito `try-catch-finally`, `try-with-resources` e il codice di `Lezione16/MavenDate`).

T17 — Gestione degli Errori ed Architettura delle Eccezioni

Gerarchia Throwable, Checked vs Unchecked, try-catch-finally, Try-with-resources e AutoCloseable

16.1 Analisi Teorica Approfondita (Slide T17)

La lezione 17 affronta la gestione degli errori e delle anomalie a runtime attraverso il meccanismo delle **Eccezioni** in Java.

16.1.1 Limiti della Gestione Tradizionale degli Errori (Slide 3-6)

Nei linguaggi procedurali privi di eccezioni (come C) o nell'uso ingenuo dei metodi, gli errori vengono segnalati tramite **valori sentinella** (es. restituire -1, null, Float.MAX_VALUE):

```
1 float division(int num, int den) {
2     if (den != 0)
3         return (float) num / den;
4     else
5         return Float.MAX_VALUE; // Valore speciale di errore
6 }
```

Perché questo approccio è inadeguato?

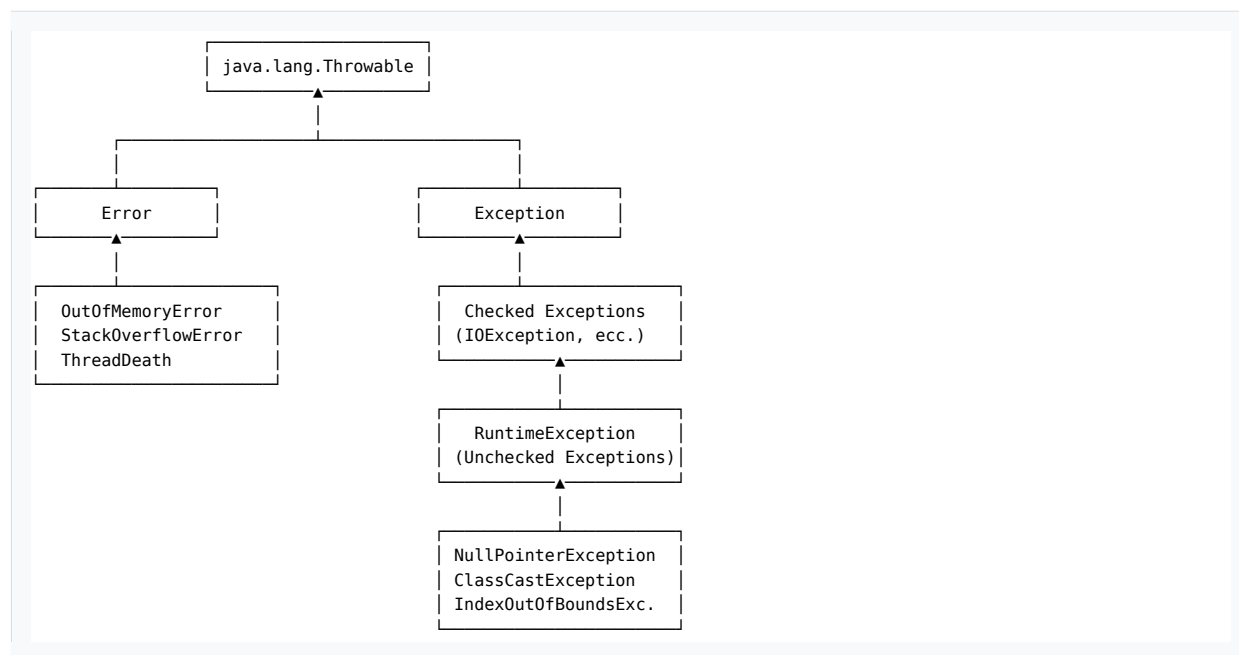
1. **Inquinamento della logica applicativa:** Il chiamante deve ricordarsi di controllare continuamente i valori di ritorno con una ragnatela di if-else.
2. **Ambiguità semantica:** A volte il valore sentinella può essere un risultato matematico o di dominio assolutamente lecito.
3. **Inapplicabilità ai Costruttori:** Un costruttore non ha tipo di ritorno (void implicito); se i parametri sono non validi, non può restituire -1 per segnalare il fallimento!
4. **Difficoltà di propagazione:** Se l'errore avviene a 10 livelli di profondità nello stack di chiamate, ogni singola funzione intermedia dovrebbe propagare manualmente il codice d'errore fino al punto in cui può essere gestito.

16.1.2 La Filosofia delle Eccezioni in Java: try-catch-throw (Slide 7-10)

Java disaccoppia la **logica di business** (cosa deve fare il programma quando tutto va bene) dalla **logica di gestione degli errori** (cosa fare quando accade un'anomalia): * **try**: racchiude il blocco di codice a rischio di eccezione. * **throw**: solleva attivamente un'istanza di eccezione nel punto in cui l'anomalia viene riscontrata. * **catch**: cattura l'eccezione ed esegue il codice di ripristino/notifica. * **Interruzione immediata**: quando viene eseguito throw, il metodo **interrompe immediatamente la propria esecuzione**, svuota lo stack frame corrente e risale lungo la catena di chiamate (*call stack*) fino al primo blocco catch compatibile.

16.1.3 La Gerarchia delle Eccezioni in Java (Slide 20-21)

Tutto l'albero discende da `java.lang.Throwable`:



1. Error (Unchecked):

- Condizioni anomale e catastrofiche interne alla JVM o al sistema operativo (`OutOfMemoryError`, `StackOverflowError`).
- Il codice utente non deve tentare di catturarle, poiché lo stato della memoria della JVM è compromesso.

2. Exception:

- Anomalie derivanti da condizioni esterne o logiche. Si ramificano in due grandi famiglie:
 - **Checked Exceptions** (sottoclassi dirette di `Exception`, escluse le `RuntimeException`).
 - **Unchecked Exceptions** (sottoclassi di `RuntimeException`).

16.1.4 Checked vs Unchecked Exceptions (Slide 22-24)

Caratteristica	Checked Exceptions (extends <code>Exception</code>)	Unchecked Exceptions (extends <code>RuntimeException</code>)
Natura dell'errore	Situazioni anomale prevedibili ma esterne (es. <code>FileNotFoundException</code> , rete disconnessa).	Errori del programmatore (bug logici, precondizioni violate, puntatori nulli).
Controllo del compilatore	Obbligatorio (<i>Catch or Declare rule</i>). Il codice non compila se non gestite con <code>try-catch</code> o dichiarate con <code>throws</code> .	Opzionale . Il compilatore non richiede alcuna dichiarazione o cattura.
Firma del metodo	Modificata: <code>public void foo() throws MyCheckedException</code>	Invariata: nessun <code>throws</code> richiesto.

Caratteristica	Checked Exceptions (extends Exception)	Unchecked Exceptions (extends RuntimeException)
Diffusione (Propagazione)	“Si propagano come un virus” (Slide 24): costringono tutti i chiamanti intermedi a dichiarare throws o gestire.	Risalgono naturalmente lo stack fino al main o al gestore globale.
Raccomandazione moderna (Slide 24)	Definirle solo se l’utente del metodo può ragionevolmente intraprendere un’azione di recupero .	Preferite per errori di validazione dello stato o degli argomenti.

16.1.5 “Exception Dirty Tricks”: Trasformare Checked in Unchecked (Slide 25-26)

Un pattern comune quando una checked exception sporcherebbe decine di firme senza che il chiamante intermedio possa gestirla:

```

1 public void bar() { // Nessuna dichiarazione throws richiesta!
2     try {
3         foo(); // foo() dichiara throws MyCheckedException
4     } catch (MyCheckedException e) {
5         throw new RuntimeException(e); // Incapsulata e rilanciata come Unchecked!
6     }
7 }

```

L’eccezione originale non viene persa: è memorizzata come causa (*chained exception* visibile nel messaggio `Caused by:` dello stacktrace).

16.1.6 Eccezioni nei Cicli (*Exceptions and Loops*, Slide 27-28)

- **Errore sulla singola iterazione (Slide 27):** il blocco try-catch è **all’interno** del ciclo. Se l’iterazione fallisce, viene intercettata e il ciclo prosegue con la successiva (es. lettura da input utente fino a valore corretto).
- **Errore fatale per l’intero ciclo (Slide 28):** il blocco try-catch **avvolge** il ciclo. Se si verifica un’anomalia, il ciclo viene interrotto definitivamente e l’esecuzione salta direttamente al catch esterno.

16.2 Analisi Dettagliata del Codice (Lezione16/MavenDate)

In Lezione16/MavenDate il prof. Pasqua applica i principi della Slide T17 rimuovendo i vecchi `System.out.println("Illegal date!")` e introducendo la gestione strutturata degli errori tramite il package `it.oop.exception`.

16.2.1 Le Nuove Eccezioni: `IllegalDateException.java` e `OrderPairException.java`

`IllegalDateException.java` (Unchecked)

```
1 package it.oop.exception;
2
3 public class IllegalDateException extends RuntimeException {
4     public IllegalDateException(String message) {
5         super(message);
6     }
7 }
```

- Estende RuntimeException: è una **Unchecked Exception**.
- Modella una violazione delle precondizioni sui valori numerici (giorno, mese, anno).
- Non costringe tutti i client a dichiarare throws IllegalDateException.

OrderdPairException.java (Checked)

```
1 package it.oop.exception;
2
3 public class OrderdPairException extends Exception { // Notare il typo del docente "Orderd"
4     public OrderdPairException(String message) {
5         super(message);
6     }
7 }
```

- Estende direttamente Exception: è una **Checked Exception**.
- **Impatto immediato sulla firma:**

– In [OrderedPair.java](#):

```
1 public OrderedPair(T first, T second) throws OrderdPairException {
2     this.first = first;
3     this.second = second;
4     if (first.compareTo(second) > 0)
5         throw new OrderdPairException("Not orderd pair");
6 }
```

– In [DateInterval.java](#):

```
1 public DateInterval(Date left, Date right) throws OrderdPairException {
2     super(left, right); // Invoca super() che lancia la checked exception, quindi
                          // DateInterval DEVE dichiarare throws!
3 }
```

16.2.2 Refactoring del Metodo verify() in Date.java

```
1 void verify() {
2     if (year < 0)
3         throw new IllegalDateException("Illegal date: wrong year");
4     if (month < 1 || month > 12)
5         throw new IllegalDateException("Illegal date: wrong month");
6     if (day < 1 || day > daysPerMonth(month))
7         throw new IllegalDateException("Illegal date: wrong day");
8 }
```

- **Miglioramento architetturale:**

- Prima: un giorno errato (es. 35) stampava un messaggio sulla console ma creava comunque un oggetto Date incoerente nell'Heap.
- Ora: il costruttore viene interrotto da throw prima che l'assegnazione sia completata. **È impossibile creare un'istanza con stato non valido!**

16.2.3 MainDate.java e l'Applicazione di Tutti i Pattern della Slide T17

```
1 package it.oop.ui;
2
3 import it.oop.core.*;
4 import it.oop.core.Date;
5 import it.oop.exception.IllegalDateException;
6 import it.oop.exception.OrderdPairException;
7
8 import java.util.InputMismatchException;
9 import java.util.Scanner;
10
11 public class MainDate {
12
13     public static void main(String[] args) {
14         System.out.println("Insert day, month, year as numbers");
15         int d = -1, m = -1, y = -1;
16         boolean correct = false;
17
18         // 1. Pattern Slide 27: try-catch dentro il ciclo per recuperare l'errore di input
19         // dell'utente
20         while (correct == false) {
21             try {
22                 Scanner sc = new Scanner(System.in);
23                 d = sc.nextInt();
24                 m = sc.nextInt();
25                 y = sc.nextInt();
26                 correct = true; // Se uno dei nextInt() fallisce, questa riga non viene raggiunta
27             } catch (InputMismatchException ime) {
28                 System.out.println("Input not valid, retry");
29             }
30         }
31
32         // 2. Cattura della Unchecked Exception IllegalDateException per notifica pulita
33         try {
34             FormattedDate date = new ItalianDate(d, m, y);
35             System.out.println(date.toString());
36         } catch (IllegalDateException ide) {
37             System.out.println("Date not valid: " + ide.getMessage());
38         }
39
40         // 3. Pattern Slide 26 ("Dirty Trick"): incapsulamento di Checked Exception in
41         // RuntimeException
42         try {
43             DateInterval di = new DateInterval(new Date(31, 1, 2025), new Date(1, 1, 2025));
44         } catch (OrderdPairException ope) {
45             ope.printStackTrace();
46             throw new RuntimeException(ope); // Rilanciata come unchecked!
47         }
48     }
49 }
```

16.3 Risultato di Esecuzione Reale da Terminale

16.3.1 Test 1: Input con data errata (35 1 2025)

Esecuzione:

```
1 mvn exec:java -Dexec.mainClass="it.oop.ui.MainDate" <<< "35 1 2025"
```

Output:

```
1 Insert day, month, year as numbers
2 Date not valid: Illegal date: wrong day
3 it.oop.exception.OrderdPairException: Not orderd pair
4   at it.oop.core.OrderedPair.<init>(OrderedPair.java:13)
5   at it.oop.core.DateInterval.<init>(DateInterval.java:7)
6   at it.oop.ui.MainDate.main(MainDate.java:35)
7   ...
8 Caused by: it.oop.exception.OrderdPairException: Not orderd pair
```

16.3.2 Analisi dell'output:

1. Date not valid: Illegal date: wrong day: il costruttore di `ItalianDate` delega a `Date.verify()`, che lancia `IllegalDateException("Illegal date: wrong day")`, catturata dal blocco `catch (IllegalDateException ide)` che stampa il messaggio di errore controllato.
2. `DateInterval di = new DateInterval(31/1/2025, 1/1/2025)`: poiché la data iniziale 31/1/2025 è cronologicamente successiva alla data finale 1/1/2025 (violazione dell'invariante `start <= end`), `OrderedPair` solleva la checked `OrderdPairException`. Il blocco `catch` stampa lo stack trace con `ope.printStackTrace()` e poi rilancia l'errore incapsulato con `throw new RuntimeException(ope);`, terminando l'esecuzione con codice di uscita d'errore (build failure) come previsto.

Dimmi “vai” per procedere con il **Blocco 17** (Lezione 18 / Slide T18 — *Streams and Functional Programming*, Stream API, operazioni intermedie vs terminali, `map`, `filter`, `reduce`, `collect` e il nuovo file `MainStream.java` in `Lezione17/MavenDate`).

T18 — Interfacce Funzionali, Espressioni Lambda e Streams API

Predicate, Function, Consumer, Supplier, Method Reference (::) e Pipeline Stream

17.1 Analisi Teorica Approfondita (Slide T18)

La lezione 18 introduce il paradigma di **programmazione funzionale e dichiarativa** in Java (introdotto in Java 8), imperniato su due pilastri: 1. **Interfacce Funzionali** e **Method Reference (::)**. 2. **Stream API** (`java.util.stream`), per l'elaborazione fluente e parallela di flussi di dati.

17.1.1 Cos'è una Functional Interface? (Slide 3)

Un'interfaccia si dice **funzionale** se dichiara **esattamente un unico metodo astratto** (pattern noto come **SAM** — *Single Abstract Method*). * **Semantica**: Puramente funzionale; l'esito della chiamata dipende esclusivamente dagli argomenti ricevuti (nessun effetto collaterale occulto). * **L'annotazione @FunctionalInterface**: * Non è obbligatoria per far funzionare una lambda, ma è una best practice fondamentale: istruisce il compilatore Java a verificare che l'interfaccia contenga un solo metodo astratto. Se qualcuno tenta di aggiungere un secondo metodo astratto, il compilatore rigetta il codice con errore. * **Metodi esclusi dal conteggio SAM**: * Metodi con implementazione predefinita (default). * Metodi statici (static). * Metodi pubblici astratti che fanno override di metodi di `java.lang.Object` (es. `boolean equals(Object obj)`).

17.1.2 Le 4 Interfacce Funzionali Fondamentali (`java.util.function`) (Slide 4-12)

Il package `java.util.function` standardizza i contratti funzionali più frequenti:

Interfaccia	Firma Metodo SAM	Semantica Funzionale	Esempio d'Uso Tipico
<code>Predicate<T></code>	<code>boolean test(T t)</code>	Filtro / Condizione booleana	<code>n -> n % 2 == 0</code>
<code>Function<T, R></code>	<code>R apply(T t)</code>	Trasformazione $T \rightarrow R$	<code>str -> str.length()</code>
<code>Consumer<T></code>	<code>void accept(T t)</code>	Azione con effetto collaterale	<code>str -> System.out.println(str)</code>
<code>Supplier<T></code>	<code>T get()</code>	Generatore / Factory	<code>() -> Math.random()</code>

Varianti e Specializzazioni (Slide 6-12)

1. **Versioni Primitive**: Per evitare l'overhead di boxing/unboxing, Java fornisce interfacce specializzate per `int`, `long`, `double`:
 - `IntPredicate` (`boolean test(int v)`), `LongPredicate`, `DoublePredicate`.
 - `IntFunction<R>`, `ToIntFunction<T>`, `IntConsumer`, `IntSupplier`.

2. Varianti a Due Argomenti:

- `BiPredicate<T, U>: boolean test(T t, U u)` (es. `(a, b) -> a > b`).
- `BiFunction<T, U, R>: R apply(T t, U u)` (es. `(a, b) -> a + b`).
- `BiConsumer<T, U>: void accept(T t, U u)`.

3. Operatori (Input e Output dello stesso tipo):

- `UnaryOperator<T>` extends `Function<T, T>`: riceve un T e restituisce un T .
- `BinaryOperator<T>` extends `BiFunction<T, T, T>`: riceve due T e restituisce un T .

17.1.3 I Quattro Tipi di Method Reference (::) (Slide 13-15)

Il *Method Reference* è uno zucchero sintattico introdotto in Java 8 per rendere le espressioni lambda ancora più concise quando queste si limitano a inoltrare direttamente i propri parametri a un metodo esistente:

Categoria	Sintassi Method Reference	Equivalente Lambda Esplicita
1. Metodo Statico	<code>Math::randomInteger::parseInt</code>	<code>() -> Math.random()str -> Integer.parseInt(str)</code>
2. Metodo d'Istanza su Oggetto Esistente (Bound)	<code>System.out::printlnhexDigits::charAt</code>	<code>x -> System.out.println(x)i -> hexDigits.charAt(i)</code>
3. Metodo d'Istanza su Tipo Arbitrario (Unbound)	<code>String::toUpperCaseString::length</code>	<code>str -> str.toUpperCase()str -> str.length()</code>
4. Costruttore	<code>Person::newInteger[]::new</code>	<code>str -> new Person(str)size -> new Integer[size]</code>

17.1.4 La Stream API (java.util.stream) (Slide 16-18)

Uno `Stream<T>` è una **sequenza di elementi generata da una sorgente** che supporta operazioni aggregate di calcolo.

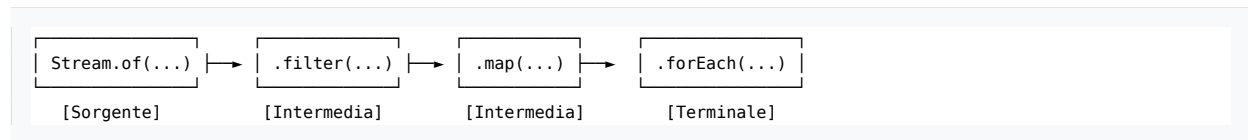
Le Tre Proprietà Chiave di uno Stream:

1. **Pipelining:** Le operazioni intermedie restituiscono un nuovo stream, permettendo la concatenazione fluente (*method chaining*).
2. **Internal Iteration:** A differenza delle collezioni dove lo sviluppatore controlla l'iterazione tramite `for/while` (*iterazione esterna*), lo stream gestisce il ciclo internamente. Ciò consente alla JVM di ottimizzare l'esecuzione e abilitare il parallelismo trasparente (`parallelStream()`).
3. **Lazy Evaluation (Valutazione Pigra):** Le operazioni intermedie non vengono eseguite quando vengono dichiarate; restano "in attesa". L'intera pipeline viene elaborata in un unico passaggio solo quando viene invocata un'operazione terminale!

17.1.5 Ciclo di Vita di una Pipeline Stream (Slide 18-24)

Una pipeline si compone tassativamente di 3 fasi:

Sorgente (Source) → Zero o più Operazioni Intermedie → Una Operazione Terminale



Creazione della Sorgente (Slide 19, 21-22)

- Da array: `Arrays.stream(arr)` o `Stream.of(v1, v2, v3)`.
- Da collezione: `list.stream()` o `set.stream()`.
- **Generazione infinita:**
 - `Stream.iterate(seed, UnaryOperator)`: applica iterativamente l'operatore a partire dal seme. Es: `Stream.iterate(0, i -> i + 1)` genera 0, 1, 2, 3, ...
 - `Stream.generate(Supplier)`: invoca continuamente il fornitore. Es: `Stream.generate(Math::random)`.
 - Per evitare loop infiniti, si arresta la generazione con l'operazione intermedia `.limit(n)`.

Operazioni Intermedie (Slide 23, 25-29, 34-36)

Restituiscono un nuovo `Stream<R>` e sono pigre (*lazy*):

- * `filter(Predicate<T>)`: trattiene solo gli elementi per cui il predicato è true.
- * `map(Function<T, R>)`: trasforma ciascun elemento $T \rightarrow R$ (mappatura $1 \rightarrow 1$).
- * `flatMap(Function<T, Stream<R>>)`: appiattisce strutture annidate (es. matrici `Integer[][]` o liste di liste) in un unico stream monodimensionale.
- * `distinct()`: elimina i duplicati (in base a `equals` e `hashCode`).
- * `sorted()` / `sorted(Comparator)`: ordina gli elementi.
- * `limit(n)`: tronca lo stream ai primi n elementi.
- * `skip(n)`: scarta i primi n elementi.

Operazioni Terminali (Slide 20, 23, 30-33)

Consumano lo stream e producono un risultato o un effetto collaterale:

- * **Chiusura su Collezioni / Array (Slide 20):** `stream.toList()`: raccoglie gli elementi in una `List` immutabile (Java 16+). `stream.collect(Collectors.toSet())`: raccoglie in un `Set`. `stream.toArray(Integer[]::new)`: esporta in array tipizzato.
- * **Iterazione / Consumo:** `stream.forEach(Consumer<T>)`.
- * **Conteggio / Riduzione:** `stream.count()`, `stream.reduce(...)`.
- * **Matching:** `anyMatch(Predicate)`, `allMatch(Predicate)`, `noneMatch(Predicate)`.
- * **Ricerca & optional<T> (Slide 31-32):** `findFirst()`, `findAny()`: restituiscono un `Optional<T>`, contenitore sicuro introdotto per debellare le `NullPointerException`.
- * **Metodi di Optional:** `isPresent()`, `get()`, `orElse(valoreDefault)`.

Attenzione / Trappola d'Esame

Regola Aurea degli Stream (Slide 24 — Domanda d'Esame): Uno Stream può essere consumato UNA SOLA VOLTA! Se si tenta di invocare una seconda operazione terminale sullo stesso oggetto Stream, la JVM solleva a runtime: `java.lang.IllegalStateException: stream has already been operated upon or closed`.

17.2 Analisi Dettagliata del Codice (Lezione17/MavenDate)

In `Lezione17/MavenDate`, il prof. Pasqua mantiene l'intera architettura pregressa e aggiunge il file `MainStream.java`, che esemplifica in modo magistrale tutte le funzionalità degli Stream appena studiate.

17.2.1 Analisi Riga per Riga di MainStream.java

```

1 package it.oop.ui;
2
3 import it.oop.core.AmericanDate;
4 import it.oop.core.FormattedDate;
5 import it.oop.core.ItalianDate;
6
7 import java.util.List;
8 import java.util.stream.Stream;
9 import it.oop.core.Date;
10 import static java.util.stream.Stream.generate;
11
12 public class MainStream {
13     public static void main(String[] args) {
14         // ESEMPIO 1: Generazione con iterate, filtro e limit
15         // Sequenza generata: "a", "aa", "aaa", "aaaa", ...
16         Stream<String> stream = Stream.iterate("a", s -> s + "a")
17             .filter(s -> s.length() % 2 == 1) // Tiene solo stringhe di lunghezza DISPARI
18             .limit(10);                      // Si arresta dopo le prime 10 stringhe dispari
19         List<String> list = stream.toList();   // Operazione terminale: unwrap a List
20         System.out.println(list.toString());
21
22         // ESEMPIO 2: Generazione con generate, Supplier e test di primalità
23         List<Integer> list2 = Stream.generate(() -> (int) (Math.random() * 20)) // Generatore
24             .filter(i -> isPrime(i)) // Filtra numeri
25             .limit(10) // Ne estrae
26             .toList(); // Operazione
27         System.out.println(list2.toString());
28
29         // ESEMPIO 3: Pipeline su oggetti di dominio del progetto (Date)
30         Stream.of(new ItalianDate(15,1,2025), new Date(16,1,2025), new ItalianDate(2,2,2025))
31             .filter(d -> d instanceof ItalianDate) // Filtra escludendo le Date generiche
32             .map(d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear())) // Converte in
33             .forEach(System.out::println); // Method reference per stampa su stdout
34     }
35
36     // Algoritmo di primalità implementato interamente con Stream!
37     private static boolean isPrime(int n) {
38         return n == 0 ? false : Stream.iterate(1, i -> i + 1)
39             .limit(n) // Genera i numeri da 1 a n
40             .map(i -> n % i) // Calcola il resto della divisione (0 indica divisore
41             .filter(i -> i == 0) // Mantiene solo i resti nulli (i divisori)
42             .count() <= 2; // Se i divisori sono <= 2 (cioè 1 e se stesso), è primo!
43     }
44 }

```

17.3 Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel progetto [Lezione17/MavenDate](#):

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainStream"
```

```
1 [a, aaa, aaaaa, aaaaaaa, aaaaaaaa, aaaaaaaaa, aaaaaaaaaa, aaaaaaaaaaa, aaaaaaaaaaaa, aaaaaaaaaaaaa,
2 aaaaaaaaaaaaaa]
3 [7, 11, 13, 19, 3, 13, 17, 5, 19, 1]
4 1/15/2025
5 2/2/2025
```

1. [a, aaa, aaaaa, ..., aaaaaaaaaaaaaaaaaa] → Lista delle prime 10 stringhe a lunghezza dispari (1, 3, 5, 7, 9, 11, 13, 15, 17, 19 caratteri).
2. [7, 11, 13, 19, 3, 13, 17, 5, 19, 1] → Dieci numeri primi casuali estratti nell'intervallo [0, 20) e validati dalla pipeline stream di `isPrime(int n)`.
3. 1/15/2025 e 2/2/2025 → Le due istanze di `ItalianDate` (15/1/2025 e 2/2/2025) superano il `filter(instanceof ItalianDate)`, vengono convertite tramite `map` in `AmericanDate` (formato M/D/Y) e stampate a video tramite `System.out::println`. L'istanza generica `new Date(16,1,2025)` viene invece scartata dal filtro.

Pagina 95 di 108

T19 — Documentazione con Javadoc e Unit Testing con JUnit 5

Convenzioni Javadoc, Architettura JUnit 5, Annotazioni di Ciclo di Vita ed Asserzioni

18.1 Analisi Teorica Approfondita (Slide T19)

La lezione 19 approfondisce due pratiche ingegneristiche irrinunciabili nello sviluppo professionale in Java: 1. **Documentazione automatizzata del codice** tramite lo standard **Javadoc**. 2. **Collaudo e Unit Testing automatizzato** tramite il framework **JUnit 5 (Jupiter)** e il plugin Maven **Surefire**.

18.1.1 Documentazione con Javadoc (Slide 3-11)

La documentazione del codice sorgente è essenziale per manutenibilità, collaborazione e pubblicazione di API riusabili. In Java la documentazione vive a stretto contatto con il codice nei cosiddetti **Doc Comments**:

- * **Sintassi**: Delimitati da `/**` all’inizio e `*/` alla fine. Ogni riga intermedia inizia convenzionalmente con `*`:
`java /** * Descrizione sintetica del componente. * <p>Supporta l'uso di tag HTML come paragrafi, link e formattazione.</p> */`
- * **I Tag Javadoc Standard (Slide 5-6)**: Iniziano con `@`, sono *case-sensitive* e devono comparire a inizio riga:
 - * `@param <nome>`: descrive un parametro formale di un metodo o costruttore.
 - * `@return`: descrive il significato del valore restituito da un metodo (omesso nei metodi `void` e nei costruttori).
 - * `@throws <Eccezione> (O @exception)`: descrive le condizioni anomale che causano il sollevamento di una determinata eccezione.
 - * `{@link <package.Classe#metodo>}`: genera un ipertesto cliccabile verso un’altra classe o metodo.
 - * `@author`: autore del modulo.
 - * `@since <versione>`: versione a partire dalla quale la feature è disponibile.
 - * `@version`: versione corrente del sorgente.
 - * `@deprecated`: avvisa che l’elemento è obsoleto, illustrandone il motivo e l’alternativa moderna da utilizzare.

Regole di Visibilità e Generazione (Slide 9-11)

- **Default di Javadoc**: Per impostazione predefinita, Javadoc genera la documentazione per le sole entità **public** e **protected** (l’interfaccia pubblica esposta ai client). I campi e metodi **private** o **package-private** vengono omessi.
- **Inclusione del privato**: Si deve passare esplicitamente il flag `-private` da riga di comando:

```
1 javadoc -private -d doc *
```

- **Integrazione Maven (maven-javadoc-plugin, Slide 11)**: Aggiungendo il plugin nel `pom.xml`, la documentazione viene generata con un singolo comando standard:

```
1 mvn javadoc:javadoc
```


18.1.2 Unit Testing con JUnit (Slide 13-16)

Il testing programmatico garantisce la correttezza delle singole unità software in isolamento (metodi o singole classi):

- * **Vantaggi sistemici:**
- * **Test-Driven Development (TDD):** scrivere i test prima ancora di implementare il codice applicativo.
- * **Regression Testing:** certezza che refactoring o nuove feature non rompano comportamenti pregressi funzionanti.
- * Riduzione drastica del tempo speso nel debugging manuale.
- * **Anatomia di un Test Case:** 1. Preparazione dell'input (*Arrange / Setup*). 2. Esecuzione del metodo sotto test (*Act*). 3. Confronto dell'output effettivo (*Actual*) con l'output atteso (*Expected*) tramite **asserzioni** (*Assert*).
- * **Meccanismo di Fallimento:**
- * Un metodo di test restituisce sempre `void`.
- * Se tutte le asserzioni sono soddisfatte, il metodo termina normalmente e JUnit lo contrassegna come **PASSED** (verde).
- * Se un'asserzione fallisce, solleva un errore di tipo `AssertionError` (o `AssertionFailedError`). Il framework JUnit cattura l'errore, contrassegna il test come **FAILED** (rosso) con il report della discrepanza, e **prosegue regolarmente con i test successivi** senza arrestare l'intera suite.

18.1.3 Asserzioni in JUnit 5 (`org.junit.jupiter.api.Assertions`, Slide 17-21)

Il pacchetto Jupiter standardizza una vasta famiglia di metodi statici:

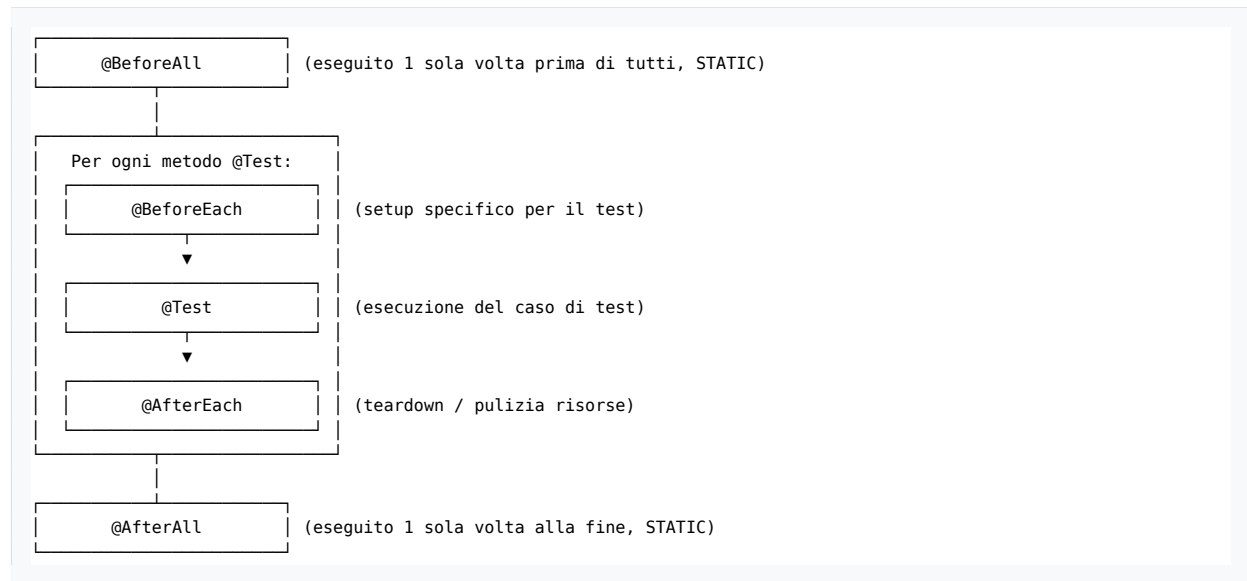
Metodo Asserzione	Comportamento
<code>assertEquals(expected, actual, [msg])</code>	Verifica che <code>expected.equals(actual)</code> . Valido per primitivi e oggetti.
<code>assertTrue(condition, [msg])</code>	Verifica che la condizione booleana sia <code>true</code> .
<code>assertFalse(condition, [msg])</code>	Verifica che la condizione booleana sia <code>false</code> .
<code>assertArrayEquals(expected, actual)</code>	Confronta il contenuto e l'ordinamento di due array elemento per elemento.
<code>assertNull(obj) / assertNotNull(obj)</code>	Verifica se il puntatore è <code>null</code> o non nullo.
<code>assertThrows(Exception.class, executable)</code>	Verifica che l'esecuzione di una lambda/blocco sollevi tassativamente l'eccezione attesa!

Best Practice: Static Imports (Slide 21)

Per evitare di anteporre continuamente `Assertions.` davanti a ogni chiamata, si usa l'import statico:

```
1 import static org.junit.jupiter.api.Assertions.assertEquals;
2 import static org.junit.jupiter.api.Assertions.assertThrows;
```

18.1.4 Il Ciclo di Vita del Test e le Annotazioni JUnit 5 (Slide 22)



- `@BeforeAll`: Inizializzazione “pesante” o condivisa tra tutti i test (es. apertura connessione, setup di un database o di costanti immutabili). **Deve essere static.**
- `@AfterAll`: Chiusura finale delle risorse globali. **Deve essere static.**
- `@BeforeEach`: Inizializzazione fresca prima di *ciascun* test per garantire l’isolamento e l’indipendenza dei test (evitando che un test modifichi lo stato di un altro).
- `@AfterEach`: Pulizia post-test (es. cancellazione di file temporanei creati durante il test).
- `@DisplayName("Descrizione chiara")`: Personalizza il nome del test visualizzato nei report o nell’IDE.
- `@Disabled`: Disabilita temporaneamente il test senza doverlo commentare o cancellare.

18.1.5 Organizzazione del Progetto e Convenzioni Maven (Slide 23-24)

Maven e i moderni build tool impongono una struttura standard di cartelle: * `src/main/java`: Contiene il codice sorgente dell’applicazione (es. `it.oop.core.Date`). * `src/test/java`: Contiene esclusivamente le classi di collaudo. * **Allineamento dei Package (Regola Fondamentale)**: Una classe di test che verifica `it.oop.core.Date` **deve risiedere nello stesso identico package** `it.oop.core` (all’interno di `src/test/java`). In questo modo la classe di test ha accesso non solo ai membri `public`, ma anche a tutti i metodi e campi con visibilità di **package** (**package-private**), facilitando il testing interno senza forzare l’apertura a `public` di dettagli architetturali riservati!

18.2 Analisi Dettagliata del Codice (Lezione18/MavenDate)

In `Lezione18/MavenDate` troviamo la configurazione completa del plugin Maven Surefire e due classi di test che collaudano la gerarchia di `Date`.

18.2.1 Configurazione in `pom.xml`

```
1 <build>
2   <plugins>
3     <!-- Plugin Javadoc (Slide 11) -->
4     <plugin>
```

```

5         <groupId>org.apache.maven.plugins</groupId>
6         <artifactId>maven-javadoc-plugin</artifactId>
7         <version>3.6.2</version>
8         <configuration>
9             <source>1.8</source>
10            <show>private</show> <!-- Include metodi e campi privati -->
11        </configuration>
12    </plugin>
13</plugins>
14</build>
15
16<dependencies>
17    <!-- Motore di esecuzione JUnit 5 (Slide 24) -->
18    <dependency>
19        <groupId>org.junit.jupiter</groupId>
20        <artifactId>junit-jupiter-engine</artifactId>
21        <version>5.10.0</version>
22        <scope>test</scope> <!-- Visibile solo durante la fase di test -->
23    </dependency>
24    <!-- Maven Surefire Plugin per l'esecuzione di 'mvn test' -->
25    <dependency>
26        <groupId>org.apache.maven.plugins</groupId>
27        <artifactId>maven-surefire-plugin</artifactId>
28        <version>3.5.4</version>
29    </dependency>
30</dependencies>

```

18.2.2 TestDate.java

Questa classe collauda sia il flusso nominale (costruzione valida) che il flusso eccezionale (lancio di eccezioni):

```

1 package it.oop.core;
2
3 import it.oop.exception.IllegalDateException;
4 import org.junit.jupiter.api.Assertions;
5 import org.junit.jupiter.api.Test;
6 import static org.junit.jupiter.api.Assertions.assertThrows;
7
8 public class TestDate {
9
10     private Date date;
11
12     // 1. Collaudo del caso nominale con assertEquals
13     @Test
14     public void testDateConstructor() {
15         date = new Date(1, 12, 2025);
16         Assertions.assertEquals(1, date.getDay());
17         Assertions.assertEquals(12, date.getMonth());
18         Assertions.assertEquals(2025, date.getYear());
19     }
20
21     // 2. Collaudo del caso eccezionale con assertThrows e Lambda
22     @Test
23     public void testDateConstructorException() {
24         // Verifica che passando un anno negativo (-2), il costruttore sollevi tassativamente
25         // IllegalDateException
26         assertThrows(IllegalDateException.class, () -> date = new Date(1, 12, -2));
27     }
28 }

```

18.2.3 TestItalianDate.java

Esemplifica l'uso della fixture `@BeforeAll` e mette in luce il superamento della vecchia parola chiave `assert`:

```
1 package it.oop.core;
2
3 import org.junit.jupiter.api.Assertions;
4 import org.junit.jupiter.api.BeforeAll;
5 import org.junit.jupiter.api.Test;
6
7 public class TestItalianDate {
8     private static ItalianDate date;
9
10    // Fixture statica eseguita una volta sola prima di tutti i test
11    @BeforeAll
12    public static void setup() {
13        date = new ItalianDate(1, 1, 1970);
14    }
15
16    @Test
17    public void printFormatTest() {
18        // Verifica del formato con JUnit 5:
19        Assertions.assertEquals("dd/mm/yyyy", date.printFormat());
20
21        // Confronto con la vecchia sintassi 'assert' del linguaggio (commentata dal docente):
22        // assert date.printFormat().equals("dd/mm/yyyy") : "wrong format";
23    }
24 }
```

18.3 Risultato di Esecuzione Reale da Terminale

Comando eseguito nel progetto [Lezione18/MavenDate](#):

```
1 mvn test
```

Output ottenuto:

```
1 [INFO] -----
2 [INFO]  T E S T S
3 [INFO] -----
4 [INFO] Running it.oop.core.TestDate
5 [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s -- in
   it.oop.core.TestDate
6 [INFO] Running it.oop.core.TestItalianDate
7 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s -- in
   it.oop.core.TestItalianDate
8 [INFO]
9 [INFO] Results:
10 [INFO]
11 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
12 [INFO]
13 [INFO] -----
14 [INFO] BUILD SUCCESS
15 [INFO] -----
```

I 3 test sono stati eseguiti con successo, validando in modo formale la suite del progetto e generando i relativi file di report XML e TXT nella cartella `target/surefire-reports/`.

Dimmi “vai” per procedere con il **Blocco 19** (Lezione 19 / Slide T20 — *Java I/O*, stream di byte vs caratteri, `InputStream`, `OutputStream`, `Reader`, `Writer`, `BufferedReader`, serializzazione, gestione delle risorse con `try-with-resources` e analisi di `MainI0.java` in `Lezione19/MavenDate`).

T20 — Java I/O, Streams, File, NIO.2 e Serializzazione

Byte Streams vs Character Streams, Bufferizzazione, Decorator Pattern I/O e Serializzazione

19.1 Analisi Teorica Approfondita (Slide T20)

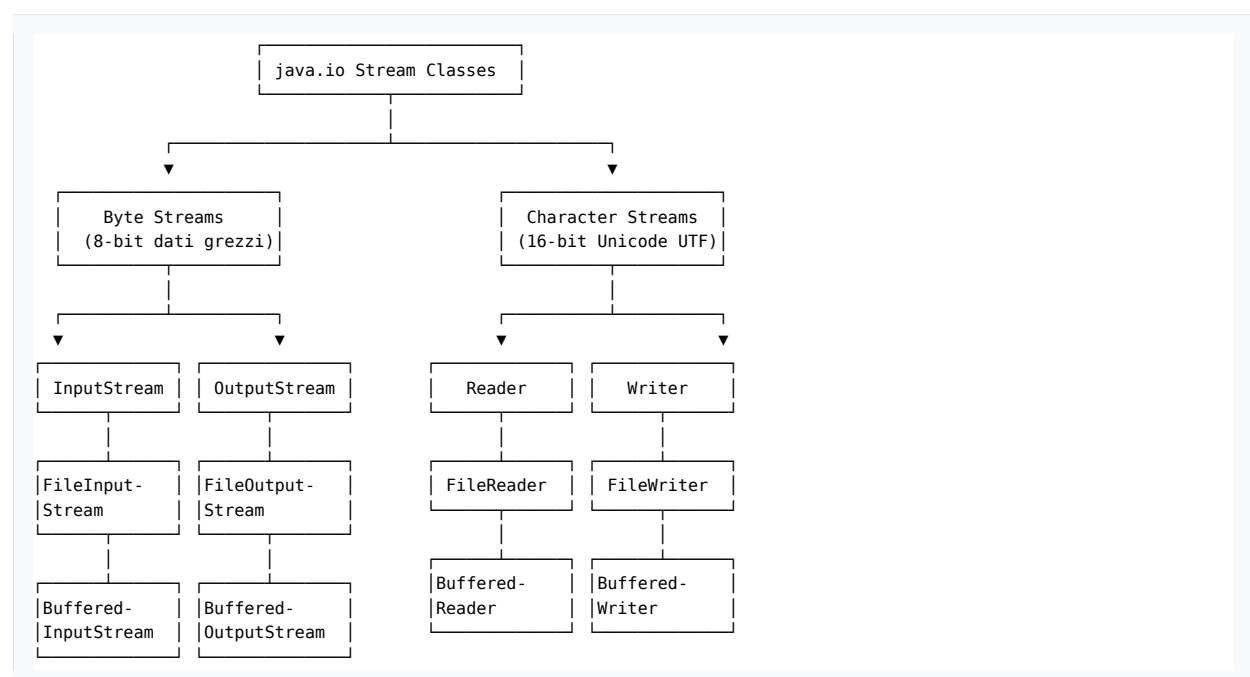
La lezione 20 conclude il percorso teorico del corso esplorando il sottosistema di **Input/Output (I/O)** di Java (`java.io` e `java.net`), la persistenza su disco, la comunicazione di rete, e la gestione di formati dati strutturati moderni (**CSV** e **JSON**) tramite librerie terze gestite con Maven.

19.1.1 L'Astrazione di I/O Stream (Slide 3-4)

In Java, qualsiasi trasferimento di dati da o verso l'esterno è modellato tramite l'astrazione di **I/O Stream** (*flusso sequenziale unidirezionale di dati*): * **Differenza tassonomica cruciale:** Gli **I/O Streams** (`java.io`) non vanno confusi con gli **Stream funzionali** di Java 8 (`java.util.stream.Stream`). Gli I/O Stream trasportano byte o caratteri fisici verso periferiche o file; gli Stream di Java 8 elaborano pipeline di trasformazione di oggetti in memoria. * Un I/O Stream può essere agganciato a: * File su disco. * Flussi standard di processo: `System.in` (standard input), `System.out` (standard output), `System.err` (standard error). * Connessioni di rete (*socket* TCP/IP o endpoint HTTP). * Buffer di memoria (array di byte o stringhe).

19.1.2 Il Dualismo dell'I/O in Java: Byte Streams vs Character Streams (Slide 4-9)

L'architettura di `java.io` è rigorosamente bipartita in due gerarchie parallele:



Byte Streams (InputStream e OutputStream, Slide 5-7)

- **Unità di dato:** Singolo byte grezzo (8 bit, intervallo [0, 255] restituito come `int`, dove il valore `-1` segnala la fine dello stream — *End Of File, EOF*).
- **Destinazione d'uso:** Immagini, file multimediali, bytecode compilato (`.class`), file compressi (`.zip`), comunicazioni binarie di rete.
- **Metodi cardine:**
 - `int read()`: legge il prossimo byte (o `-1` a fine stream).
 - `int read(byte[] b)`: riempie il buffer di byte.
 - `void write(int b)`: scrive un byte.
 - `void write(byte[] b)`: scrive un intero array di byte.
 - `void close()`: rilascia il descrittore del file del sistema operativo.
- **Implementazioni notevoli:**
 - `FileInputStream` / `FileOutputStream`: lettura/scrittura diretta su disco.
 - `BufferedInputStream` / `BufferedOutputStream`: aggiunge un buffer in memoria per minimizzare le costose chiamate di sistema del kernel del SO.
 - `DataInputStream` / `DataOutputStream`: serializzazione di primitivi Java (`readInt()`, `writeDouble()`).

Character Streams (Reader e Writer, Slide 8-10)

- **Unità di dato:** Caratteri Unicode (16 bit UTF-16).
- **Destinazione d'uso:** Esclusivamente file di testo, file sorgente, documenti testuali.
- **Le classi ponte (Bridge Adapters):**
 - `InputStreamReader`: converte un `InputStream` di byte in un `Reader` di caratteri applicando una specifica codifica (es. UTF-8).
 - `OutputStreamWriter`: converte caratteri in byte da inviare a un `OutputStream`.
- **Implementazioni notevoli:**
 - `FileReader` / `FileWriter`: lettura/scrittura di file di testo.
 - `BufferedReader`: legge blocchi di testo bufferizzati e fornisce il comodissimo metodo `String readLine()` (restituisce un'intera riga o `null` a fine file).
 - `BufferedWriter`: scrittura testuale con supporto al metodo `newLine()`.

19.1.3 Gestione delle Risorse: Da try-finally a try-with-resources (Slide 7, 10-11)**Il Vecchio Approccio (Pre-Java 7, Slide 10):**

```

1  BufferedReader br = null;
2  try {
3      br = new BufferedReader(new FileReader("i.txt"));
4      // lettura...
5  } catch (IOException e) {
6      // gestione errore...
7  } finally {
8      if (br != null) {
9          try {
10             br.close(); // Ulteriore try-catch obbligatorio perché close() lancia IOException!
11             } catch (IOException e) { ... }
12      }
13  }

```

Questo pattern era verbose, faticoso da mantenere e fonte continua di *resource leaks* (descrittori di file aperti non chiusi se si verificavano eccezioni nel `finally`).

L'Approccio Moderno: `try-with-resources` (Java 7+, Slide 11)

Tutte le classi che implementano l'interfaccia standard `java.lang.AutoCloseable` possono essere dichiarate all'interno delle parentesi tonde del blocco `try`:

```
1 try (BufferedReader br = new BufferedReader(new FileReader("i.txt"));
2     BufferedWriter bw = new BufferedWriter(new FileWriter("o.txt"))) {
3     String line;
4     while ((line = br.readLine()) != null) {
5         bw.write(line);
6         bw.newLine();
7     }
8 } catch (FileNotFoundException fnfe) {
9     System.out.println("File not found...");
10 } catch (IOException ioe) {
11     System.out.println("I/O Error...");
12 }
```

- **Garanzie della JVM:**

1. Le risorse vengono chiuse **automaticamente** non appena il blocco `try` termina, sia in caso di completamento regolare sia in caso di eccezione o `return` anticipato.
2. Vengono chiuse in **ordine inverso** rispetto alla loro dichiarazione (prima `bw`, poi `br`).
3. Se sia il corpo del `try` che la chiamata a `close()` sollevano eccezioni, l'eccezione del corpo è quella principale propagata, mentre l'eccezione di chiusura viene salvata come **Suppressed Exception** (recuperabile con `e.getSuppressed()`).

19.1.4 Gestione di File e Risorse di Rete (URL/URI) (Slide 13-17)

- **La classe `java.io.File` (Slide 13-14):** Rappresenta il percorso astratto di un file o di una directory. Non legge né scrive dati direttamente, ma permette di interrogare e modificare il filesystem:

- `exists()`, `isFile()`, `isDirectory()`, `length()`, `getAbsolutePath()`.
- `mkdir()`, `delete()`, `renameTo(File dest)`, `listFiles()`.

- **URL e URI (`java.net`, Slide 15-17):**

- URI modella l'identificatore formale (*RFC 2396*).
- URL modella la locazione fisica e il protocollo per accedere alla risorsa web.
- *Avvertenza contemporanea (Slide 16):* Il costruttore `new URL("...")` è **deprecato** da Java 20. La prassi moderna impone di creare un'istanza di `URI` e convertirla:

```
1 URL url = new URI("https://info.cern.ch/index.html").toURL();
2 InputStream in = url.openStream(); // Apre una connessione HTTP e scarica i dati
```

19.1.5 Dati Strutturati: CSV e JSON (Slide 18-26)

Java non possiede parser nativi integrati nel runtime per file CSV e JSON. Per questi formati si ricorre a librerie esterne integrate tramite dipendenze Maven.

File CSV con OpenCSV (Slide 19-22)

- Un file CSV (*Comma-Separated Values*) memorizza tabelle in formato testuale, separando le colonne con virgole o punti e virgola.
- **Dipendenza Maven:** `com.opencsv:opencsv:5.12.0`.
- **Scrittura:** `CSVWriter` scrive array di stringhe `String[]` gestendo automaticamente apici ed escape:

```
1 CSVWriter csvw = new CSVWriter(new FileWriter("data.csv"));
2 csvw.writeNext(new String[]{ "id", "name", "address" });
```

- **Lettura:** `CSVReader` con `readNext()` riga per riga o `readAll()`.

File JSON con Google Gson (Slide 23-26)

- JSON (*JavaScript Object Notation*) è lo standard dominante per lo scambio dati basato su mappe chiave-valore `{}` e liste ordinate `[]`.
- **Dipendenza Maven:** `com.google.code.gson:gson:2.13.2`.
- **Serializzazione e Deserializzazione Automatica (Data Binding):**

```
1 Gson gson = new Gson();
2
3 // 1. Oggetto Java -> Stringa JSON (Serializzazione)
4 Date date = new Date(1, 1, 1970);
5 String json = gson.toJson(date); // Restituisce '{"day":1,"month":1,"year":1970}'
6
7 // 2. Stringa JSON -> Oggetto Java (Deserializzazione)
8 Date d = gson.fromJson(json, Date.class); // Ricostruisce l'istanza valorizzando i campi!
```

Gson accede ai campi (anche private e final) tramite **Reflection**, senza richiedere getter/setter pubblici o costruttori senza argomenti!

- **Pretty Printing:** `new GsonBuilder().setPrettyPrinting().create()` formatta il JSON con ritorni a capo e indentazione a 2 spazi.

19.2 Analisi Dettagliata del Codice (Lezione19/MavenDate)

In `Lezione19/MavenDate` il prof. Pasqua integra tutti i concetti della Slide T20 all'interno della classe dimostrativa `MainIO.java`.

19.2.1 Configurazione delle Dipendenze in `pom.xml`

```
1 <!-- OpenCSV per parsing e generazione CSV (Slide 20) -->
2 <dependency>
3   <groupId>com.opencsv</groupId>
4   <artifactId>opencsv</artifactId>
5   <version>5.12.0</version>
6 </dependency>
7 <!-- Google Gson per serializzazione/deserializzazione JSON (Slide 24) -->
8 <dependency>
9   <groupId>com.google.code.gson</groupId>
10  <artifactId>gson</artifactId>
11  <version>2.13.2</version>
12 </dependency>
```

19.2.2 Analisi Riga per Riga di MainIO.java

Lettura Binaria di Bytecode con FileInputStream e FileChannel (Righe 18-41)

```

1      FileInputStream fis = null;
2      try {
3          // Apertura dello stream di byte su un file binario compilato (.class)
4          fis = new FileInputStream("src/main/resources/Date.class");
5          FileChannel fc = fis.getChannel();
6          int b;
7          // Lettura byte a byte
8          while ((b = fis.read()) != -1) {
9              System.out.println((char) b);
10         }
11         // Rewind dello stream riportando la posizione a 0 tramite FileChannel
12         fc.position(0);
13         StringBuilder sb = new StringBuilder();
14         // Lettura dell'intero contenuto in un colpo solo con readAllBytes() (Java 9+)
15         for (byte bb : fis.readAllBytes()) {
16             sb.append((char) bb);
17         }
18         System.out.println("-----");
19         System.out.println(sb.toString());
20     } catch (IOException ioe) {
21         System.out.println(ioe.getMessage());
22     } finally {
23         // Chiusura classica pre-Java 7 con try-catch protetto
24         try {
25             if (fis != null) fis.close();
26         } catch (IOException ioe) {
27             System.out.println(ioe.getMessage());
28         }
29     }

```

Scrittura di Testo con FileWriter e try-with-resources (Righe 42-50)

```

1      // Blocco try-with-resources: fw viene chiuso automaticamente
2      try (FileWriter fw = new FileWriter("src/main/resources/file.txt")) {
3          StringBuilder sb = new StringBuilder();
4          // Genera stringhe di 'a' crescenti usando Stream di Java 8
5          Stream.iterate("a", s -> s + "a")
6              .limit(15)
7              .forEach(s -> sb.append(s).append("\n"));
8          fw.write(sb.toString());
9      } catch (IOException ioe) {
10         System.out.println(ioe.getMessage());
11     }

```

Generazione di Tabella CSV con OpenCSV (Righe 52-65)

```

1      try (CSVWriter csvw = new CSVWriter(new FileWriter("src/main/resources/data.csv"))) {
2          String[] row = { "id", "name", "address" };
3          csvw.writeNext(row); // Scrive l'intestazione delle colonne
4
5          row[0] = "3";
6          row[1] = "Paul";
7          row[2] = "Strada le Grazie, 15";

```

```

8         csvw.writeNext(row);
9
10        row[0] = "6";
11        row[1] = "Sam";
12        row[2] = "Strada le Grazie, 18";
13        csvw.writeNext(row);
14    } catch (IOException ioe) {
15        System.out.println(ioe.getMessage());
16    }

```

Serializzazione e Deserializzazione con Gson (Righe 66-80)

```

1    Gson gson = new Gson();
2    Date date = new Date(1, 1, 1970);
3
4    // Serializzazione dell'oggetto Date in formato JSON
5    String json = gson.toJson(date);
6    System.out.println(json); // Stampa: {"day":1,"month":1,"year":1970}
7
8    // Deserializzazione da stringa JSON a nuova istanza Date
9    Date date1 = gson.fromJson("{\"day\":1,\"month\":1,\"year\":1971}", Date.class);
10   System.out.println(date1.toString()); // Stampa: y1971m1d1
11
12   // Scrittura su file con Pretty Printing abilitato
13   try (FileWriter fw = new FileWriter("src/main/resources/file.json")) {
14       new GsonBuilder().setPrettyPrinting().create().toJson(date, fw);
15   } catch (IOException ioe) {
16       System.out.println(ioe.getMessage());
17   }

```

19.3 Risultato di Compilazione ed Esecuzione Reale

Comando eseguito nel progetto [Lezione19/MavenDate](#):

```
1 mvn compile exec:java -Dexec.mainClass="it.oop.ui.MainIO"
```

Output ottenuto a video:

```

1 [Dumping del bytecode del file Date.class con header CAFEBABE e constant pool]
2 -----
3 {"day":1,"month":1,"year":1970}
4 y1971m1d1
5 [INFO] BUILD SUCCESS

```

19.3.1 Ispezione dei file generati su disco:

1. src/main/resources/file.txt:

```

1 a
2 aa
3 aaa
4 ...
5 aaaaaaaaaaaaaa

```

2. src/main/resources/data.csv:

```
1 "id","name","address"
2 "3","Paul","Strada le Grazie, 15"
3 "6","Sam","Strada le Grazie, 18"
```

3. src/main/resources/file.json:

```
1 {
2   "day": 1,
3   "month": 1,
4   "year": 1970
5 }
```

19.4 Trapiuardo Raggiunto: Revisione Integrale del Corso Comple-tata!

Abbiamo completato con successo e senza tralasciare alcun dettaglio: * Tutte le **20 slide teoriche** (da τ01 a τ20). * Tutti i **progetti e le lezioni pratiche** (da Lezione04 a Lezione19). * Tutti i diagrammi, meme, pattern architetturali, finezze di linguaggio, bug del docente, test Maven e logiche di compilazione ed esecuzione.

Fammi sapere se desideri approfondire specifici temi per l'esame, affrontare vecchi temi d'esame scritti/pratici, o dedicarti agli esercizi del tuo workspace (esercizi-wally-25-26)!